

43. Serial Peripheral Interface (SPI)

This is the SPI_B version of the SPI peripheral module.

SPI_B is referred to as SPI in this chapter.

43.1 Overview

The Serial Peripheral Interface (SPI) has 2 channels. The SPI provides high-speed full-duplex synchronous serial communications with multiple processors and peripheral devices. [Table 43.1](#) lists the SPI specifications, [Figure 43.1](#) shows a block diagram of SPI, [Figure 43.2](#) shows a clock source selector block diagram and [Table 43.2](#) lists the I/O pins.

In this section, PCLK refers to PCLKA.

Table 43.1 SPI specifications (1 of 2)

Parameter	Specifications
Number of channels	Two channels
SPI transfer functions	- Use of MOSI (master out/slave in), MISO (master in/slave out), SSL (slave select), and RSPCK (SPI clock) signals allows serial communications through SPI operation (4-wire method) or clock synchronous operation (3-wire method) - Transmit-only operation available - Receive-only operation is available - Communication mode selectable to full-duplex, transmit-only or receive-only - RSPCK polarity switching - RSPCK phase switching
Data format	- MSB-first or LSB-first selectable - Transfer bit length selectable from 4 to 32 bits 32 bit × 4 stages FIFO is available as transmit buffer or receive buffer - Byte swap operating function - Transmit/receive data can be inverted.
Operation clock (TCLK)	Synchronized clock (PCLK) or independent clock (SPICKL) can be selected.
Bit rate	- In master mode, the on-chip baud rate generator generates RSPCK by frequency-dividing TCLK (the division ratio ranges from divided by 2 to divided by 4096) - In slave mode, the minimum TCLK clock divided by 2 can be input as RSPCK (TCLK divided by 2 is the maximum RSPCK frequency) Width at high level: 1 TCLK cycle; width at low level: 1 TCLK cycle
Buffer configuration	Double buffer configuration for the transmit and receive buffers
Error detection	- Mode fault error detection - Underrun error detection - Overrun error detection^{*1} - Parity error detection - Receive data ready detection
SSL control function	[motorola SPI mode/TI SSP mode common] - Four SSL pins (SSLn: SSLn0 to SSLn3) (n = A, B) for each channel - In single-master mode, SSLn0 to SSLn3 pins are output - In multi-master mode, SSLn0 pin for input, and SSLn1 to SSLn3 pins either for output or unused - In slave mode, SSLn0 pin for input and SSLn1 to SSLn3 pins unused - Controllable wait for next-access SSL output assertion (next-access delay) Range: 1 to 8 RSPCK cycles (set in RSPCK-cycle units) - Function for changing SSL polarity - Delay between frames in burst transfer is settable [only Motorola mode] - Controllable delay from RSPCK stop to SSL output negation (SSL negation delay) Range: 1 to 8 RSPCK cycles (set in RSPCK-cycle units) - Controllable delay from SSL output assertion to RSPCK operation (RSPCK delay) Range: 1 to 8 RSPCK cycles (set in RSPCK-cycle units) [only TI SSP mode] - Controllable delay from RSPCK stop to SSL output negation (SSL negation delay) Range: 0 to 8 RSPCK cycles (set in RSPCK-cycle units) - Controllable delay from OE output assertion to RSPCK operation (RSPCK delay) Range: 1 to 8 RSPCK cycles (set in RSPCK-cycle units)
Communication protocol	- Motorola SPI - TI SSP (Synchronous Serial Protocol)

Table 43.1 SPI specifications (2 of 2)

Parameter	Specifications
Synchronization bypass function	Synchronization circuit can be bypassed only when the same clock is input to bus clock (PCLK) and operation clock (TCLK).
Control in master transfer	• Transfers of up to eight commands each can be executed sequentially in looped execution • For each command, the following can be set: SSL signal value, bit rate, RSPCK polarity and phase, transfer data length, MSB- or LSB-first, burst, RSPCK delay, SSL negation delay, and next-access delay • Transfers can be initiated by writing to the transmit buffer • MOSI signal value specifiable in SSL negation • RSPCK auto-stop function
Interrupt sources	Interrupt sources: • Receive buffer full / Receive data ready interrupt • Transmit buffer empty interrupt • SPI error interrupt (mode fault error, under run error, over run error, parity error, receive data ready) • SPI idle interrupt (SPI idle) • Communication end interrupt
Event link function	The following events can be output to the Event Link Controller (ELC): • Receive buffer full / receive data ready signal • Transmit buffer empty signal • Mode fault, underrun, overrun, parity error, or receive data ready signal • SPI idle signal • Communication end signal
Other functions	• Switching between CMOS output and open-drain output • SPI initialization function • Loopback mode • SPE bit status polling function
Module-stop function	Module-stop state can be set for each channel to reduce power consumption.
TrustZone Filter	Security and Privilege attribution can be set for each channel.

Note 1. In master reception and when the RSPCK auto-stop function is enabled, an overrun error does not occur because the transfer clock is stopped on overrun error detection.

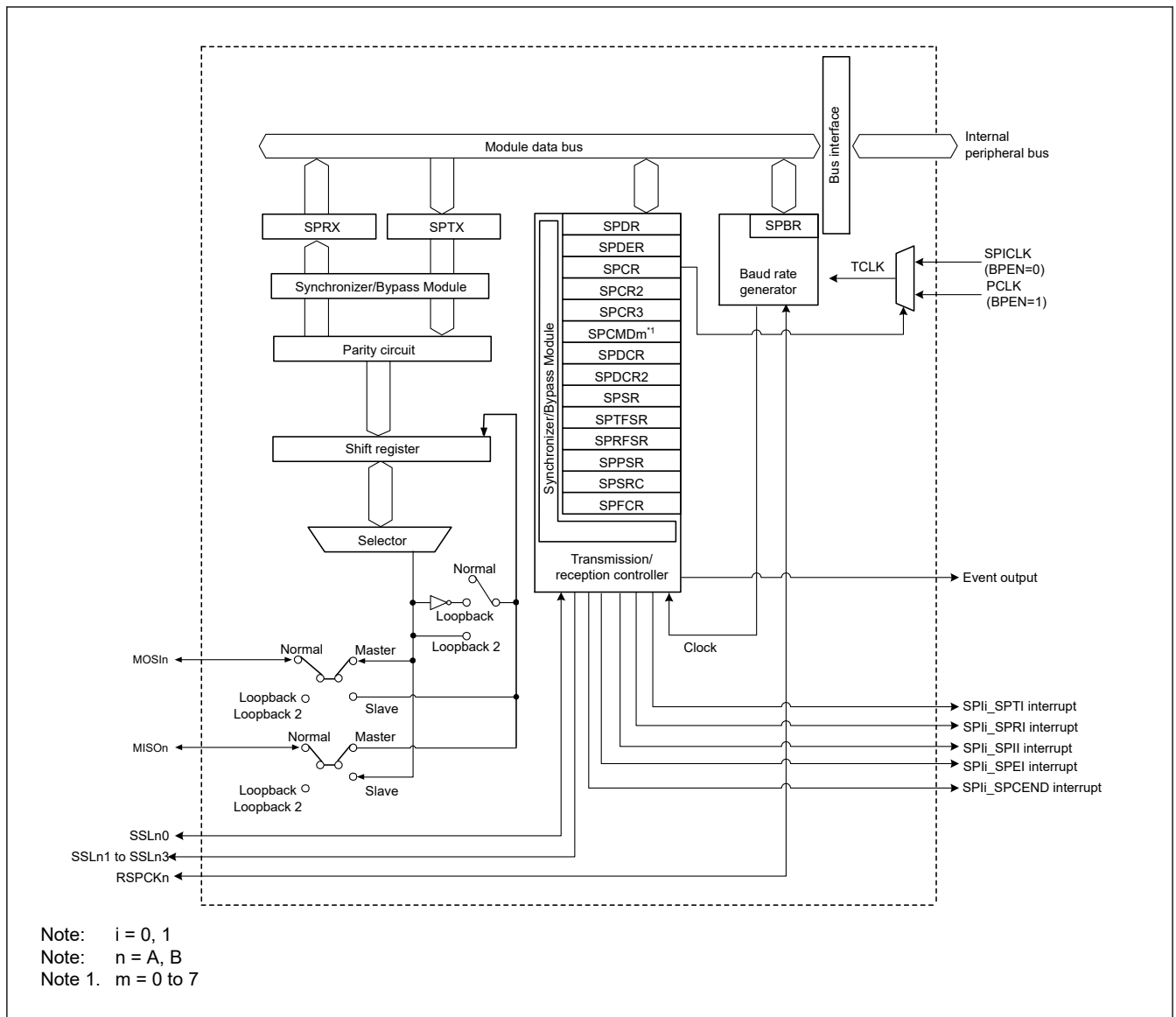
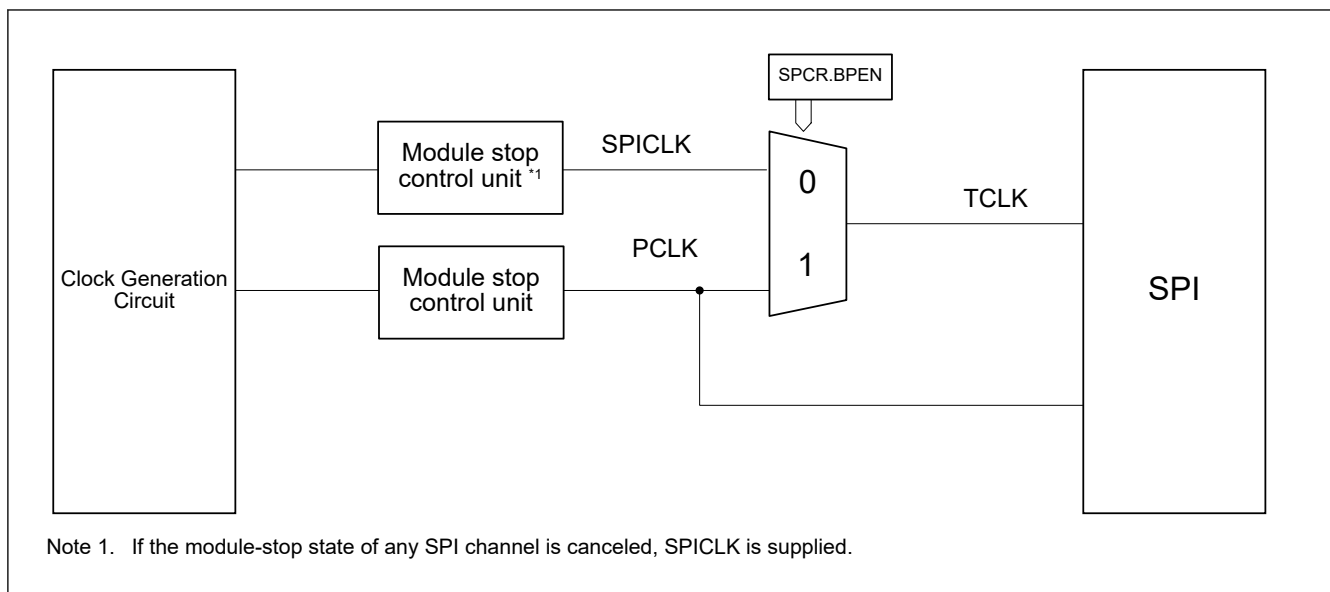


Figure 43.1 SPI block diagram

**Figure 43.2** Clock source selector block diagram

The SPI automatically switches the I/O direction of the SSLn0 pin. SSLn0 is set as an output when the SPI is a single master, and as an input when the SPI is a multi-master or a slave. The RSPCKn, MOSIn, and MISOn pins are automatically set as inputs or outputs based on the master or slave setting and the level input on the SSLn0 pin. For details, see [section 43.3.2. Controlling the SPI Pins](#).

Table 43.2 SPI I/O pins

Channel	Pin name	I/O	Description
SPI0	RSPCKA	I/O	Clock input/output pin
	SSLA0	I/O	Slave selection input/output
	SSLA1 to SSLA3	Output	Slave selection output
	MOSIA	I/O	Master transmit data input/output
	MISOA	I/O	Slave transmit data input/output
SPI1	RSPCKB	I/O	Clock input/output pin
	SSLB0	I/O	Slave selection input/output
	SSLB1 to SSLB3	Output	Slave selection output
	MOSIB	I/O	Master transmit data input/output
	MISOB	I/O	Slave transmit data input/output

Note: Pin names are indicated as "...A" or "...An" for SPI0, and "...B" or "...Bn" for SPI1 (n = 0, 1, 2, or 3).

43.2 Register Descriptions

43.2.1 SPDR : SPI Data Register

Base address: $\text{SPIn_B} = 0x4035_C000 + 0x0100 \times n$ ($n = 0, 1$)
 $\text{SPIn_B_NS} = 0x5035_C000 + 0x0100 \times n$ ($n = 0, 1$)

Offset address: 0x00

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	SPD[31:16]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	SPD[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
31:0	SPD[31:0]	These bits are the interface with the buffers that hold data for transmission and reception by the SPI.	R/W

Note: S-TYPE-3, P-TYPE-3

SPDR is the interface with the buffers that hold data for transmission and reception by the SPI. When accessing this register in words, access SPD_R. The transmit buffer (SPTX) and receive buffer (SPRX) are independent, but both are mapped to SPD_R. Figure 43.3 shows the configuration of the SPD_R register.

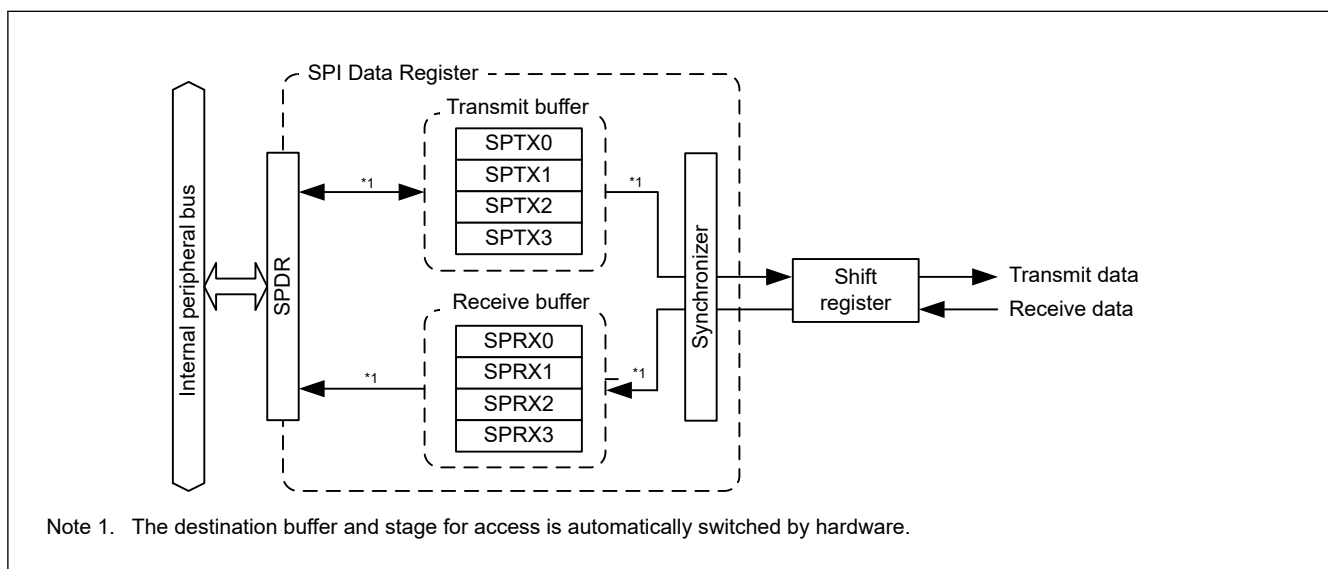


Figure 43.3 Structure of SPDR

32 bits × 4 stage transmit FIFO and 32 bits × 4 stage receive FIFO are provided. These 8 stage FIFO are mapped to one address in the SPD_R. Transmit buffers (SPTX_n, $n = 0$ to 3) can be written by writing data to SPD_R to transmit written data.

Upon completion of receiving data, receive buffers store received data. When an overrun error occurs, data in the receive buffer is not updated.

(1) Bus Interface

The SPI data register has 32 bits × 4 stage transmit FIFO and 32 bits × 4 stage receive FIFO (32 bytes in total). These 32 bytes are mapped to the 4-byte space of SPD_R. Write transmit data from the LSB. Received data is stored from the LSB.

SPD_R register write operation and read operation are described below.

1. Write

A transmit buffer write pointer is provided for transmit buffers. When data is written to SPDR, the pointer automatically switches to the next buffer. The following illustrates the structure of the transmit buffer bus interface (write).

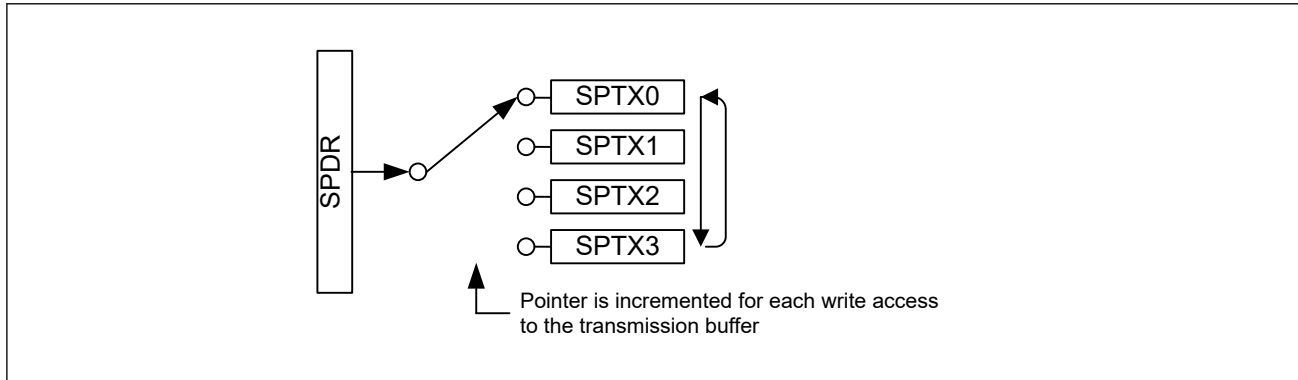


Figure 43.4 Structure of SPDR (Write)

The transmit buffer (SPTX0 to SPTX3) switching order:

SPTX0→SPTX1→SPTX2→SPTX3→SPTX0→SPTX1→...

When writing transmit data to transmit buffers (SPTX_n), write transmit data of frames +1 specified by the Transmission FIFO threshold setting bits of SPI data control register 2 (SPDCR2.TTRG[1:0]) while an SPI transmit buffer empty interrupt is present (SPSR.SPTEF flag = 1). Writing to the transmit buffer (SPTX_n, n = 0 to 3) in the state where there is no empty stage in the transmit FIFO does not update the buffer value.

2. Read

Values can be read from receive buffers (SPRX_n, n = 0 to 3) or transmit buffers (SPTX_n, n = 0 to 3) by reading the SPDR register. Reading a receive buffer or reading a transmit buffer can be selected by the SPI receive data or transmit data select bit (SPDCR.SPRDTD) in the SPI data control register.

The SPDR register is read according to the independent receive buffer read pointer and the transmit buffer read pointer. The following illustrates the structure of the receive buffer and transmit buffer bus interface (read).

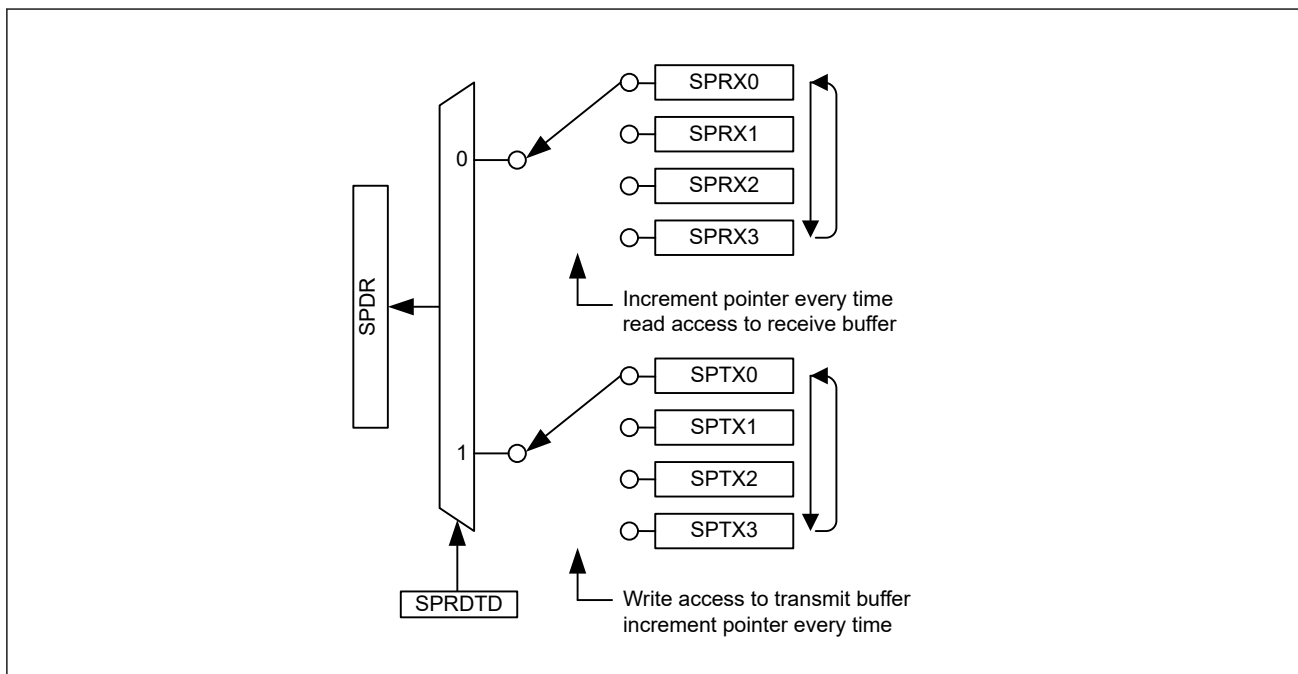


Figure 43.5 Structure of SPDR (Read)

When a receive buffer is read, the receive buffer read pointer automatically switches to the next buffer. The receive buffer read pointer switches in the same order as the transmit buffer write pointer.

The transmit buffer read pointer is updated during the SPDR write access, but it is not updated during the transmit buffer read access. When a transmit buffer is read, the value written to SPDR last can be read.

43.2.2 SPDECR : SPI Delay Control Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
 SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x04

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	SPNDL[2:0]		
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	SLNDL[2:0]			—	—	—	—	—	SCKDL[2:0]		
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
2:0	SCKDL[2:0]	RSPCK Delay 0 0 0: 1RSPCK 0 0 1: 2RSPCK 0 1 0: 3RSPCK 0 1 1: 4RSPCK 1 0 0: 5RSPCK 1 0 1: 6RSPCK 1 1 0: 7RSPCK 1 1 1: 8RSPCK	R/W
7:3	—	These bits are read as 0. The write value should be 0.	R/W
10:8	SLNDL[2:0]	SSL Negation Delay [Master Mode] 0 0 0: 1RSPCK 0 0 1: 2RSPCK 0 1 0: 3RSPCK 0 1 1: 4RSPCK 1 0 0: 5RSPCK 1 0 1: 6RSPCK 1 1 0: 7RSPCK 1 1 1: 8RSPCK [Motorola-SPI case in Slave Mode] 0 0 0: 1RSPCK Others: Setting prohibited [TI-SSP case in Slave Mode] 0 0 0: 1 TCLK 0 0 1: 2 TCLK 0 1 0: 3 TCLK 0 1 1: 4 TCLK 1 0 0: 5 TCLK 1 0 1: 6 TCLK 1 1 0: 7 TCLK 1 1 1: 8 TCLK	R/W
15:11	—	These bits are read as 0. The write value should be 0.	R/W
18:16	SPNDL[2:0]	SPI Next-Access Delay 0 0 0: 1RSPCK + 5TCLK 0 0 1: 2RSPCK + 5TCLK 0 1 0: 3RSPCK + 5TCLK 0 1 1: 4RSPCK + 5TCLK 1 0 0: 5RSPCK + 5TCLK 1 0 1: 6RSPCK + 5TCLK 1 1 0: 7RSPCK + 5TCLK 1 1 1: 8RSPCK + 5TCLK	R/W
31:19	—	These bits are read as 0. The write value should be 0.	R/W

Note: S-TYPE-3, P-TYPE-3

SCKDL[2:0] bit (RSPCK Delay)

[In the Motorola-SPI case]

The RSPCK delay bits (SCKDL) are used to set the period (RSPCK delay) from SSL signal assertion start until RSPCK oscillates while the SCKDEN bit in the SPI command register (SPCMD) is 1. If SCKDL is modified while the MSTR bit and the SPE bit in the SPI control register (SPCR) are 1, subsequent operation is not guaranteed.

To use the SPI in slave mode, set SCKDL[2:0] bits to 000b.

[In the TI-SSP case]

The RSPCK delay bits (SCKDL) are used to set the period (RSPCK delay) from SSL signal assertion start until RSPCK oscillates while the SCKDEN bit in the SPI command register (SPCMD) is 1. Also that is used to set the period until the SSL signal is negated. If SCKDL is modified while the MSTR bit and the SPE bit in the SPI control register (SPCR) are 1, subsequent operation is not guaranteed.

To use the SPI in slave mode, set SCKDL[2:0] bits to 000b.

SLNDL[2:0] bit (SSL Negation Delay)

[In the Motorola-SPI case]

The SSL negation delay bits (SLNDL) are used to set the period (SSL negation delay) after the SPI in master mode sends the final RSPCK edge during serial transfer until it negates the SSL signal while the SLNDEN bit in the SPI command register (SPCMD) is 1. If SLNDL is modified while the MSTR bit and the SPE bit in the SPI control register (SPCR) are 1, subsequent operation is not guaranteed.

To use the SPI in slave mode except TI-SSP, set SLNDL[2:0] bits to 000b.

[In the TI-SSP case]

The SSL negation delay bits (SLNDL) are used to set the period (OE negation delay) after the SPI in master mode sends the final RSPCK edge during serial transfer until it negates the OE signal while the SLNDEN bit in the SPI command register (SPCMD) is 1. Also, that is used to set the period from when the SPI in slave mode detects the last RSPCK edge of serial transfer to when the OE signal is negated. If SLNDL is modified while the SPE bit in the SPI control register (SPCR) are 1, subsequent operation is not guaranteed.

SPNDL[2:0] bit (SPI Next-Access Delay)

The SPI next-access delay register (SPDECR.SPNDL) is used to set the SSL signal inactive period (next-access delay) after completion of serial transfer while the SPNDEN bit in the SPI command register (SPCMD) is 1. If SPNDL is modified while the MSTR bit and the SPE bit in the SPI control register (SPCR) are 1, subsequent operation is not guaranteed.

These bits are used to set the next-access delay value when the SPNDEN bit in SPCMD is 1. To use the SPI in slave mode, set SPNDL[2:0] bits to 000b.

43.2.3 SPCR : SPI Control Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x08

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	BPEN	MSTR	TXMD[1:0]		—	—	SPFR F	SPMS	—	—	CENDI E	SPTIE	SPDR ES	SPIIE	SPRIE	SPEIE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	MODF EN	BDFS	SCKA SE	PTE	—	SPOE	SPPE	—	—	—	—	—	—	—	SPE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	SPE	SPI Function Enable 0: SPI function is disabled. 1: SPI function is enabled.	R/W
7:1	—	These bits are read as 0. The write value should be 0.	R/W
8	SPPE	Parity Enable 0: A parity bit is not added to transmit data. Received-data parity check is not performed. 1: A parity bit is added to transmit data. Received-data parity check is performed.	R/W
9	SPOE	Parity Mode 0: Even parity is used for transmission and reception. 1: Odd parity is used for transmission and reception.	R/W
10	—	This bit is read as 0. The write value should be 0.	R/W
11	PTE	Parity Self-Diagnosis Enable 0: Parity circuit self-diagnosis function is disabled. 1: Parity circuit self-diagnosis function is enabled.	R/W
12	SCKASE	RSPCK Auto-Stop Function Enable 0: RSPCK auto-stop function is disabled. 1: RSPCK auto-stop function is enabled.	R/W
13	BFDS	Between Burst Transfer Frames Delay Select 0: Delay (RSPCK delay, SSL negation delay and next-access delay) between frames is inserted in burst transfer 1: Delay between frames is not inserted in burst transfer.	R/W
14	MODFEN	Mode Fault Error Detection Enable 0: Mode fault error detection is disabled. 1: Mode fault error detection is enabled.	R/W
15	—	This bit is read as 0. The write value should be 0.	R/W
16	SPEIE	SPI Error Interrupt Enable 0: SPI error interrupt request is disabled. 1: SPI error interrupt request is enabled.	R/W
17	SPRIE	SPI Receive Buffer Full Interrupt Enable 0: SPI receive buffer full interrupt request is disabled. 1: SPI receive buffer full interrupt request is enabled.	R/W
18	SPIIE	SPI Idle Interrupt Enable 0: Idle interrupt request is disabled. 1: Idle interrupt request is enabled.	R/W
19	SPDRES	SPI receive data ready error select Select the interrupt request to be generated when the reception data ready is detected 0: Receive data full interrupt 1: Error interrupt	R/W
20	SPTIE	SPI Transmit Buffer Empty Interrupt Enable 0: SPI transmit buffer empty interrupt request is disabled. 1: SPI transmit buffer empty interrupt request is enabled.	R/W
21	CENDIE	SPI Communication End Interrupt Enable 0: Communication end interrupt request is disabled. 1: Communication end interrupt request is enabled.	R/W
23:22	—	These bits are read as 0. The write value should be 0.	R/W
24	SPMS	SPI Mode Select 0: SPI operation (4-wire) 1: Clock synchronous operation (3-wire)	R/W
25	SPFRF	SPI Frame Format Select 0: Motorola-SPI 1: TI-SSP Note: When SPMS = 1 (clock synchronous operation (3-wire)), this bit setting is invalid.	R/W

Bit	Symbol	Function	R/W
27:26	—	These bits are read as 0. The write value should be 0.	R/W
29:28	TXMD[1:0]	Communication Mode Select 0 0: Transmit-Receive 0 1: Transmit only Others: Receive only	R/W
30	MSTR	SPI Master/Slave Mode Select 0: Slave mode 1: Master mode	R/W
31	BPEN	Synchronization Circuit Bypass Enable 0: Non-Bypass 1: Bypass	R/W

Note: S-TYPE-3, P-TYPE-3

The SPI control register (SPCR) is used to set operating mode of the SPI. If the set BPEN, MSTR, TXMD[1:0], SPFRF, SPMS, MODFEN, BFDS, SCKASE, PTE, SPOE, SPPE bit value is modified while the SPE bit = 1, subsequent operation is not guaranteed.

SPE bit (SPI Function Enable)

This bit is used to enable or disable SPI functions. Setting this bit to 1 enables SPI functions. When the MODF flag in the SPI status register (SPSR) is 1, the SPE bit is cleared to 0 and the SPE bit cannot be set to 1 until the MODF flag is cleared to 0. (See [section 43.3.10. Error Detection](#)) Setting the SPE bit to 0 disables SPI functions and initializes a part of module functions. (See [section 43.3.11. Initializing the SPI](#))

SPPE bit (Parity Enable)

This bit is used to enable or disable the parity function.

SPOE bit (Parity Mode)

This bit is used to specify even parity or odd parity.

In even parity mode, the parity bit is determined so that the sum of 1 (parity bit + transmit characters or receive characters) becomes an even number. In the same way, in odd parity mode, a parity bit is determined so that the sum of 1 (parity bit + transmit characters or receive characters) becomes an odd number. The SPOE bit is valid only when the SPPE bit in SPCR is set to 1.

PTE bit (Parity Self-Diagnosis Enable)

This bit is used to enable or disable self-diagnosis of the parity circuit to confirm that the parity function is normal.

SCKASE bit (RSPCK Auto-Stop Function Enable)

This bit is used to enable or disable the RSPCK auto-stop function. When this function is enabled, the RSPCK clock stops immediately before an overrun error occurs during data reception in master mode. For details, see [section 43.3.10.1. Overrun Errors](#).

BFDS bit (Between Burst Transfer Frames Delay Select)

This bit controls whether insert the delay time between the burst transfer frames.

Valid in the master mode (SPCR.MSTR = 1) for frames with the SPCMDn.SSLKP bit set to 1.

This bit should be set to 0 in slave mode. The usage of SSL delay control between transfer frames is shown as below. For details, see [section 43.3.12.1. Master Mode Operation](#).

1. Non-burst transmits
2. Burst transmit with delay between frames
 - 2-1. From the 1st frame to the last previous frame
 - 2-2. The last frame
3. Burst transmit with no delay between frames
 - 3-1. From the 1st frame to the last previous frame

3-2. The last frame

Table 43.3 Usage of SSL delay control between transfer frames (Master mode)

	SPCMDn. SSLKP bit	SPCR.BF DS bit	SSL delay control register*1 (RSPCK clock delay, SSL negation delay, next access delay)
1	0	0	Any given value. You can control each delay value according to setting for RSPCK clock delay, SSL negation delay and next access delay
2-1	1	0	
2-2	0	0	
3-1	1	1	Any given value. But delay is inserted only below. - RSPCK clock delay of the 1st frame - SSL negation delay and next access delay of the last frame
3-2	0	1	

Note 1. Whether the setting value of following bits are valid or not depends on the setting value of the SPCMD.SPNDEN bit. (See [section 43.2.6. SPCMDm : SPI Command Register \(m = 0 to 7\).](#))
The SPDECR.SCKDL[2:0] bits: RSPCK delay
The SPDECR.SLNDL[2:0] bits: SSL negate delay
The SPDECR.SPNDL[2:0] bits: Next access delay

< Setting / operation example > (Motorola SPI, BFDS = 1 Case)

SPCMD0.SSLKP = 1 → Burst transfer / no interframe delay between 0 and 1 (SSL keep active)

SPCMD1.SSLKP = 1 → Burst transfer / no interframe delay between 1 and 2 (SSL keep active)

SPCMD2.SSLKP = 1 → Burst transfer / no interframe delay between 2 and 3 (SSL keep active)

SPCMD3.SSLKP = 1 → Burst transfer / no interframe delay between 3 and 4 (SSL keep active)

SPCMD4.SSLKP = 0 → do not Burst Transfer, and inactive SSL. (BFDS setting is invalid because it does not Burst Transfer.)

SPCMD5.SSLKP = 1 → Burst transfer / no interframe delay between 5 and 6 (SSL keep active)

SPCMD6.SSLKP = 1 → Burst transfer / no interframe delay between 6 and 7 (SSL keep active)

SPCMD7.SSLKP = 0 → do not Burst Transfer, and inactive SSL. (BFDS setting is invalid because it does not Burst Transfer.)

MODFEN bit (Mode Fault Error Detection Enable)

This bit is used to enable or disable detection of a mode fault error. (See [section 43.3.10. Error Detection.](#)) The SPI determines SSL0 pin input or output direction according to the combination of the MODFEN and MSTR bits. (See [section 43.3.2. Controlling the SPI Pins.](#))

SPEIE bit (SPI Error Interrupt Enable)

This bit is used to enable or disable an SPI error interrupt request when the SPI detects a mode fault error or an underrun error and sets the MODF flag in the SPI status register (SPSR) to 1, when the SPI detects an overrun error and sets the OVRF flag in SPSR to 1, or when the SPI detects a parity error and sets the PERF flag in SPSR to 1. (See [section 43.3.10. Error Detection](#))

SPRIE bit (SPI Receive Buffer Full Interrupt Enable)

This bit is used to enable or disable a receive buffer full interrupt request of the SPI.

SPIIE bit (SPI Idle Interrupt Enable)

This bit is used to enable or disable an idle interrupt request of the SPI after the SPI detects the idle state and sets the IDLNF flag in the SPI status register (SPSR) to 0.

SPDRES bit (SPI receive data ready error select)

When a receive data ready is detected (SPSR.SPDRF = 1), select whether to use SPIi_SPRI (i = 0, 1) interrupt request or SPIi_SPEI (i = 0, 1) interrupt request.

SPTIE bit (SPI Transmit Buffer Empty Interrupt Enable)

This bit is used to enable or disable a transmit buffer empty interrupt request of the SPI.

A transmit buffer empty interrupt request at the beginning of transmission is generated by setting the SPE bit to 1 simultaneously when or after the SPTIE bit is set to 1. Note that a transmit buffer empty interrupt is generated while the SPTIE bit is 1 even though SPI functions are disabled (SPE bit = 0).

CENDIE bit (SPI Communication End Interrupt Enable)

This bit controls generation of a communication end interrupt request.

SPMS bit (SPI Mode Select)

This bit is used to select SPI operation (4-wire) or clock synchronous operation (3-wire).

For clock synchronous operation, the SSL pin is not used but three pins RSPCK, MOSI, and MISO are used for communication. When SPMS = 1 (clock synchronous operation (3-wire)), the setting of the SPFRF bit is invalid.

To perform clock synchronous operation in master mode (SPCR.MSTR = 1), set the CPHA bit in the SPI command register (SPCMD) to 0 or 1. To perform clock synchronous operation in slave mode (SPCR.MSTR = 0), set the CPHA bit to 1. If this bit is set to 0 for clock synchronous operation in slave mode (SPCR.MSTR = 0), subsequent operation is not guaranteed.

The communication status according to the settings of the MSTR bit, TXMD[1:0] bits, SPFRF bit, and SPMS bit of the SPI control register (SPCR) as follows.

Table 43.4 SPI Communication Status

SPCR. MSTR	SPCR.T XMD[1]	SPCR.T XMD[0]	SPCR. SPFRF	SPCR. SPMS	Communication Status	Communication Status No
1	0	0	0	0	Transmit-Receive Master / Motorola SPI / SPI operation (4-wire)	1-(1)
1	0	0	1	0	Transmit-Receive Master / TI-SSP / SPI operation (4-wire)	1-(2)
1	0	0	—	1	Transmit-Receive Master / Clock synchronous operation (3-wire)	1-(3)
1	0	1	0	0	Transmit only Master / Motorola SPI / SPI operation (4-wire)	1-(4)
1	0	1	1	0	Transmit only Master / TI-SSP / SPI operation (4-wire)	1-(5)
1	0	1	—	1	Transmit only Master / Clock synchronous operation (3-wire)	1-(6)
1	1	—	0	0	Receive only Master / Motorola SPI / SPI operation (4-wire)	1-(7)
1	1	—	1	0	Receive only Master / TI-SSP / SPI operation (4-wire)	1-(8)
1	1	—	—	1	Receive only Master / Clock synchronous operation (3-wire)	1-(9)
0	0	0	0	0	Transmit-Receive Slave / Motorola SPI / SPI operation (4-wire) (default)	0-(1)
0	0	0	1	0	Transmit-Receive Slave / TI-SSP / SPI operation (4-wire)	0-(2)
0	0	0	—	1	Transmit-Receive Slave / Clock synchronous operation (3-wire)	0-(3)
0	0	1	0	0	Transmit only Slave / Motorola SPI / SPI operation (4-wire)	0-(4)
0	0	1	1	0	Transmit only Slave / TI-SSP / SPI operation (4-wire)	0-(5)
0	0	1	—	1	Transmit only Slave / Clock synchronous operation (3-wire)	0-(6)
0	1	—	0	0	Receive only Slave / Motorola SPI / SPI operation (4-wire)	0-(7)
0	1	—	1	0	Receive only Slave / TI-SSP / SPI operation (4-wire)	0-(8)
0	1	—	—	1	Receive only Slave / Clock synchronous operation (3-wire)	0-(9)

SPFRF bit (SPI Frame Format Select)

This bit selects the communication protocol.

The format of the SPI terminal (RSPCK, SSL0 to 7) can be set according to the set communication protocol.

When SPMS = 1 (clock synchronous operation (3-wire)), this bit is invalid because SSL is not used.

TXMD[1:0] bit (Communication Mode Select)

This bit is used to select the transmit-receive, transmit-only, and receive-only serial communication.

When TXMD[1:0] is set to 01 for communication, transmit-only is performed without reception.

When TXMD[1] is set to 1 for communication, receive-only is performed without transmission.

When TXMD[1:0] is set to 01 for communication, a receive buffer full interrupt request cannot be used.

When TXMD[1] is set to 1 for communication, a transmit buffer empty interrupt request cannot be used.

(See [section 43.3.6. Communication Operating Mode.](#))

MSTR bit (SPI Master/Slave Mode Select)

This bit is used to select master mode or slave mode of the SPI. The SPI determines input/output directions of pins RSPCK, MOSI, MISO, and SSL1 to SSL3 according to the MSTR bit setting.

BPEN bit (Synchronization Circuit Bypass Enable)

This bit selects whether to enable or disable the synchronization bypass function. Set this bit to 1 when the bus clock (PCLK) is also used as operation clock (TCLK), see [Figure 43.2.](#)

43.2.4 SPCR2 : SPI Control Register 2

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x0C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	MOIFE	MOIFV	—	—	SPLP2	SPLP
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	SPDRC[7:0]								RMST TG	RMED TG	—	RMFM[4:0]				
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
4:0	RMFM[4:0]	Frame processing count setting in Master Receive only The number of received frames can be adjusted in master receive only. 0x00: This function is not used*1 0x01: Automatically stop communication after processing 1 received frame ⋮ 0x1F: Automatically stop communication after processing 31 received frames	R/W
5	—	This bit is read as 0. The write value should be 0.	R/W
6	RMEDTG	End Trigger in Master Receive only 1: Receive End (Writable only when Master Receive only) Reading value is always 0	W
7	RMSTTG	Start Trigger in Master Receive only 1: Receive Start (Writable only when Master Receive only) Reading value is always 0	W
15:8	SPDRC[7:0]	SPI received data ready detect adjustment 0x00: Disable receive data ready detection function 0x01: Performs reception data ready judgment after 1 TCLK ⋮ 0xFF: Performs reception data ready judgment after 255 TCLK	R/W
16	SPLP	SPI Loopback 0: Normal mode 1: Loopback mode (inverted transmit data = receive data)	R/W
17	SPLP2	SPI Loopback 2 0: Normal mode 1: Loopback mode (transmit data = receive data)	R/W

Bit	Symbol	Function	R/W
19:18	—	These bits are read as 0. The write value should be 0.	R/W
20	MOIFV	MOSI Idle Fixed Value 0: The fixed value of MOSI idle = 0. 1: The fixed value of MOSI idle = 1.	R/W
21	MOIFE	MOSI Idle Fixed Value Enable 0: The MOSI output value is the last data of previous transfer. 1: The MOSI output value is the set MOIFV bit value.	R/W
31:22	—	These bits are read as 0. The write value should be 0.	R/W

Note: S-TYPE-3, P-TYPE-3

Note 1. See SW flow in the [section 43.3.12.1. Master Mode Operation](#).

RMFM[4:0] bit (Frame processing count setting in Master Receive only)

The number of received frames can be adjusted when operating in master receive only. Valid only when the master mode (SPCR.MSTR = 1) and the communication operation mode select bits (SPCR.TXMD [1:0]) are 10b.

Only the start bit in master mode reception automatically stops communication after starts frame processing according to the value set in this bit after reception starts.

If the RMFM [4:0] bits are rewritten while the SPE bit of the SPI control register (SPCR) is 1, subsequent operations are not guaranteed.

RMEDTG bit (End Trigger in Master Receive only)

This bit is used to end reception when master receive only. Valid only when the master mode (SPCR.MSTR = 1) and the communication mode select bits (SPCR.TXMD [1:0]) are 10b.

RMSTTG bit (Start Trigger in Master Receive only)

This bit is used to start reception when master receive only. Valid only when the master mode (SPCR.MSTR = 1) and the communication mode select bits (SPCR.TXMD [1:0]) are 10b.

Writing 1 to this bit during reception is not accepted. Write again after reception is completed.

SPDRC[7:0] bit (SPI received data ready detect adjustment)

The receive data ready detection function can be disabled or, if used, the period until detection can be set from 1 to 255 TCLK.

The value set in the SPDRC [7:0] bits is used to 1 set the SPDRF flag. For details, see the description of SPDRF in [section 43.2.9. SPSR : SPI Status Register](#).

If the set value is changed while the SPE bit is 1, subsequent operations are not guaranteed.

SPLP bit (SPI Loopback)

When the SPLP bit is set to 1, the SPI shuts down the route between the MISO pin and the shift register (when the MSTR bit in the SPI control register is 1) or shuts down the route between the MOSI pin and the shift register, inverts the input route value in the shift register, and then connects the route to the output route (when the MSTR bit in the SPI control register is 0) (loopback mode).

SPLP2 bit (SPI Loopback 2)

When the SPLP2 bit is set to 1, the SPI shuts down the route between the MISO pin and the shift register (when the MSTR bit in the SPI control register is 1) or shuts down the route between MOSI pin and the shift register and then connects the route to the output route without inverting the input route value in the shift register (when the MSTR bit in the SPI control register is 0) (loopback mode). If this bit is set to 1 together with the SPLP bit, setting this bit takes precedence.

MOIFV bit (MOSI Idle Fixed Value)

This bit is used to select the MOSI pin output value during the SSL negation period (including SSL retention period in burst transfer) when the MOIFE bit is 1 in master mode.

If this bit is modified with the SPE bit in the SPI control register (SPCR) set to 1, subsequent operation is not guaranteed.

MOIFE bit (MOSI Idle Fixed Value Enable)

This bit is used for the SPI in master mode to fix the MOSI output value during the SSL negation period (including SSL retention period in burst transfer). When MOIFE bit = 0, the SPI outputs the last data of the previous serial transfer to MOSI during the SSL negation period. When MOIFE bit = 1, the SPI outputs the fixed MOIFV bit value to MOSI.

If this bit is modified with the SPE bit in the SPI control register (SPCR) set to 1, subsequent operation is not guaranteed.

43.2.5 SPCR3 : SPI Control Register 3

Base address: $\text{SPIn_B} = 0x4035_C000 + 0x0100 \times n$ ($n = 0, 1$)
 $\text{SPIn_B_NS} = 0x5035_C000 + 0x0100 \times n$ ($n = 0, 1$)

Offset address: 0x10

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	SPSLN[2:0]			—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	SPBR[7:0]								—	—	—	—	SSL3P	SSL2P	SSL1P	SSL0P
Value after reset:	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	SSL0P	SSL0 Signal Polarity [In the Motorola-SPI case] 0: The SSL0 signal is active low. 1: The SSL0 signal is active high. [In the TI-SSP case] 0: The SSL0 signal is active high. 1: The SSL0 signal is active low.	R/W
1	SSL1P	SSL1 Signal Polarity [In the Motorola-SPI case] 0: The SSL1 signal is active low. 1: The SSL1 signal is active high. [In the TI-SSP case] 0: The SSL1 signal is active high. 1: The SSL1 signal is active low.	R/W
2	SSL2P	SSL2 Signal Polarity [In the Motorola-SPI case] 0: The SSL2 signal is active low. 1: The SSL2 signal is active high. [In the TI-SSP case] 0: The SSL2 signal is active high. 1: The SSL2 signal is active low.	R/W
3	SSL3P	SSL3 Signal Polarity [In the Motorola-SPI case] 0: The SSL3 signal is active low. 1: The SSL3 signal is active high. [In the TI-SSP case] 0: The SSL3 signal is active high. 1: The SSL3 signal is active low.	R/W
7:4	—	These bits are read as 0. The write value should be 0.	R/W
15:8	SPBR[7:0]	SPI Bit Rate	R/W
23:16	—	These bits are read as 0. The write value should be 0.	R/W

Bit	Symbol	Function	R/W
26:24	SPSLN[2:0]	SPI Sequence Length 0 0 0: Sequence Length is 1 (Referenced SPCMDn, n = 0→0→...) 0 0 1: Sequence Length is 2 (Referenced SPCMDn, n = 0→1→0→...) 0 1 0: Sequence Length is 3 (Referenced SPCMDn, n = 0→1→2→0→...) 0 1 1: Sequence Length is 4 (Referenced SPCMDn, n = 0→1→2→3→0→...) 1 0 0: Sequence Length is 5 (Referenced SPCMDn, n = 0→1→2→3→4→0→...) 1 0 1: Sequence Length is 6 (Referenced SPCMDn, n = 0→1→2→3→4→5→0→...) 1 1 0: Sequence Length is 7 (Referenced SPCMDn, n = 0→1→2→3→4→5→6→0→...) 1 1 1: Sequence Length is 8 (Referenced SPCMDn, n = 0→1→2→3→4→5→6→7→0→...)	R/W
31:27	—	These bits are read as 0. The write value should be 0.	R/W

Note: S-TYPE-3, P-TYPE-3

SSLiP bits (SSL Signal Polarity Bits)

These bits are used to specify the polarity of SSL signals. The set SSLiP bit (i = 0 to 3) values indicate the active polarity of SSLi signals.

If any of these SSLiP bits is modified with the SPE bit in the SPI control register (SPCR) set to 1, subsequent operation is not guaranteed.

Note: SSL0 is different from SSL1-SSL3. When slave or multi-master, it functions as an input.

For details, see [section 43.3.3.2. Single-master/Single-slave with the MCU as a Slave](#), and [section 43.3.3.5. Multi-master/Multi-slave with the MCU as a Master](#).

SPBR[7:0] bit (SPI Bit Rate)

The SPI bit rate bits (SPBR) is used to set the bit rate in master mode. If SPBR is modified while the MSTR bit in the SPI control register (SPCR) is 1, subsequent operation is not guaranteed.

When the SPI is used in slave mode, the bit rate depends on the input clock bit rate regardless of the SPCMD.BRDV setting. (Specify a bit rate that meets electrical characteristics.)

The bit rate is determined by a combination of the set SPBR value and the set BRDV[1:0] bits value in the SPI command register (SPCMD0 to SPCMD7).

The bit rate is calculated by the following expression, where n is the set SPBR value (0 to 255) and N is the set BRDV[1:0] bits value (0 to 3).

$$\text{Bit rate} = \frac{f(\text{TCLK})}{2 \times (n + 1) \times 2^N}$$

The following table shows an example of correspondence between bit rates and set values of SPBR and BRDV[1:0].

Table 43.5 Corresponding Between Bit Rates and Set Values (Example)

SPBR Value (n)	BRDV Value (N)	Division Ratio	Bit Rate				
			TCLK = 32 MHz	TCLK = 36 MHz	TCLK = 40 MHz	TCLK = 50 MHz	TCLK = 120MHz
0	0	2	16.0 Mbps	18.0 Mbps	20.0 Mbps	25.0 Mbps	60.0Mbps
1	0	4	8.00 Mbps	9.00 Mbps	10.0 Mbps	12.5 Mbps	30.0Mbps
2	0	6	5.33 Mbps	6.00 Mbps	6.67 Mbps	8.33 Mbps	20.0Mbps
3	0	8	4.00 Mbps	4.50 Mbps	5.00 Mbps	6.25 Mbps	15.0Mbps
4	0	10	3.20 Mbps	3.60 Mbps	4.00 Mbps	5.00 Mbps	12.0Mbps
5	0	12	2.67 Mbps	3.00 Mbps	3.33 Mbps	4.16 Mbps	10.0Mbps
5	1	24	1.33 Mbps	1.50 Mbps	1.67 Mbps	2.08 Mbps	5.0Mbps
5	2	48	677 kbps	750 kbps	833 kbps	1.04 Mbps	2.5Mbps
5	3	96	333 kbps	375 kbps	417 kbps	521 kbps	1.25Mbps
255	3	4096	7.81 kbps	8.80 kbps	9.78 kbps	12.2 kbps	29.3kbps

SPSLN[2:0] bit (SPI Sequence Length)

These bits are used to set the sequence length for the SPI in master mode to perform sequence operation. According to the sequence length specified by SPSLN[2:0] bits, the SPI in master mode changes SPI command registers 0 to 7 (SPCMD0 to SPCMD7) to be referenced and the reference sequence. For details, see [section 43.3.13.1. Master Mode Operation](#).

The SPI in slave mode always references SPCMD0.

43.2.6 SPCMDm : SPI Command Register (m = 0 to 7)

Base address: SPln_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPln_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x14 + 0x04 × m

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	SSLA[2:0]				—	—	—	SPB[4:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	SCKD EN	SLND EN	SPND EN	LSBF	—	—	—	—	SSLK P	—	—	—	BRDV[1:0]		CPOL	CPHA
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	CPHA	RSPCK Phase 0: Data is sampled at an odd edge and changes at an even edge. 1: Data changes at an odd edge and is sampled at an even edge.	R/W
1	CPOL	RSPCK Polarity 0: RSPCK in idle state is 0. 1: RSPCK in idle state is 1.	R/W
3:2	BRDV[1:0]	Bit Rate Division 0 0: Base bit rate 0 1: Base bit rate divided by 2 1 0: Base bit rate divided by 4 1 1: Base bit rate divided by 8	R/W
6:4	—	These bits are read as 0. The write value should be 0.	R/W
7	SSLKP	SSL Signal Level Hold 0: All SSL signals are negated at the end of transfer. 1: SSL signal level is held after the transfer ends until the next access starts.	R/W
11:8	—	These bits are read as 0. The write value should be 0.	R/W
12	LSBF	SPI LSB First 0: MSB first 1: LSB first	R/W
13	SPNDEN	SPI Next-Access Delay Enable 0: Next-access delay is 1RSPCK + 5TCLK 1: Next-access delay is the set value of the SPI next-access delay (SPDECR.SPNDL).	R/W
14	SLNDEN	SSL Negation Delay Setting Enable 0: [Master] SSL negation delay is 1RSPCK. [Slave in the TI-SSP] SSL negation delay is 1TCLK 1: SSL negation delay is the set value of the slave select negation delay (SPDECR.SLNDL).	R/W

Bit	Symbol	Function	R/W
15	SCKDEN	RSPCK Delay Setting Enable [In the Motorola-SPI case] 0: RSPCK delay is 1 RSPCK. 1: RSPCK delay is the set value of the RSPCK delay (SPDECR.SCKDL). [In the TI-SSP case] 0: RSPCK delay is 0 RSPCK. 1: RSPCK delay is the set value of the RSPCK delay (SPDECR.SCKDL).	R/W
20:16	SPB[4:0]	SPI Data Length 0x00 to 0x02: Setting prohibited 0x03: 4 bits 0x04: 5 bits 0x05: 6 bits ⋮ 0x1E: 31 bits 0x1F: 32 bits	R/W
23:21	—	These bits are read as 0. The write value should be 0.	R/W
26:24	SSLA[2:0]	SSL Signal Assertion 0 0 0: SSL0 0 0 1: SSL1 0 1 0: SSL2 0 1 1: SSL3 Others: Setting prohibited	R/W
31:27	—	These bits are read as 0. The write value should be 0.	R/W

Note: S-TYPE-3, P-TYPE-3

The SPI has eight SPI command registers (SPCMD0 to SPCMD7) that are used to set the transfer format of the SPI in master mode. Furthermore, some bits in SPCMD0 are used to set the transfer format of the SPI in slave mode. The SPI in master mode sequentially references SPCMD0 to SPCMD7 according to the setting of the SPSLN[2:0] bits in the SPI Control register 3 (SPCR3), and then performs serial transfer specified in the referenced SPCMD.

SPI set the SPCMD register before setting data to be transmitted by referencing the SPCMD when the transmit buffer is empty (while the next-transfer data has not been set).

The SPCMD referenced by the SPI in master mode is indicated by SPCP[2:0] in the SPI status register (SPSR). If SPCMD0 is modified while the SPI in slave mode is enabled (SPCR.SPE = 1), subsequent operation is not guaranteed.

CPHA bit (RSPCK Phase)

This bit is used to set the RSPCK phase of the SPI in master mode or slave mode. To perform data communication between SPI modules, the same RSPCK phase must be set for both modules.

When SPCR.SPMS = 0 and SPCR.SPFRF = 1 (in TI SSP mode), setting CPHA = 0 is invalid.

CPOL bit (RSPCK Polarity)

This bit is used to set the RSPCK polarity of the SPI in master mode or slave mode. To perform data communication between SPI modules, the same RSPCK polarity must be set for both modules.

BRDV[1:0] bit (Bit Rate Division)

This register is used to determine the bit rate with a combination of the set values of the BRDV[1:0] bits and the SPI bit rate register (SPCR3.SPBR). The set SPBR value determines the base bit rate. The set BRDV[1:0] bits value is used to select undivided, 2-divided, 4-divided, or 8-divided base bit rate. SPCMD0 to SPCMD7 enable setting of different BRDV[1:0] values. This makes it possible to perform serial transfer with a different bit rate for each command.

SSLKP bit (SSL Signal Level Hold)

This bit is used to set whether to hold or negate the SSL signal level of the current command during a period from the SSL negation timing for the current command to the SSL assertion timing for the next command when the SPI in master mode performs serial transfer. Setting this bit to 1 enables burst transfer in SPI operation master mode. For details, see [section 43.3.12.1. Master Mode Operation](#).

To use the SPI in slave mode, set SSLKP bit to 0.

LSBF bit (SPI LSB First)

This bit is used to set the data format of the SPI in master mode or slave mode to MSB first or LSB first.

SPNDEN bit (SPI Next-Access Delay Enable)

This bit is used to set the period (next-access delay) after the SPI in master mode inactivates the SSL signal at the end of serial transfer until it enables SSL signal assertion of the next access. When SPNDEN bit = 0, the SPI sets the next-access delay to 1 RSPCK + 5TCLK. When SPNDEN bit = 1, the SPI inserts the next-access delay in accordance with the SPI next-access delay register (SPDECR.SPNDL) setting.

To use the SPI in slave mode, set SPNDEN bit to 0.

SLNDEN bit (SSL Negation Delay Setting Enable)

[In the Motorola-SPI case]

This bit is used to set the period (SSL negation delay) after the SPI in master mode stops RSPCK oscillation until it inactivates the SSL signal. When SLNDEN bit = 0, the SPI sets the SSL negation delay to 1 RSPCK. When SLNDEN bit = 1, the SPI negates the SSL signal with the RSPCK delay in accordance with the slave select negation delay register (SPDECR.SLNDL) setting.

To use the SPI in slave mode, set SLNDEN bit to 0.

[In the TI-SSP case]

This bit is used to set the period from when the master mode SPI stops RSPCK oscillation to when the OE signal is inactivated, or when the slave mode SPI detects the last edge of RSPCK and then negates the OE signal. When the SLNDEN bit is 0, the SSL negate delay is 1 RSPCK in master mode and 1 TCLK in slave mode. When SLNDEN bit = 1, the SPI negates the SSL signal with the RSPCK delay in accordance with the slave select negation delay register (SPDECR.SLNDL) setting.

When using SPI in slave mode except TI SSP setting, set the SLNDEN bit to 0.

SCKDEN bit (RSPCK Delay Setting Enable)

[In the Motorola-SPI case]

This bit is used to set the period (RSPCK delay) after the SPI in master mode activates the SSL signal until it oscillates RSPCK. When SCKDEN bit = 0, the SPI sets the RSPCK delay to 1 RSPCK. When SCKDEN bit = 1, the SPI starts RSPCK oscillation with the RSPCK delay in accordance with the RSPCK delay register (SPDECR.SCKDL) setting.

To use the SPI in slave mode, set SCKDEN bit to 0.

[In the TI-SSP case]

This bit is used to set the period from the start of assertion of the SSL signal to the RSPCK oscillation (RSPCK delay) and the period of the SSL signal to negation by the SPI in master mode. When SCKDEN bit = 0, the SPI does not set the RSPCK delay. When SCKDEN bit = 1, the SPI starts RSPCK oscillation with the RSPCK delay in accordance with the RSPCK delay register (SPDECR.SPCKDL) setting.

To use the SPI in slave mode, set SCKDEN bit to 0.

SPB[4:0] bit (SPI Data Length)

These bits are used to set the transfer data length of the SPI in master mode or slave mode.

SSLA[2:0] bit (SSL Signal Assertion)

These bits are used to control SSL signal assertion for the SPI in master mode to perform serial transfer. The set SSLA[2:0] bits value controls assertion of the SSL3 to SSL0 signals. The signal polarity when the SSL signal is asserted depends on the set value of the SPI slave select polarity register (SPCR3.SSLiP). When SSLA[2:0] bits are set to 000b in multi-master mode, serial transfer is performed with all SSL signals negated (because SSL0 is input).

To use the SPI in slave mode, set SSLA[2:0] bits to 000b.

43.2.7 SPDCR : SPI Data Control Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
 SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x40

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	SPFC[1:0]	—	—	—	SINV	SPRDT	TD	—	—	BYSW
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	BYSW	Byte Swap Operating Mode Select 0: Byte Swap OFF 1: Byte Swap ON	R/W
2:1	—	These bits are read as 0. The write value should be 0.	R/W
3	SPRDTD	SPI Receive Data or Transmit Data Select 0: The SPDR reads the receive buffer. 1: The SPDR reads the transmit buffer	R/W
4	SINV	Serial data invert bit 0: Not invert serial data 1: Invert serial data.	R/W
7:5	—	These bits are read as 0. The write value should be 0.	R/W
9:8	SPFC[1:0]	Frame Count 0 0: 1 frame 0 1: 2 frames 1 0: 3 frames 1 1: 4 frames	R/W
31:10	—	These bits are read as 0. The write value should be 0.	R/W

Note: S-TYPE-3, P-TYPE-3

The SPI data control register (SPDCR) controls the data format.

If the value set in this register is changed while the SPE bit is 1, subsequent operations are not guaranteed.

BYSW bit (Byte Swap Operating Mode Select)

It is a setting bit, that is to swap a transmit/receive data in byte units. A data after byte swap is different by a data length (setting of SPCMD.SPB[4:0]).

When byte swap, A data length (setting of SPB[4:0]) must be set to 32bit or 16bit. Other case of data length (i.e. 4 to 15, 17 to 31-bit length), byte swap is not guaranteed. For the arrangement of data before and after swapping data lengths of 32 bits and 16 bits, see [section 43.3.4.3. Byte Swap Transmission](#) and [section 43.3.4.4. Byte Swap Reception](#).

When the parity function set to valid, the behavior is not guaranteed.

SPRDTD bit (SPI Receive Data or Transmit Data Select)

This bit is used to select receive buffer or transmit buffer from which the SPI data register (SPDR) value is read.

When the transmit buffer is read, the value that was written to SPDR immediately before is read.

SINV bit (Serial data invert bit)

This bit is used to invert transmit data and receive data.

When the SINV bit is set to 1, transmit buffer (SPTX) data is inverted to invert transmit data and receive data, and then the inverted data is stored in the receive buffer (SPRX). The parity bit is the value corresponding to the inverted transmission/reception data.

SPFC[1:0] bit (Frame Count)

Used for the condition to set the CENDF flag in slave receive only mode.

For details on the CENDF flag setting conditions, see [section 43.2.9. SPSR : SPI Status Register](#).

Note that this bit is invalid except in the slave receive only mode.

43.2.8 SPDCR2 : SPI Data Control Register 2

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x44

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	TTRG[1:0]		—	—	—	—	—	—	RTRG[1:0]	
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
1:0	RTRG[1:0]	Receive FIFO threshold setting 0 0: threshold 0 0 1: threshold 1 1 0: threshold 2 1 1: threshold 3	R/W
7:2	—	These bits are read as 0. The write value should be 0.	R/W
9:8	TTRG[1:0]	Transmission FIFO threshold setting 0 0: threshold 0 0 1: threshold 1 1 0: threshold 2 1 1: threshold 3	R/W
31:10	—	These bits are read as 0. The write value should be 0.	R/W

Note: S-TYPE-3, P-TYPE-3

SPI data control register 2 (SPDCR2) controls the FIFO threshold. If the value set in this register is changed while the SPE bit is 1, subsequent operations are not guaranteed.

RTRG[1:0] bit (Receive FIFO threshold setting)

Set the receive FIFO threshold.

When the number of data stored in the receive FIFO > the number of frames set by RTRG[1:0], the receive buffer full flag is set.

TTRG[1:0] bit (Transmission FIFO threshold setting)

Set the transmit FIFO threshold.

When the number of empty stages in the transmit FIFO > the number of frames set in TTRG[1:0], the transmit buffer empty flag is set.

43.2.9 SPSR : SPI Status Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
 SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x50

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	SPRF	CEND F	SPTE F	UDRF	PERF	MODF	IDLNF	OVRF	SPDR F	—	—	—	—	—	—	—
Value after reset:	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	SPECM[2:0]			—	SPCP[2:0]			—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	—	These bits are read as 0. The write value should be 0.	R
10:8	SPCP[2:0]	SPI Command Pointer 0 0 0: SPCMD0 0 0 1: SPCMD1 0 1 0: SPCMD2 0 1 1: SPCMD3 1 0 0: SPCMD4 1 0 1: SPCMD5 1 1 0: SPCMD6 1 1 1: SPCMD7	R
11	—	This bit is read as 0. The write value should be 0.	R
14:12	SPECM[2:0]	SPI Error Command 0 0 0: SPCMD0 0 0 1: SPCMD1 0 1 0: SPCMD2 0 1 1: SPCMD3 1 0 0: SPCMD4 1 0 1: SPCMD5 1 1 0: SPCMD6 1 1 1: SPCMD7	R
22:15	—	These bits are read as 0. The write value should be 0.	R
23	SPDRF	SPI Receive Data Ready Flag 0: Receive data ready not detected 1: Receive data ready detected	R
24	OVRF	Overrun Error Flag 0: No overrun error is present. 1: An overrun error is present.	R
25	IDLNF	SPI Idle Flag 0: The SPI is in the idle state. 1: The SPI is in the transfer state.	R
26	MODF	Mode Fault Error Flag 0: Neither mode fault error nor underrun error is present. 1: A mode fault error or underrun error is present.	R
27	PERF	Parity Error Flag 0: No parity error is present. 1: A parity error is present.	R
28	UDRF	Underrun Error Flag 0: When MODF=0, neither mode fault error nor underrun error is present. When MODF=1, a mode fault error is present. 1: When MODF=0, neither mode fault error nor underrun error is present. When MODF=1, an underrun error is present.	R

Bit	Symbol	Function	R/W
29	SPTEF	SPI Transmit Buffer Empty Flag 0: The number of empty stages in the transmit FIFO $\leq$ the value set in SPDCR2.TTRG 1: The number of empty stages in the transmit FIFO $>$ the value set in SPDCR2.TTRG	R
30	CENDF	Communication End Flag 0: The SPI is not communicating or communicating. 1: The SPI communication completed.	R
31	SPRF	SPI Receive Buffer Full Flag 0: The number of data stored in the receive FIFO $\leq$ number of frames set by the SPDCR2.RTRG bit. 1: The number of data stored in the receive FIFO $>$ number of frames set by the SPDCR2.RTRG bit.	R

Note: S-TYPE-3, P-TYPE-3

The SPI status register (SPSR) stores flags that indicate SPI's operating status.

SPCP[2:0] bit (SPI Command Pointer)

These bits indicate SPI command registers 0 to 7 (SPCMD0 to SPCMD7) indicated by the current pointer in the SPI sequence control. For details about the SPI sequence control, see [section 43.3.13.1. Master Mode Operation](#).

SPECM[2:0] bit (SPI Error Command)

These bits indicate SPI command registers 0 to 7 (SPCMD0 to SPCMD7) indicated by the command pointer (SPCP[2:0] bits) when an error was detected in the SPI sequence control. The SPI updates the SPECM[2:0] bits value only when an error is detected. When no error is present (OVRF, MODF, and PERF flags in SPSR are 0), the SPECM[2:0] bits value has no meaning. For the SPI's error detection function, see [section 43.3.10. Error Detection](#). For the SPI's sequence control, see [section 43.3.13.1. Master Mode Operation](#).

SPDRF bit (SPI Receive Data Ready Flag)

During communication (SPCR.SPE = 1), a certain period of time has elapsed while the number of data stored in the reception FIFO $\leq$ the reception FIFO threshold.

This bit is set to 0 when the reception operation is not performed (SPCR.TXMD[1:0] = 01b).

[Setting condition]

All the following two conditions are met.

- SPCR2.SPDRC[7:0] $\neq$ 0x00.
- After the receive FIFO has been written, when the number of data stored in the receive FIFO $\leq$ the receive FIFO threshold and the value set by SPDRC[7:0] has elapsed

[Clearing condition]

- When 1 is written to the SPSRC.SPDRFC bit.

OVRF bit (Overrun Error Flag)

This flag indicates whether an overrun error is present. When the RSPCK clock auto-stop function is enabled (SPCR.SCKASE = 1) in master mode (SPCR.MSTR = 1), no overrun error occurs and, therefore, this flag is not set to 1. For details, see [section 43.3.10.1. Overrun Errors](#).

[Setting condition]

When serial transfer is completed in one of the following two conditions with data stored in the receive FIFO for the number of FIFO stages.

- SPCR.TXMD[1:0] = 00b. (transmit-receive mode)
- SPCR.TXMD[1:0] = 10b. (receive only)

[Clearing condition]

- When 1 is written to the SPSRC.OVRFC bit.

IDLNF bit (SPI Idle Flag)

This flag indicates transfer status of the SPI.

[Setting condition]

[Transmit-Receive, Transmit-only Master mode]

- None of the clearing conditions (Transmit-Receive / Transmit-only in Master mode) below is met.

[Receive-only Master mode]

- When 1 is written to RMSTTG of SPCR2.

[Slave mode]

- The SPE bit in SPCR is 1 (SPI function enabled).

[Clearing conditions]

Communication status: 1-(1) to (6) * For details of communication status, see [Table 43.4](#).

[Transmit-Receive, Transmit-only Master mode]

Any of the following two conditions is met.

- The SPE bit in SPCR is 0 (SPI initialization).
- All the following three conditions are met.
 - The next transfer data is not set in the transmission buffer (SPTXn, n = 0 to 3)
 - The SPCP bits in SPSR are 000b (at the beginning of sequence control).
 - The operation completed by the next access delay (the master main state machine has transitioned to the idle state)

[Receive-only Master mode]

Communication status: 1-(7) to (9)

Any of the following two conditions is met.

- The SPE bit in SPCR is 0 (SPI initialization).
- Any of the following 3 conditions is met.
 - When RMFM[4:0] = 0x00, after writing 1 to RMEDTG, the operation completed by the next access delay (the master main state machine has transitioned to the idle state)
 - When RMFM[4:0] ≠ 0x00, after writing 1 to RMEDTG, the operation completed by the next access delay (the master main state machine has transitioned to the idle state)
 - When RMFM[4:0] ≠ 0x00, the operation completed by the next access delay after processing is completed for the number of received frames set in RMFM[4:0] (the master main state machine has transitioned to the idle state)

[Slave mode]

Communication status: 0-(1) to (9)

- The SPE bit in SPCR is 0 (SPI initialization).

MODF bit (Mode Fault Error Flag)

This flag indicates whether a mode fault error or an underrun error is present. The UDRF flag allows you to see which error (mode fault error or underrun error) has occurred.

[Setting condition]

[Multi-master mode]

- The SSL0 pin input level becomes active level while the SPCR.MSTR bit = 1 (master mode) and the SPCR.MODFEN bit = 1 (mode fault error detection enabled), and then the SPI has detected a mode fault error.

[Slave, Motorola-SPI mode]

Any of the following two conditions is met.

- The SSL0 pin is negated before the RSPCK cycles necessary for data transfer end while the SPCR.MSTR bit = 0 (slave mode), SPCR.SPFRF bit = 0 (Motorola-SPI) and the SPCR.MODFEN bit = 1 (mode fault error detection enabled), and then the SPI has detected a mode fault error.
- Serial transfer is started before transmit data output becomes ready while the SPCR.SPE bit = 1 (SPI function enabled), and then the SPI has detected an underrun error.

[Slave, TI-SSP mode]

Any of the following two conditions is met.

- The SSL0 pin is asserted before the RSPCK cycles necessary for data transfer end while the SPCR.MSTR bit = 0 (slave mode), SPCR.SPFRF bit = 1 (TI-SSP) and the SPCR.MODFEN bit = 1 (mode fault error detection enabled), and then the SPI has detected a mode fault error.
- Serial transfer is started before transmit data output becomes ready while the SPCR.SPE bit = 1 (SPI function enabled), and then the SPI has detected an underrun error.

The SSL0 signal active level depends on the SPCR3.SSLiP bits (SSL signal polarity bits).

[Clearing condition]

- When 1 is written to the SPSRC.MODFC bit.

PERF bit (Parity Error Flag)

This flag indicates whether a parity error is present.

[Setting condition]

When the serial transfer ends and a parity error is detected with the SPPE bit of SPCR set to 1 under any of the following 2 conditions.

- SPCR.TXMD[1:0] = 00b. (transmit-receive master mode or transmit-receive slave mode)
- SPCR.TXMD[1:0] = 10b. (receive-only master mode or receive-only slave mode)

[Clearing condition]

- When 1 is written to the SPSRC.PERFC bit.

UDRF bit (Underrun Error Flag)

This flag indicates that a mode fault error or an underrun error is present.

[Setting condition]

- Serial transfer is started before transmit data output becomes ready while the SPCR.MSTR bit = 0 and the SPCR.TXMD[1:0] bit = 00b or 01b (transmit-receive slave mode or transmit-only slave mode) and the SPCR.SPE bit = 1 (SPI function enabled), and then the SPI has detected an underrun error.

[Clearing condition]

- When 1 is written to the SPSRC.UDRFC bit.

SPTEF bit (SPI Transmit Buffer Empty Flag)

This flag indicates the transmit buffer (SPTX) status in the SPI data register (SPDR).

[Setting condition]

Any of the following 3 conditions is met.

- The SPE bit is set to 0 (SPI initialization).
- When the number of empty transmission FIFO stages > the value set in SPDCR2.TTRG[1:0].
- When 1 is written to SPFCR.SPFRST.

[Clearing condition]

Any of the following two conditions is met.

- At the time of final access when transmission data is written to SPDR (SPTXn, n = 0 to 3) in one processing routine using DTC / DMAC.
- When 1 is written to the SPSRC.SPTEFC bit.

Writing a value to the SPDR register is enabled only while the SPTEF flag = 1. If a value is written to the SPDR register while the SPTEF flag = 0, transmit buffer data is not updated.

CENDF bit (Communication End Flag)

This flag indicates communication end status of SPI. It turns 1 at communication end and turns 0 at starting next communication.

[Setting condition]

[Transmit-Receive / Transmit-only Master mode]

Communication status: 1-(1) to (6) * For details of communication status, see [Table 43.4](#).

The following 3 conditions are met.

- The next transfer data is not set in the transmission buffer (SPTXn, n = 0 to 3)
- The SPSR.SPCP[2:0] are 000b. (It means the head of the sequential control.)
- The operation completed by the next access delay (the master main state machine has transitioned to the idle state)

[Receive-only Master mode]

Communication status: 1-(7) to (9)

Any of the following 3 conditions is met.

- When RMFM[4:0] = 0x00, after writing 1 to RMEDTG, the operation completed by the next access delay (the master main state machine has transitioned to the idle state)
- When RMFM[4:0] ≠ 0x00, after writing 1 to RMEDTG, the operation completed by the next access delay (the master main state machine has transitioned to the idle state)
- When RMFM[4:0] ≠ 0x00, the operation completed by the next access delay after processing is completed for the number of received frames set in RMFM[4:0] (the master main state machine has transitioned to the idle state)

[Transmit-receive / transmit-only slave, Motorola-SPI mode at SPI serial communication (4-wire: the SPCR.SPMS bit is 0)]

Communication status: 0-(1), (4)

The following 3 conditions are met.

- The next transfer data is not set in the transmission buffer
- The transmission shift register is empty. (It means SPI does not do serial transfer.)
- SSL0 was negated.

[Transmit-receive / transmit-only slave, TI-SSP mode at SPI serial communication (4-wire: the SPCR.SPMS bit is 0)]

Communication status: 0-(2), (5)

The following 3 conditions are met.

- The next transfer data is not set in the transmission buffer
- The transmission shift register is empty. (It means SPI does not do serial transfer.)
- When the SSL0 negate delay is completed.

[Transmit-receive / transmit only slave mode at clock synchronous (3-wire: the SPCR.SPMS bit is 1)]

Communication status: 0-(3), (6)

The following 3 conditions are met.

- The next transfer data is not set in the transmission buffer
- The transmission shift register is empty. (It means SPI does not do serial transfer.)
- The last even edge of RSPCK of the frame was detected. (When the SPCMD.CPHA bit is 1.)

[Receive only slave, Motorola-SPI mode at SPI serial communication (4-wire: the SPCR.SPMS bit is 0)]

Communication status: 0-(7)

The following condition is met.

- SSL0 input was negated after getting frames for SPDCR.SPFC set value in the receive buffer.

[Receive only slave, TI-SSP mode at SPI serial communication (4-wire: the SPCR.SPMS bit is 0)]

Communication status: 0-(8)

The following condition is met.

- SSL0 negate delay is completed after getting frames for SPDCR.SPFC set value in the receive buffer.

[Receive only slave mode at clock synchronous (3-wire: the SPCR.SPMS bit is 1)]

Communication status: 0-(9)

The following condition is met.

- The last even edge of RSPCK of the Last frame received for SPFC sets value. (When the SPCMD.CPHA bit is 1.)

[Clearing condition]

[Transmit-Receive / Transmit-only Master mode]

Communication status: 1-(1) to (6)

Any of the following 2 conditions is met.

- The next transmit data was written to the transmit buffer (SPTX).
- When 1 is written to the SPSRC.CENDFC bit.

[Receive -only Master mode]

Communication status: 1-(7) to (9)

Any of the following 2 conditions is met.

- When 1 is written to the SPCR2.RMSTTG bit with SPE = 1.
- When 1 is written to the SPSRC.CENDFC bit.

[Transmit-receive / transmit only slave mode]

Communication status: 0-(1) to (6)

Satisfy one of following.

- The next transmit data was written to the transmit buffer (SPTX).
- When 1 is written to the SPSRC.CENDFC bit.

[Receive only slave mode at SPI serial communication (4-wire: the SPCR.SPMS bit is 0)]

Communication status: 0-(7) to (8)

Satisfy one of following.

- SSL0 assertion of next data was detected.
- When 1 is written to the SPSRC.CENDFC bit.

[Receive only slave mode at clock synchronous (3-wire: the SPCR.SPMS bit is 1)]

Communication status: 0-(9)

Satisfy one of following.

- The first edge of RSPCK of the next data was detected.
- When 1 is written to the SPSRC.CENDFC bit.

SPRF bit (SPI Receive Buffer Full Flag)

This flag indicates the receive buffer (SPRX) status in the SPI data register (SPDR).

[Setting condition]

When the number of data stored in the receive FIFO > the number of frames set in the SPDCR2.RTRG[1:0] bits in Transmit-Receive, receive-only mode. However, the SPRF flag does not change from 0 to 1 while the OVRF flag = 1. (See [section 43.3.10. Error Detection.](#))

[Clearing condition]

Any of the following 3 conditions is met.

- At the last access when read data is read from SPDR (SPRXn, n = 0 to 3) in one processing routine using DTC / DMAC
- When 1 is written to the SPSRC.SPRFC bit
- When 1 is written to the SPFCR.SPFRST bit

43.2.10 SPTFSR : SPI Transfer FIFO Status Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x58

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	TFDN[2:0]		
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

Bit	Symbol	Function	R/W
2:0	TFDN[2:0]	Transmit FIFO data empty stage number 0 0 0: Number of empty stages 0 ⋮ 1 0 0: Number of empty stages 4	R
31:3	—	These bits are read as 0. The write value should be 0.	R

TFDN[2:0] bit (Transmit FIFO data empty stage number)

Displays the number of empty transmission FIFO stages. By clearing the SPCR.SPE bit, TFDN[2:0] will be the initial value after reset (= all empty).

43.2.11 SPRFSR : SPI Receive FIFO Status Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x5C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	RFDN[2:0]		
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
2:0	RFDN[2:0]	Receive FIFO data store stage number 0 0 0: Number of store stages 0 ⋮ 1 0 0: Number of store stages 4	R
31:3	—	These bits are read as 0. The write value should be 0.	R

Note: S-TYPE-3, P-TYPE-3

RFDN[2:0] bit (Receive FIFO data store stage number)

Displays the number of stores receive FIFO stages. RFDN [2:0] is cleared by clearing the SPCR.SPE bit.

43.2.12 SPPSR : SPI Polling Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x60

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	SPEP S
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	SPEPS	SPI Polling Status 0: SPCR.SPE is 0 1: SPCR.SPE is 1	R
31:1	—	These bits are read as 0. The write value should be 0.	R

SPEPS bit (SPI Polling Status)

This bit indicates status of SPCR.SPE bit after synchronization from bus clock (PCLK) to operation clock (TCLK).

43.2.13 SPSRC : SPI Status Clear Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x68

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	SPRF C	CEND FC	SPTF FC	UDRF C	PERF C	MODF C	—	OVRF C	SPDR FC	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
22:0	—	The write value should be 0.	W
23	SPDRFC	SPI Receive Data Ready Flag Clear By writing 1, the SPI Receive Data Ready Flag can be cleared. Reading value is always 0.	W

Bit	Symbol	Function	R/W
24	OVRFC	Overrun Error Flag Clear By writing 1, the Overrun Error Flag can be cleared. Reading value is always 0.	W
25	—	The write value should be 0.	W
26	MODFC	Mode Fault Error Flag Clear By writing 1, the Mode Fault Error Flag can be cleared. Reading value is always 0.	W ^{*1}
27	PERFC	Parity Error Flag Clear By writing 1, the Parity Error Flag can be cleared. Reading value is always 0.	W
28	UDRFC	Underrun Error Flag Clear By writing 1, the Underrun Error Flag can be cleared. Reading value is always 0.	W ^{*2}
29	SPTEFC	SPI Transmit Buffer Empty Flag Clear By writing 1, the SPI Transmit Buffer Empty Flag can be cleared. Reading value is always 0.	W
30	CENDFC	Communication End Flag Clear By writing 1, the Communication End Flag can be cleared. Reading value is always 0.	W
31	SPRFC	SPI Receive Buffer Full Flag Clear By writing 1, the SPI Receive Buffer Full Flag can be cleared. Reading value is always 0.	W

Note: S-TYPE-3, P-TYPE-3

Note 1. Before setting MODFC and UDRFC, make sure that SPSR.MODF and UDRF are set to 1.

Note 2. When clearing the UDRF flag, clear the MODF flag at the same time (MODFC = 1).

The SPI status clear register (SPSRC) is a register that clears the status flag (SPSR) that indicates the operating status of SPI.

43.2.14 SPFCR : SPI FIFO Clear Register

Base address: SPIn_B = 0x4035_C000 + 0x0100 × n (n = 0, 1)
SPIn_B_NS = 0x5035_C000 + 0x0100 × n (n = 0, 1)

Offset address: 0x6C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	SPFR ST
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	SPFRST	SPI FIFO clear By writing 1, the pointer in the FIFO and the stored data are initialized. Reading value is always 0.	W
31:1	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

The FIFO clear register (SPFCR) is used to clear the FIFO.

If SPFCR is rewritten while the SPE bit of the SPI control register (SPCR) is 1, subsequent operations are not guaranteed.

SPFRST bit (SPI FIFO clear)

Initializing the pointer and stored data in the transmit / receive FIFO by writing 1

43.3 Operation

In this section, the serial transfer period refers to the period from the beginning of driving valid data to the fetching of the final valid data.

43.3.1 Overview of SPI Operation

The SPI is capable of synchronous serial transfers in the following modes:

- Slave mode (SPI operation)
- Single-master mode (SPI operation)
- Multi-master mode (SPI operation)
- Slave mode (clock synchronous operation)
- Master mode (clock synchronous operation)

The SPI mode can be selected by using the MSTR, MODFEN, SPMS, and SPFRF bits in SPCR. Table 43.6 lists the relationship between SPI modes and SPCR settings, and a description of each mode.

Table 43.6 Relationship between SPCR settings and SPI modes (1 of 2)

Mode	Slave (SPI operation)	Single-master (SPI operation)	Multi-master (SPI operation)	Slave (clock synchronous operation)	Master (clock synchronous operation)
MSTR bit setting	0	1	1	0	1
MODFEN bit setting	0 or 1	0	1	0	0
SPMS bit setting	0	0	0	1	1
SPFRF bit setting	valid	valid	valid	Invalid	Invalid
RSPCKn pins	Input	Output	Output/Hi-Z	Input	Output
MOSIn pin	Input	Output	Output/Hi-Z	Input	Output
MISOOn pin	Output/Hi-Z	Input	Input	Output	Input
SSLn0 pins	Input	Output	Input	Hi-Z ^{*1}	Hi-Z ^{*1}
SSLn1 to SSLn3 pins	Hi-Z ^{*1}	Output	Output/Hi-Z	Hi-Z ^{*1}	Hi-Z ^{*1}
SSL polarity change function	Supported	Supported	Supported	—	—
Max transfer rate	TCLK/2	TCLK/2	TCLK/2	TCLK/2	TCLK/2
Clock source	RSPCK input	On-chip baud rate generator	On-chip baud rate generator	RSPCK input	On-chip baud rate generator
Clock polarity	Two				
Clock phase	Two ^{*6}	Two ^{*6}	Two ^{*6}	One (CPHA = 1)	Two
Transfer data length	4 to 32 bits				
Burst transfer	Possible (CPHA = 1)	Possible (CPHA = 0, 1)	Possible (CPHA = 0, 1)	—	—
RSPCK delay control	Not supported	Supported	Supported	Not supported	Supported
SSL negation delay control	Not supported ^{*7}	Supported	Supported	Not supported	Supported
Next-access delay control	Not supported	Supported	Supported	Not supported	Supported
Transfer trigger	SSL input active or RSPCK oscillation	Write to transmit buffer on generation of transmit buffer empty interrupt request (SPTEF = 1)	Write to transmit buffer on generation of transmit buffer empty interrupt request (SPTEF = 1)	RSPCK oscillation	Write to transmit buffer on generation of transmit buffer empty interrupt request (SPTEF = 1)
Sequence control	Not supported	Supported	Supported	Not supported	Supported
Transmit buffer empty detection	Supported ^{*5}				
Receive buffer full detection	Supported ^{*2}				
Overrun error detection	Supported ^{*2}	Supported ^{*2 *4}	Supported ^{*2 *4}	Supported ^{*2}	Supported ^{*2}

Table 43.6 Relationship between SPCR settings and SPI modes (2 of 2)

Mode	Slave (SPI operation)	Single-master (SPI operation)	Multi-master (SPI operation)	Slave (clock synchronous operation)	Master (clock synchronous operation)
Parity error detection	Supported*3 *2				
Mode fault error detection	Supported (MODFEN = 1)	Not supported	Supported	Not supported	Not supported
Underrun error detection	Supported*5	Not supported	Not supported	Supported*5	Not supported

Note 1. This function is not supported in this mode.

Note 2. When SPI is transmit-master mode or transmit-slave mode (see Table 43.4), detection of receive buffer full, overrun error, and parity error are not performed.

Note 3. When the SPCR.SPPE bit is 0, parity error detection is not performed.

Note 4. When the SPCR.SCKASE bit is 1, overrun error detection does not proceed.

Note 5. When SPI is receive only slave mode, none of transmit buffer empty and underrun error is detected.

Note 6. CPHA = 0 is invalid in TI SSP mode. (If it is set, the operation is the same as when CPHA = 1.)

Note 7. Supported only in TI SSP mode.

43.3.2 Controlling the SPI Pins

Based on the settings of the MSTR, MODFEN, and SPMS bits in SPCR and the PmnPFS.NCODR bit for I/O Ports, the SPI can switch pin states. Table 43.7 lists the relationship between pin states and bit settings. Setting the PmnPFS.NCODR bit for an I/O port to 0 selects the CMOS output. Setting it to 1 selects the open-drain output. The I/O port settings must follow this relationship.

Table 43.7 Relationship between pin states and bit settings

Mode	Pin	Pin state*2	
		PmnPFS.NCODR bit for I/O ports = 0	PmnPFS.NCODR bit for I/O ports = 1
Single-master mode (SPI operation) (MSTR = 1, MODFEN = 0, SPMS = 0)	RSPCKn	CMOS output	Open-drain output
	SSLn0 to SSLn3	CMOS output	Open-drain output
	MOSIn	CMOS output	Open-drain output
	MISOOn	Input	Input
Multi-master mode (SPI operation) (MSTR = 1, MODFEN = 1, SPMS = 0)	RSPCKn*3	CMOS output/Hi-Z	Open-drain output/Hi-Z
	SSLn0	Input	Input
	SSLn1 to SSLn3*3	CMOS output/Hi-Z	Open-drain output/Hi-Z
	MOSIn*3	CMOS output/Hi-Z	Open-drain output/Hi-Z
	MISOOn	Input	Input
Slave mode (SPI operation) (MSTR = 0, SPMS = 0)	RSPCKn	Input	Input
	SSLn0	Input	Input
	SSLn1 to SSLn3*5	Hi-Z*1	Hi-Z*1
	MOSIn	Input	Input
	MISOOn*4	CMOS output/Hi-Z	Open-drain output/Hi-Z
Master mode (clock synchronous operation) (MSTR = 1, MODFEN = 0, SPMS = 1)	RSPCKn	CMOS output	Open-drain output
	SSLn0 to SSLn3*5	Hi-Z*1	Hi-Z*1
	MOSIn	CMOS output	Open-drain output
	MISOOn	Input	Input
Slave mode (clock synchronous operation) (MSTR = 0, SPMS = 1)	RSPCKn	Input	Input
	SSLn0 to SSLn3*5	Hi-Z*1	Hi-Z*1
	MOSIn	Input	Input
	MISOOn	CMOS output	Open-drain output

Note 1. This function is not supported in this mode.

Note 2. SPI settings are not reflected in multiplexed pins for which the SPI function is not selected.

Note 3. Motorola-SPI: When SSLn0 is at the active level, the pin state is Hi-Z. Whether or not the input signal is at the active level determines the setting of the SPCR3.SSL0P bit.

TI-SSP: From when SSL0 is at the active level until communication is completed, the pin state is Hi-Z under the condition SPCR.SPE = 1.

Note 4. Motorola-SPI: When SSLn0 is at the non-active level or the SPCR.SPE bit is 0, the pin state is Hi-Z. Whether or not the input signal is at the active level determines the setting of the SPCR3.SSL0P bit.

TI-SSP: When SSL0 is except the communication period or when the SPE bit of SPCR is 0 (assertion after SPE = 1 and communication is completed), the pin status changes to Hi-Z.

Note 5. These pins are available for use as I/O port pins.

The SPI in single-master mode (SPI operation) or multi-master mode (SPI operation) determines the MOSI signal values during the SSL negation period (including the SSL retention period during a burst transfer) based on the MOIFE and MOIFV bit settings in SPCR2, as listed in [Table 43.8](#).

Table 43.8 MOSI signal value determination during SSL negation

MOIFE bit	MOIFV bit	MOSIn signal value during SSL negation
0	0, 1	Final data from previous transfer
1	0	Low
1	1	High

43.3.3 SPI System Configuration Examples

This configuration example describes that 0 level of SSL0n signals is active level.

When connecting and using in a multi-slave or multi-master mode, the transfer format of the connected device should be unified to either Motorola-SPI or TI-SSP.

43.3.3.1 Single-master/Single-slave with the MCU as a Master

[Figure 43.6](#) shows a single-master/single-slave SPI system configuration example where the MCU is used as a master. In the single-master/single-slave configuration, the SSLni outputs of the MCU (master) are not used. The SSL input of the SPI slave is fixed to the low level, and the SPI slave is maintained in the selected state.*1

Note 1. In the transfer format configured when the SPCMDm.CPHA bit is 0, the SSL signal for some slave devices cannot be fixed to an active level. In this case, always connect the SSLni output of the MCU to the SSL input of the slave device.

The MCU (master) drives the RSPCKn and MOSIn signals. The SPI slave drives the MISO signal.

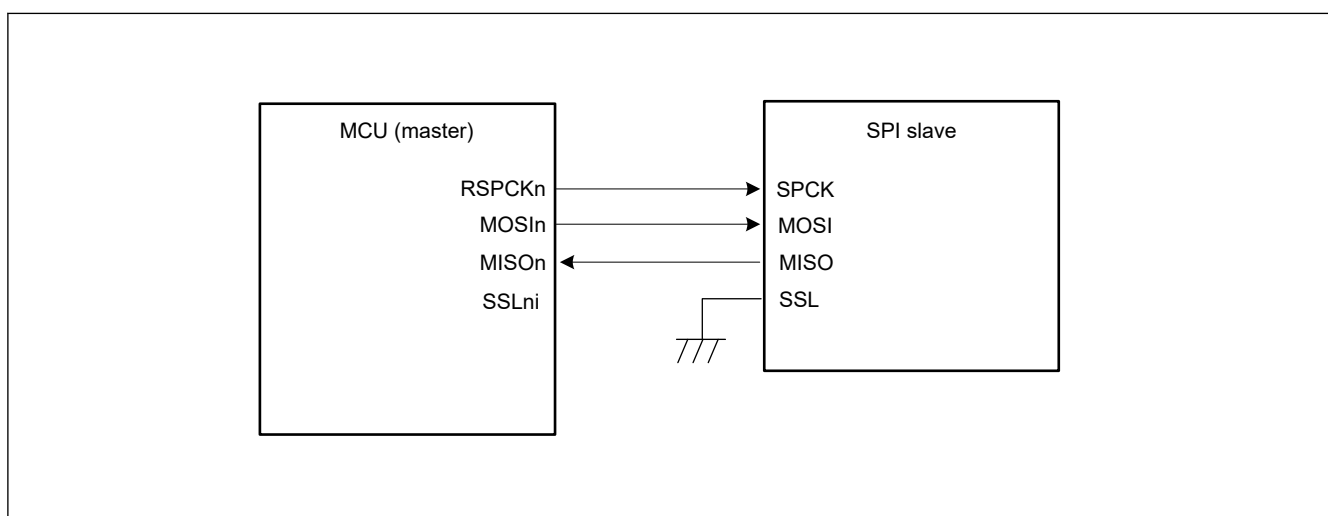


Figure 43.6 Single-master/single-slave configuration example with the MCU as a master

43.3.3.2 Single-master/Single-slave with the MCU as a Slave

Figure 43.7 shows a single-master/single-slave SPI system configuration example where the MCU is used as a slave. When the MCU operates as a slave, the SSLn0 pin is used as SSL input. The SPI master drives the RSPCK and MOSI signals. The MCU (slave) drives the MISO signal.^{*1}

Note 1. When SSLn0 is at a non-active level, the pin state is Hi-Z.

In the single-slave configuration when the SPCMDm.CPHA bit is set to 1, the SPCR.SPFRF bit is set to 0, and SPCR.SPMS is set to 0, the SSLn0 input of the MCU (slave) is fixed to the low level and the MCU (slave) is maintained in the selected state. This enables serial transfer execution (Figure 43.8). However, the communication end interrupt does not output when SSL0 input is fixed as Figure 43.8.

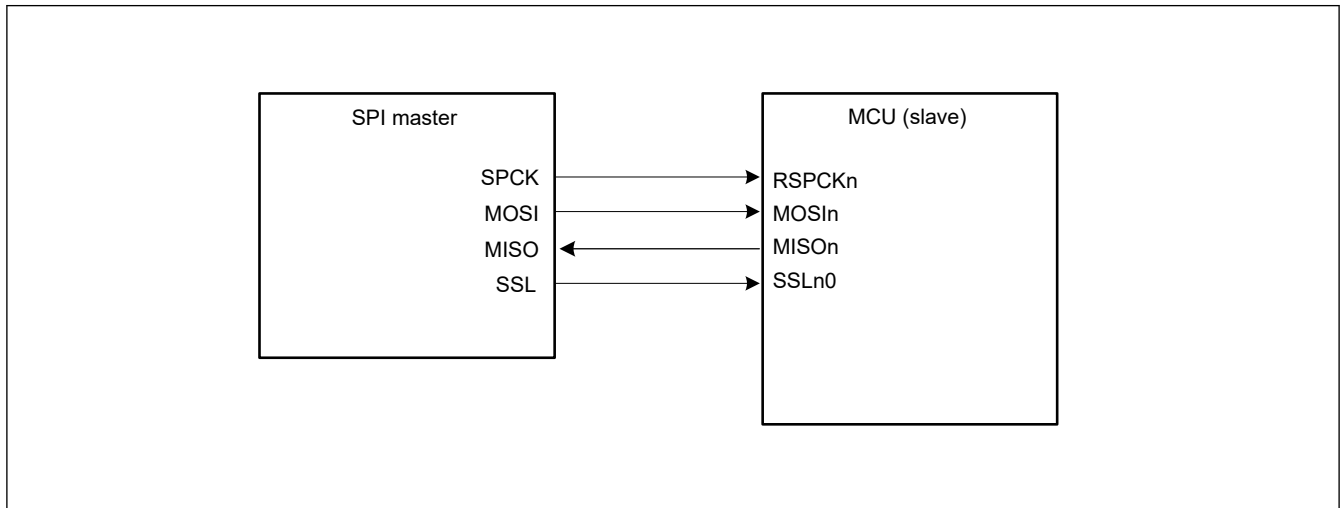


Figure 43.7 Single-master/single-slave configuration example with the MCU as a slave and CPHA = 0

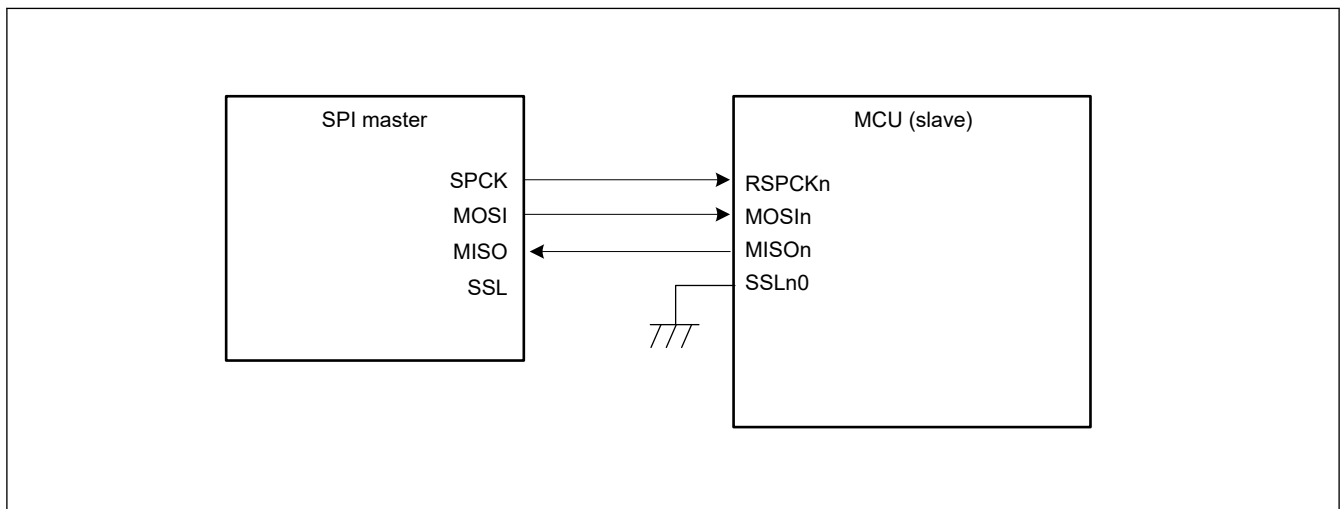


Figure 43.8 Single-master/single-slave configuration example with the MCU as a slave and CPHA = 1

43.3.3.3 Single-master/Multi-slave with the MCU as a Master

Figure 43.9 shows a single-master/multi-slave SPI system configuration example where the MCU is used as a master. In this example, the SPI system includes the MCU (master) and four slaves (SPI slave 0 to SPI slave 3).

The RSPCKn and MOSIn outputs of the MCU (master) are connected to the RSPCK and MOSI inputs of SPI slaves 0 to 3. The MISO outputs of SPI slaves 0 to 3 are all connected to the MISO input of the MCU (master). The SSLn0 to SSLn3 outputs of the MCU (master) are connected to the SSL inputs of SPI slaves 0 to 3, respectively.

The MCU (master) drives the RSPCKn, MOSIn, and SSLn0 to SSLn3 signals. Out of the SPI slaves 0 to 3, the slave that receives low-level input into the SSL input drives the MISO signal.

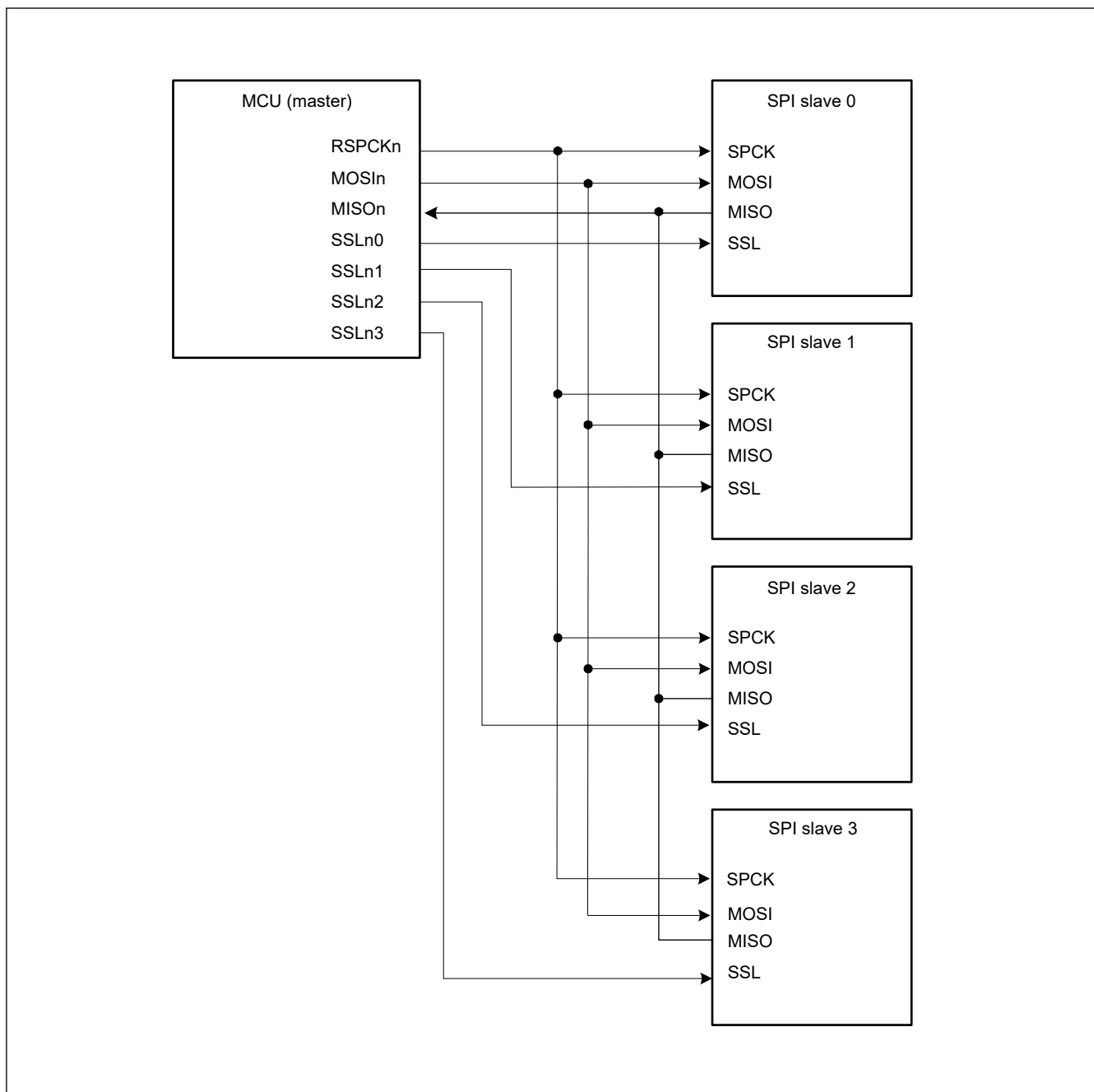


Figure 43.9 Single-master/multi-slave configuration example with the MCU as a master

43.3.3.4 Single-master/Multi-slave with the MCU as a Slave

Figure 43.10 shows a single-master/multi-slave SPI system configuration example where the MCU is used as a slave. In this example, the SPI system includes an SPI master and two MCUs (slaves X and Y).

The SPCK and MOSI outputs of the SPI master are connected to the RSPCKn and MOSIn inputs of the MCUs (slaves X and Y). The MISO_n outputs of the MCUs (slaves X and Y) are all connected to the MISO input of the SPI master. The SSLX and SSLY outputs of the SPI master are connected to the SSLn0 inputs of the MCUs (slaves X and Y, respectively).

The SPI master drives the SPCK, MOSI, SSLX, and SSLY signals. Of the MCUs (slaves X and Y), the slave that receives low-level input into the SSLn0 input drives the MISO_n signal.

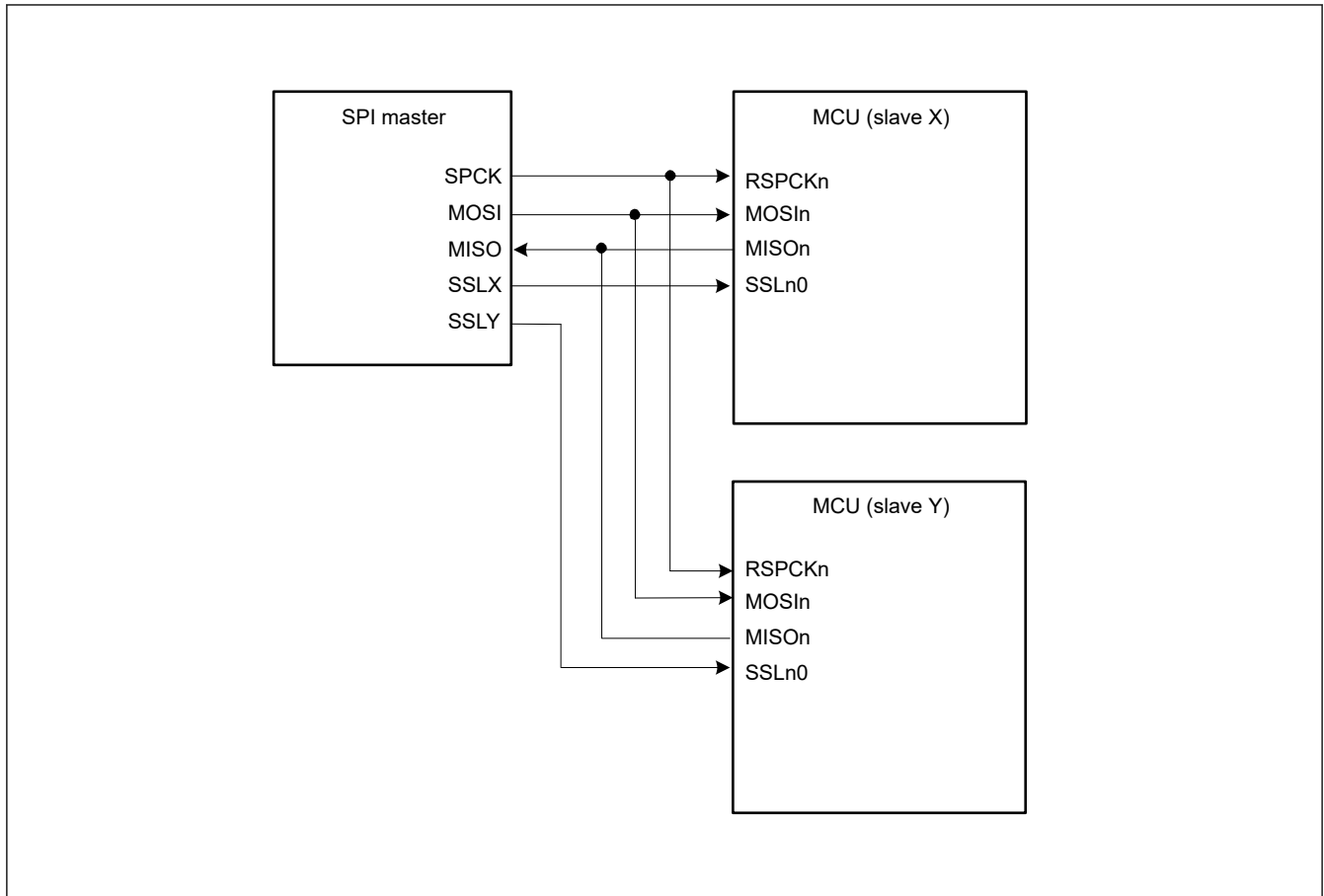


Figure 43.10 Single-master/multi-slave configuration example with the MCU as a slave

43.3.3.5 Multi-master/Multi-slave with the MCU as a Master

Figure 43.11 shows a multi-master/multi-slave SPI system configuration example where the MCU is used as a master. In this example, the SPI system includes two MCUs (masters X and Y) and two SPI slaves (SPI slaves 1 and 2).

The RSPCKn and MOSIn outputs of the MCUs (masters X and Y) are connected to the RSPCK and MOSI inputs of SPI slaves 1 and 2. The MISO outputs of SPI slaves 1 and 2 are connected to the MISOon inputs of the MCUs (masters X and Y). Any generic port Y output from the MCU (master X) is connected to the SSLn0 input of the MCU (master Y). Any generic port X output of the MCU (master Y) is connected to the SSLn0 input of the MCU (master X). The SSLn1 and SSLn2 outputs of the MCUs (masters X and Y) are connected to the SSL inputs of the SPI slaves 1 and 2. In this configuration example, because the system can be comprised solely of SSLn0 input, and SSLn1 and SSLn2 outputs for slave connections, the SSLn3 output of the MCU is not required.

The MCU drives the RSPCKn, MOSIn, SSLn1, and SSLn2 signals when the SSLn0 input level is high. When the SSLn0 input level is low, the MCU detects a mode fault error, sets RSPCKn, MOSIn, SSLn1, and SSLn2 to Hi-Z, and releases the SPI bus directly to the other master. Of the SPI slaves 1 and 2, the slave that receives low-level input into the SSL input drives the MISO signal.

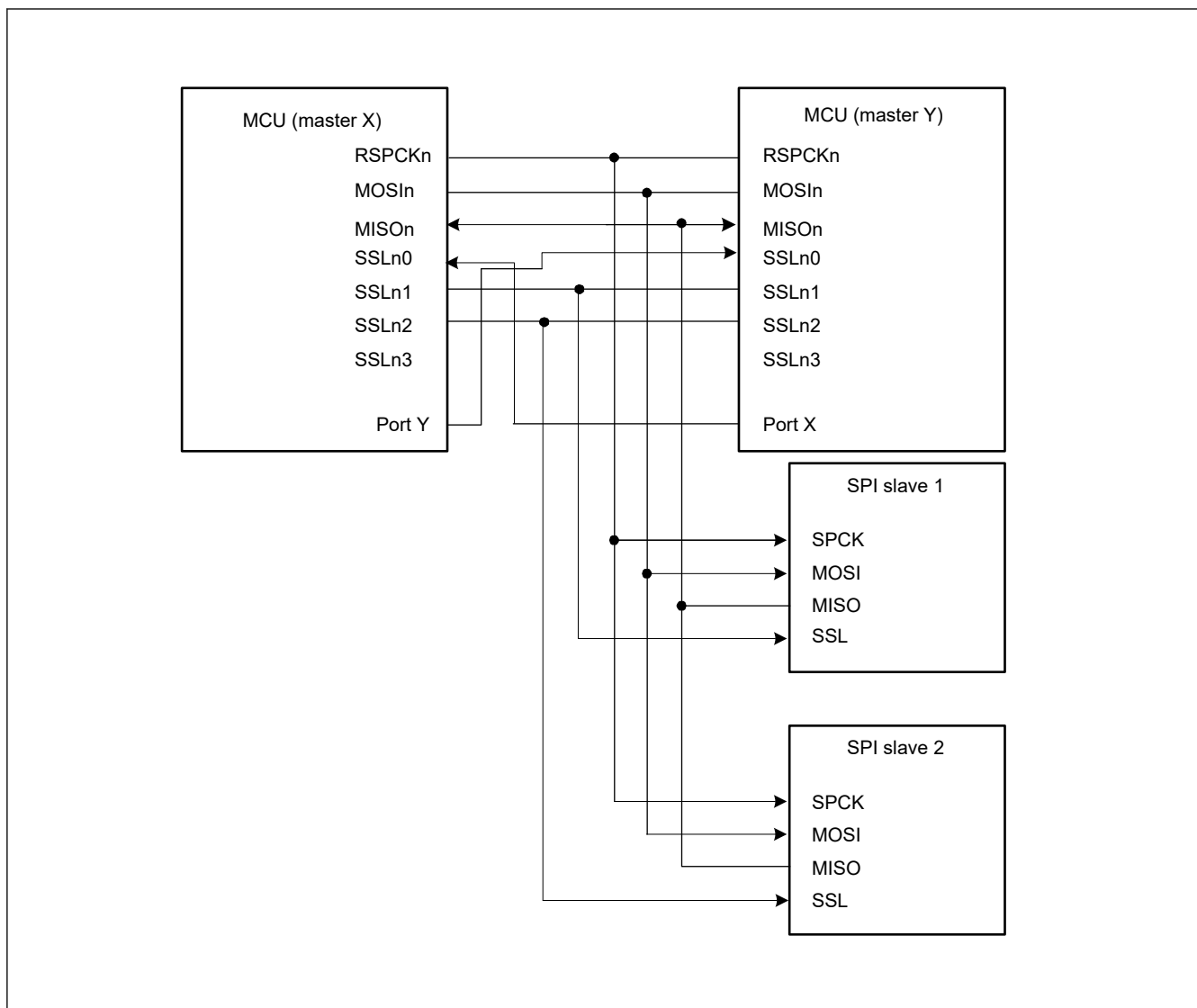


Figure 43.11 Multi-master/multi-slave configuration example with the MCU as a master

When setting TI SSP, enter the following levels for port X and port Y.

- Start of communication: the value of SPCR3.SSL0P of the other master.
- End of communication: the inverted value of SPCR3.SSL0P of the other master.

43.3.3.6 Master and Slave in Clock Synchronous Mode with the MCU Configured as a Master

Figure 43.12 shows a master and slave in clock synchronous mode configuration example where the MCU is used as a master. In this configuration, SSLni of the MCU (master) are not used.

The MCU (master) drives the RSPCKn and MOSIn signals. The SPI slave drives the MISO signal.

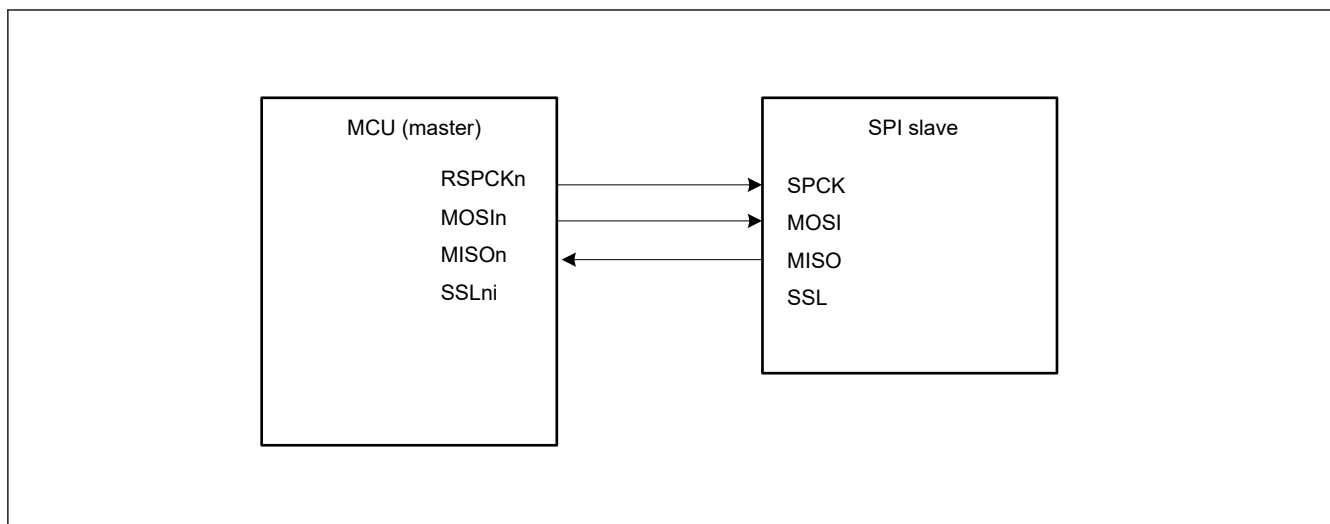


Figure 43.12 Clock synchronous master/slave configuration example with the MCU as a master

43.3.3.7 Master and Slave in Clock Synchronous Mode with the MCU as a Slave

Figure 43.13 shows a master and slave in clock synchronous mode configuration example where the MCU is used as a slave. When the MCU operates as a slave (clock synchronous operation), the MCU (slave) drives the MISOOn signal and the SPI master drives the SPCK and MOSI signals. In addition, SSLn0 to SSLn3 of the MCU (slave) are not used.

The MCU (slave) can only execute serial transfers in the single-slave configuration when the SPCMDm.CPHA bit is set to 1.

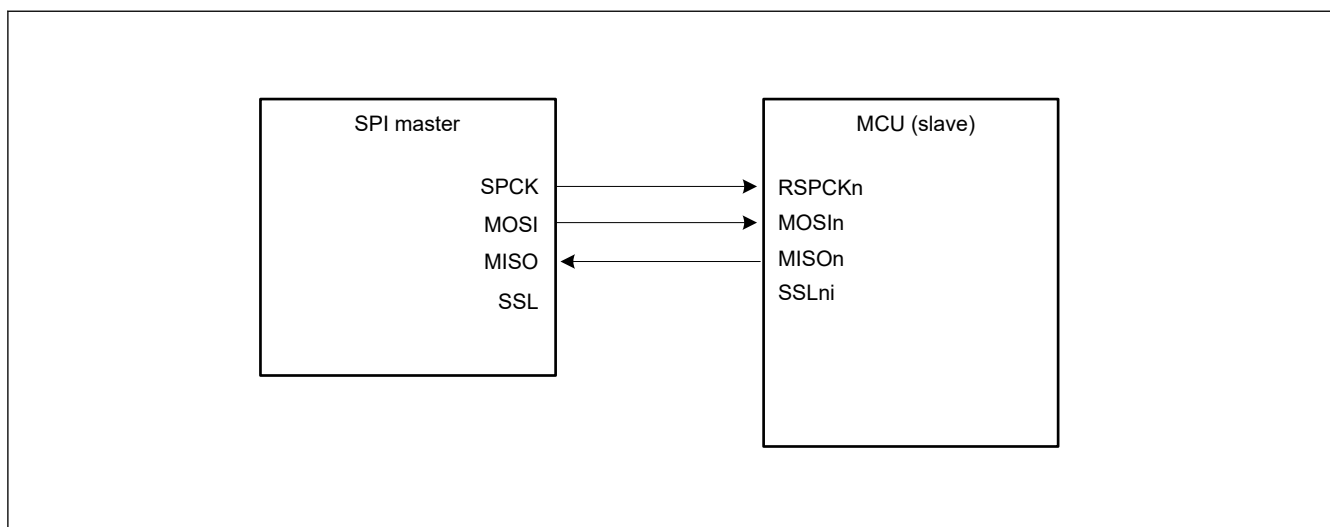


Figure 43.13 Clock synchronous master/slave configuration example with the MCU as a slave and CPHA = 1

43.3.4 Data Formats

The data format of the SPI depends on the settings in SPI Command Register m (SPCMDm) and the parity enable bit in SPI Control Register (SPCR.SPPE). Regardless of whether the MSB or LSB is first, the SPI treats the range from the LSB bit in the SPI Data Register (SPDR) to the bit associated with the selected data length, as transfer data.

This section shows the format of one frame of data before or after transfer.

Data format with parity disabled

When parity is disabled, transmission or reception of data proceeds with the length in bits selected in the SPI data length setting in SPI Command Register m (SPCMDm.SPB[4:0]).

Data format with parity enabled

When parity is enabled, transmission or reception of data proceeds with the length in bits selected in the SPI data length setting in SPI Command Register m (SPCMDm.SPB[4:0]). In this case, however, the last bit is a parity bit.

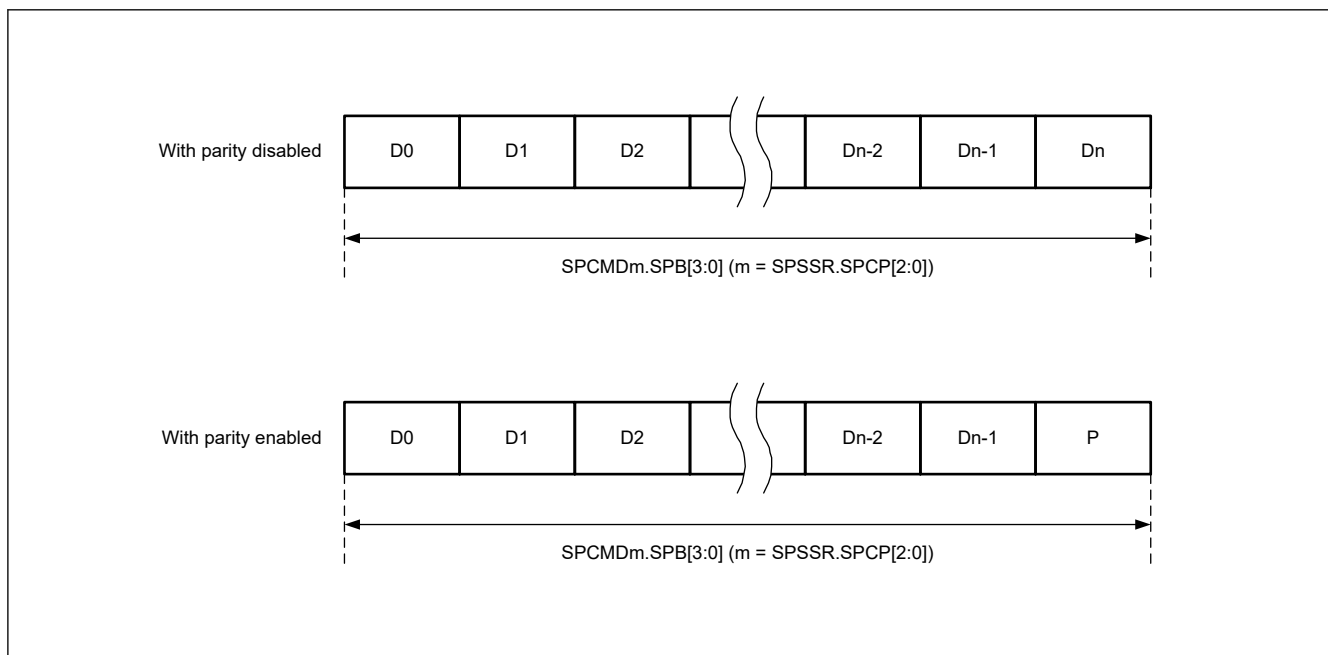


Figure 43.14 Data format with parity disabled and enabled

43.3.4.1 Operation When Parity is Disabled (SPCR.SPPE = 0)

When parity is disabled, data for transmission is copied to the shift register with no pre-processing. This section describes the connection between the SPI Data Register (SPDR) and the shift register in terms of the combination of MSB- or LSB-first order and data length.

(1) MSB-first transfer with 32-bit data

Figure 43.15 shows the operation of the SPI Data Register (SPDR) and the shift register in a transfer with parity disabled, an SPI data length of 32 bits, and MSB-first selected.

In transmission, bits T31 to T00 from the current stage of the transmit buffer are copied to the shift register. Data for transmission is shifted out from the shift register from T31 to T30, and continuing to T00.

In reception, received data is shifted in bit-by-bit through bit[0] of the shift register. When the R31 to R00 bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer.

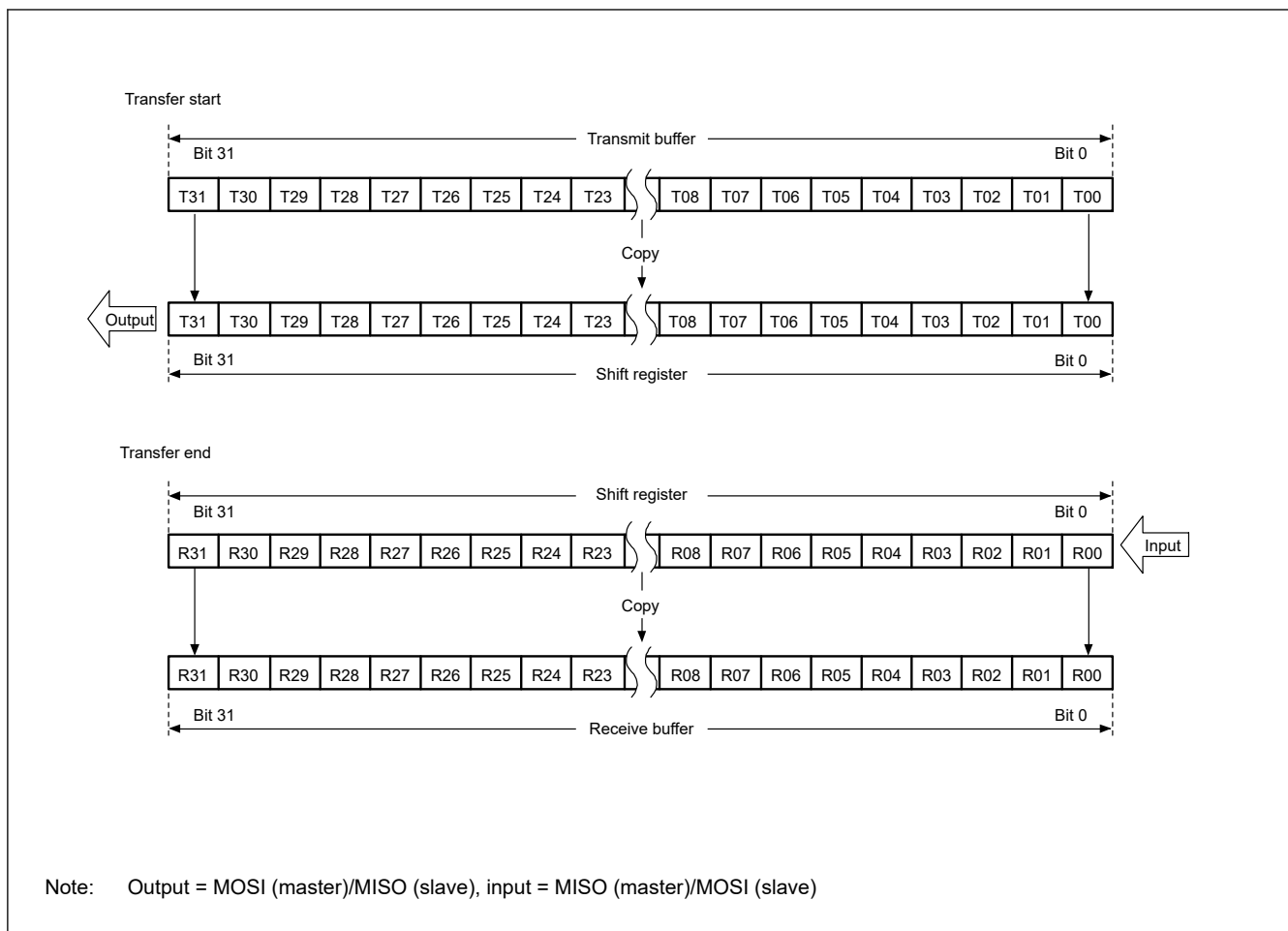


Figure 43.15 MSB-first transfer with 32-bit data and parity disabled

(2) MSB-first transfer with 24-bit data

Figure 43.16 shows the operation of the SPI Data Register (SPDR) and the shift register in a transfer with parity disabled, an SPI data length of 24 bits for an example that is not 32 bits, and MSB-first selected.

In transmission, the lower 24 bits (T23 to T00) from the current stage of the transmit buffer are copied to the shift register. Data for transmission is shifted out from the shift register from T23 to T22, and continuing to T00.

In reception, received data is shifted in bit-by-bit through bit[0] of the shift register. When the R23 to R00 bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer.

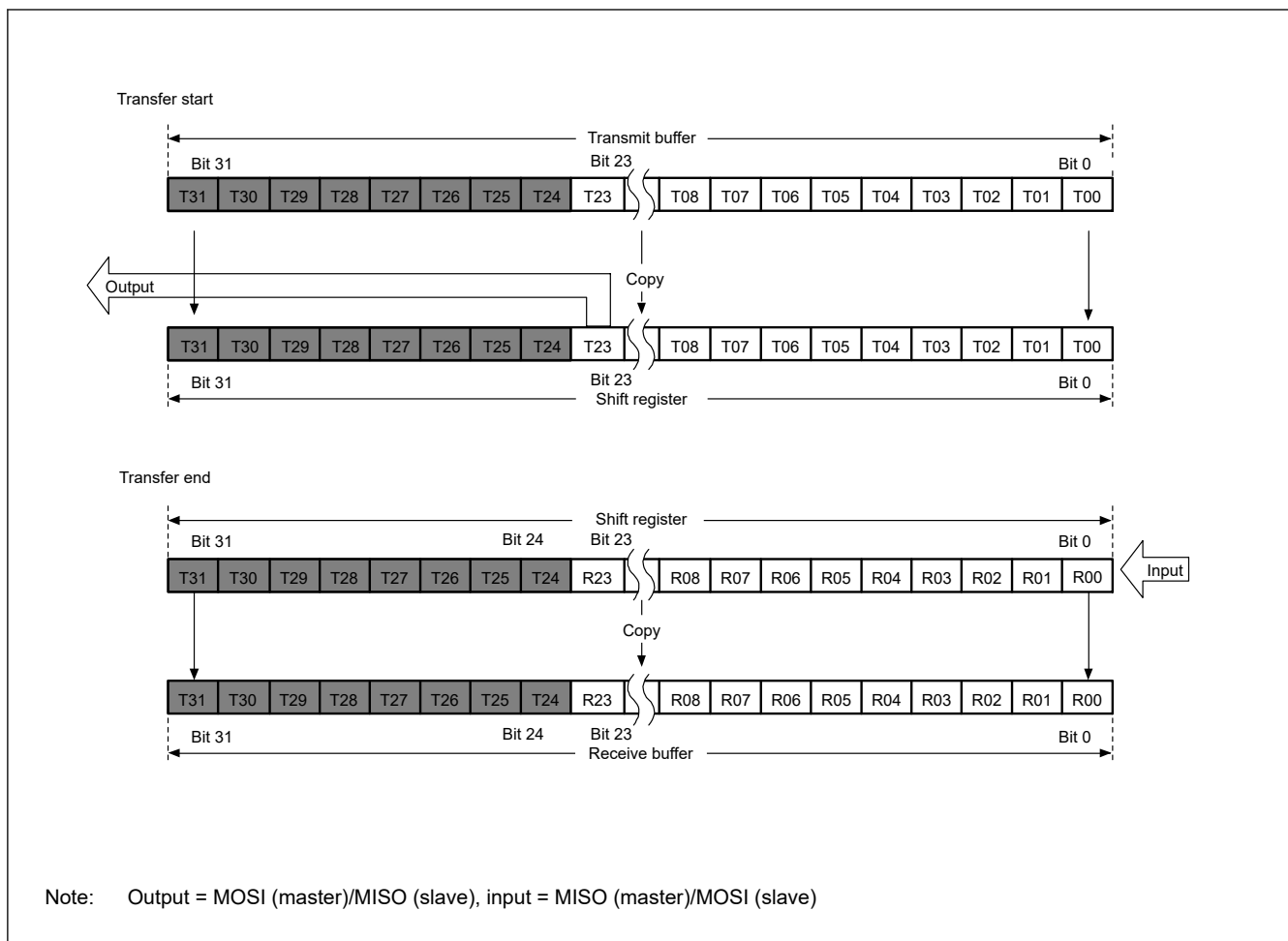


Figure 43.16 MSB-first transfer with 24-bit data and parity disabled

(3) LSB-first transfer with 32-bit data

Figure 43.17 shows the operation of the SPI Data Register (SPDR) and the shift register in a transfer with parity disabled, an SPI data length of 32 bits, and LSB-first selected.

In transmission, bits T31 to T00 from the current stage of the transmit buffer are reordered bit-by-bit to obtain the order T00 to T31 for copying to the shift register. Data for transmission is shifted out from the shift register in order from T00 to T01, and continuing to T31.

In reception, received data is shifted in bit-by-bit through bit[0] of the shift register. When the R00 to R31 bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer.

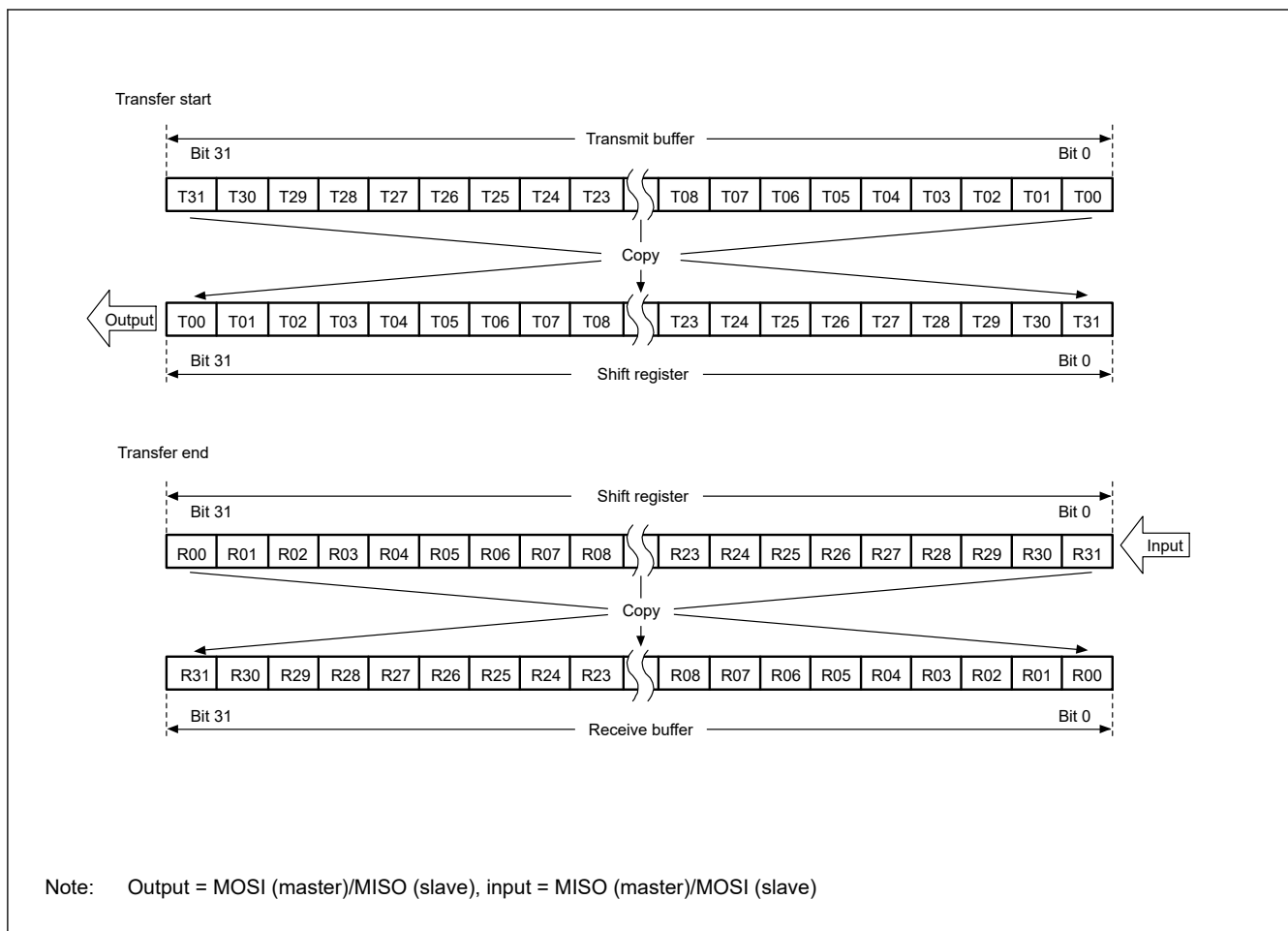


Figure 43.17 LSB-first transfer with 32-bit data and parity disabled

(4) LSB-first transfer with 24-bit data

Figure 43.18 shows the operation of the SPI Data Register (SPDR) and the shift register in transfers with parity disabled, an SPI data length of 24 bits for an example that is not 32, and LSB-first selected.

In transmission, the lower 24 bits (T23 to T00) from the current stage of the transmit buffer are reordered bit-by-bit to obtain the order T00 to T23 for copying to the shift register. Data for transmission is shifted out from the shift register from T00 to T01, and continuing to T23.

In reception, received data is shifted in bit-by-bit through bit[8] of the shift register. When the R00 to R23 bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer.

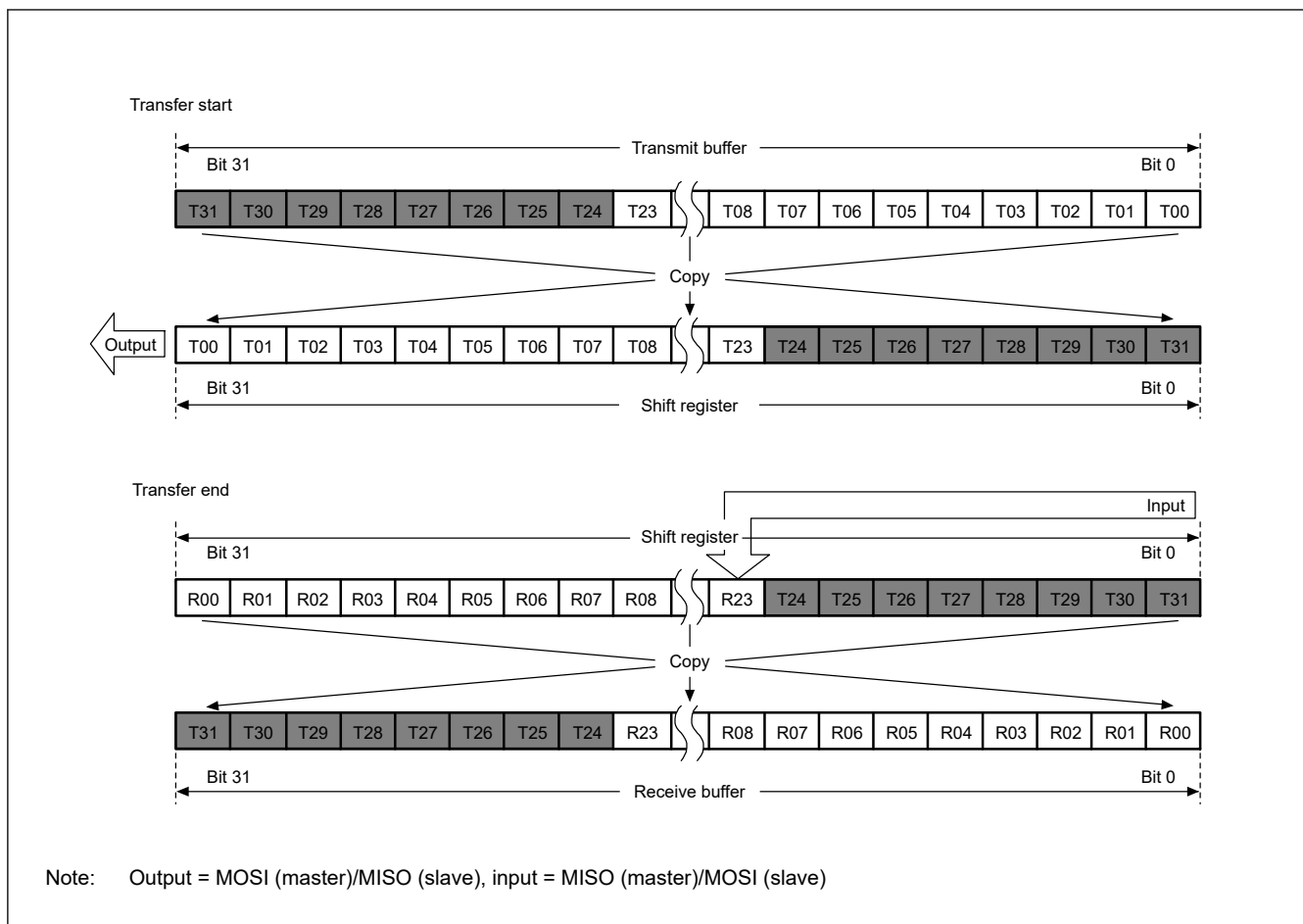


Figure 43.18 LSB-first transfer with 24-bit data and parity disabled

43.3.4.2 Operation When Parity is Enabled (SPCR.SPPE = 1)

When parity is enabled, the lowest-order bit of the data for transmission becomes a parity bit. Hardware calculates the value of the parity bit.

(1) MSB-first transfer with 32-bit data

Figure 43.19 shows the operation of the SPI Data Register (SPDR) and the shift register in a transfer with parity enabled, an SPI data length of 32 bits, and MSB-first selected.

In transmission, the value of the parity bit (P) is calculated from bits T31 to T01. This replaces the final bit, T00, and the whole value is copied to the shift register. Data is transmitted in the order T31, T30, ..., T01, and P.

In reception, received data is shifted in bit-by-bit through bit[0] of the shift register. When the R31 to P bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer. On copying of data to the shift register, the data from R31 to P is checked for parity.

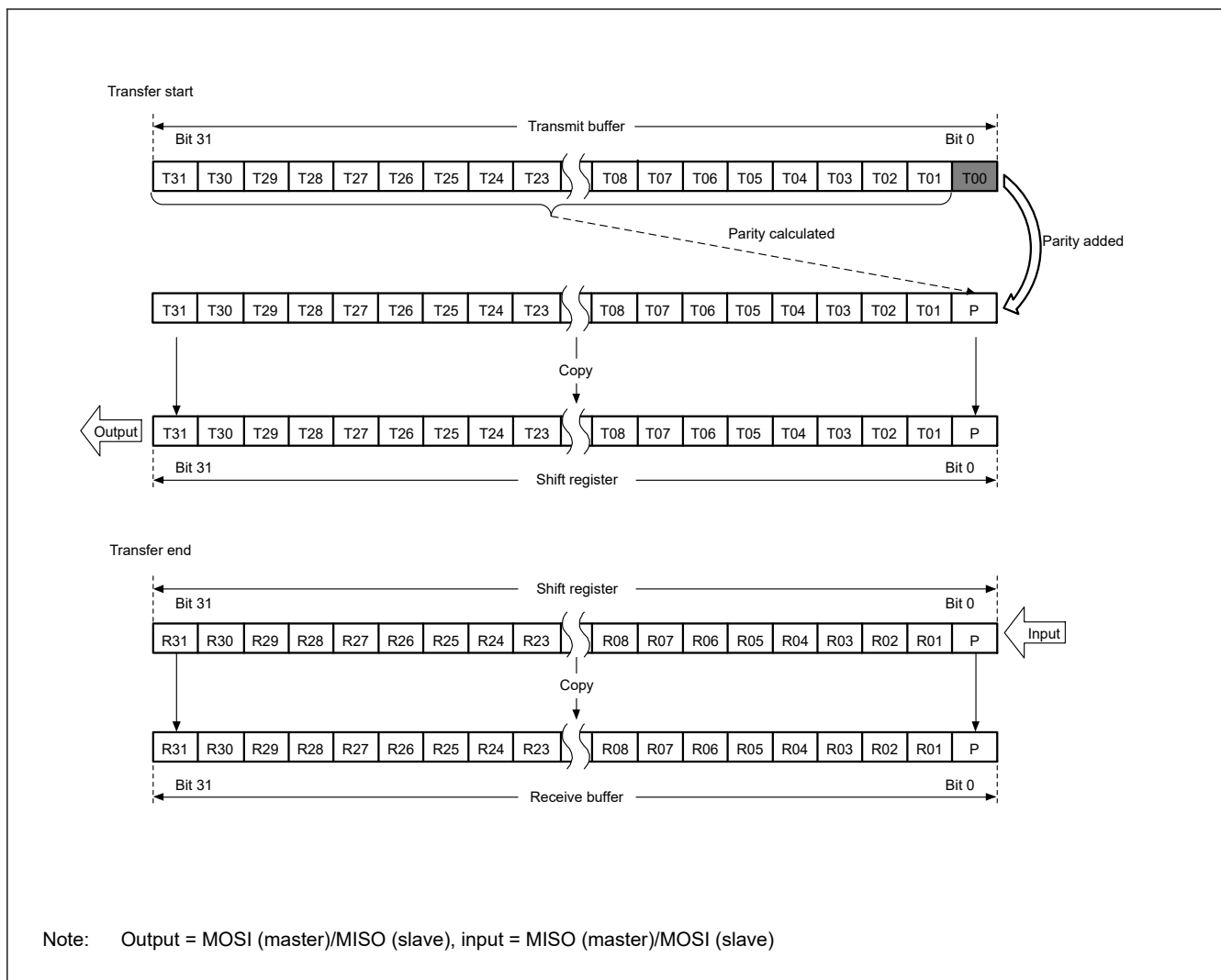


Figure 43.19 MSB-first transfer with 32-bit data and parity enabled

(2) MSB-first transfer with 24-bit data

Figure 43.20 shows the operation of the SPI Data Register (SPDR) and the shift register in a transfer with parity enabled, an SPI data length of 24 bits, and MSB-first selected.

In transmission, the value of the parity bit (P) is calculated from bits T23 to T01. This replaces the final bit, T00, and the whole value is copied to the shift register. Data is transmitted in the order T23, T22, ..., T01, and P.

In reception, received data is shifted in bit-by-bit through bit[0] of the shift register. When the R23 to P bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer. On copying of data to the shift register, the data from R23 to P is checked for parity.

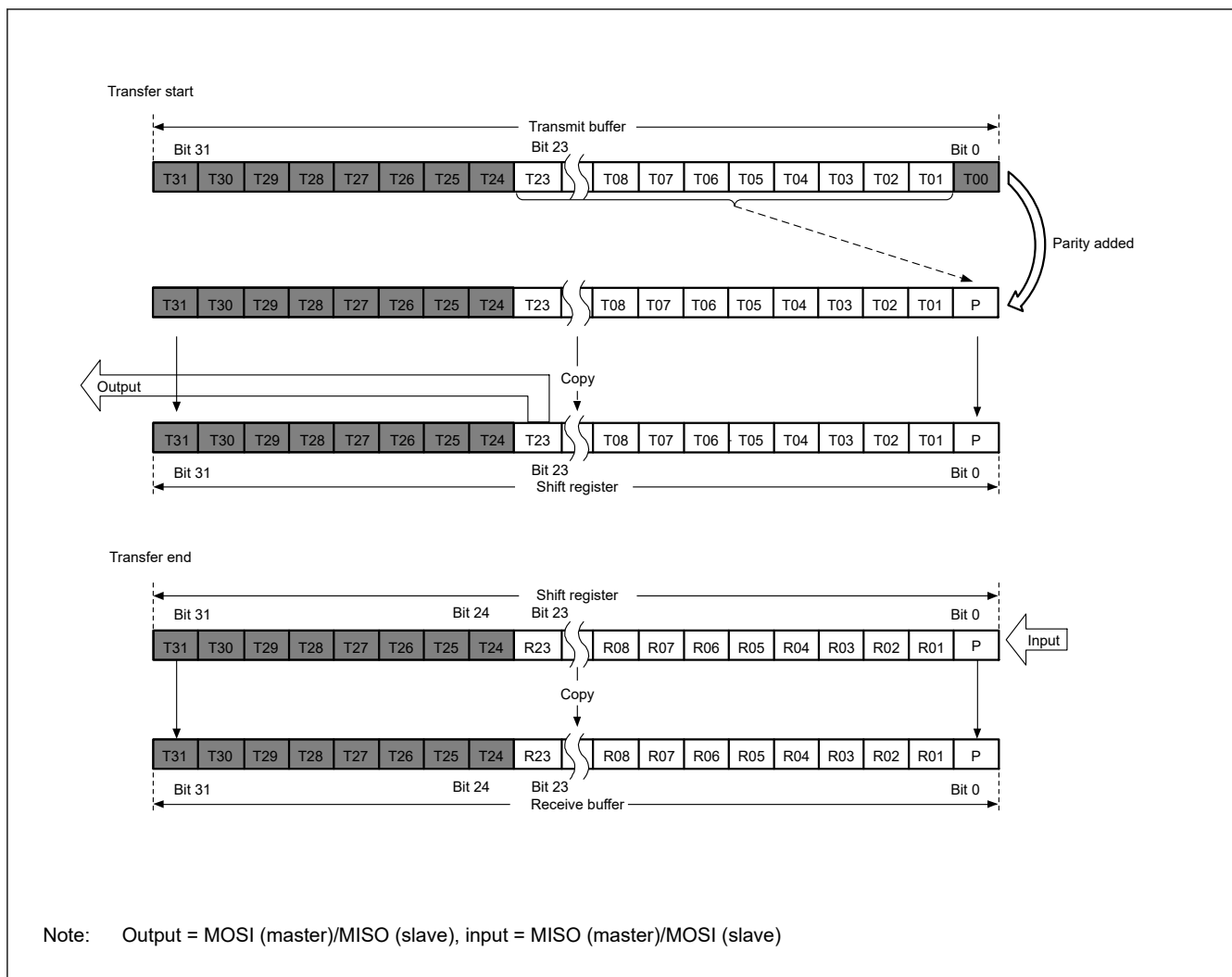


Figure 43.20 MSB-first transfer with 24-bit data and parity enabled

(3) LSB-first transfer with 32-bit data

Figure 43.21 shows the operation of the SPI Data Register (SPDR) and the shift register in a transfer with parity enabled, an SPI data length of 32 bits, and LSB-first selected.

In transmission, the value of the parity bit (P) is calculated from bits T30 to T00. This replaces the final bit, T31, and the whole value is copied to the shift register. Data is transmitted in the order T00, T01, ..., T30, and P.

In reception, received data is shifted in bit-by-bit through bit[0] of the shift register. When the R00 to P bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer. On copying of data to the shift register, the data from R00 to P is checked for parity.

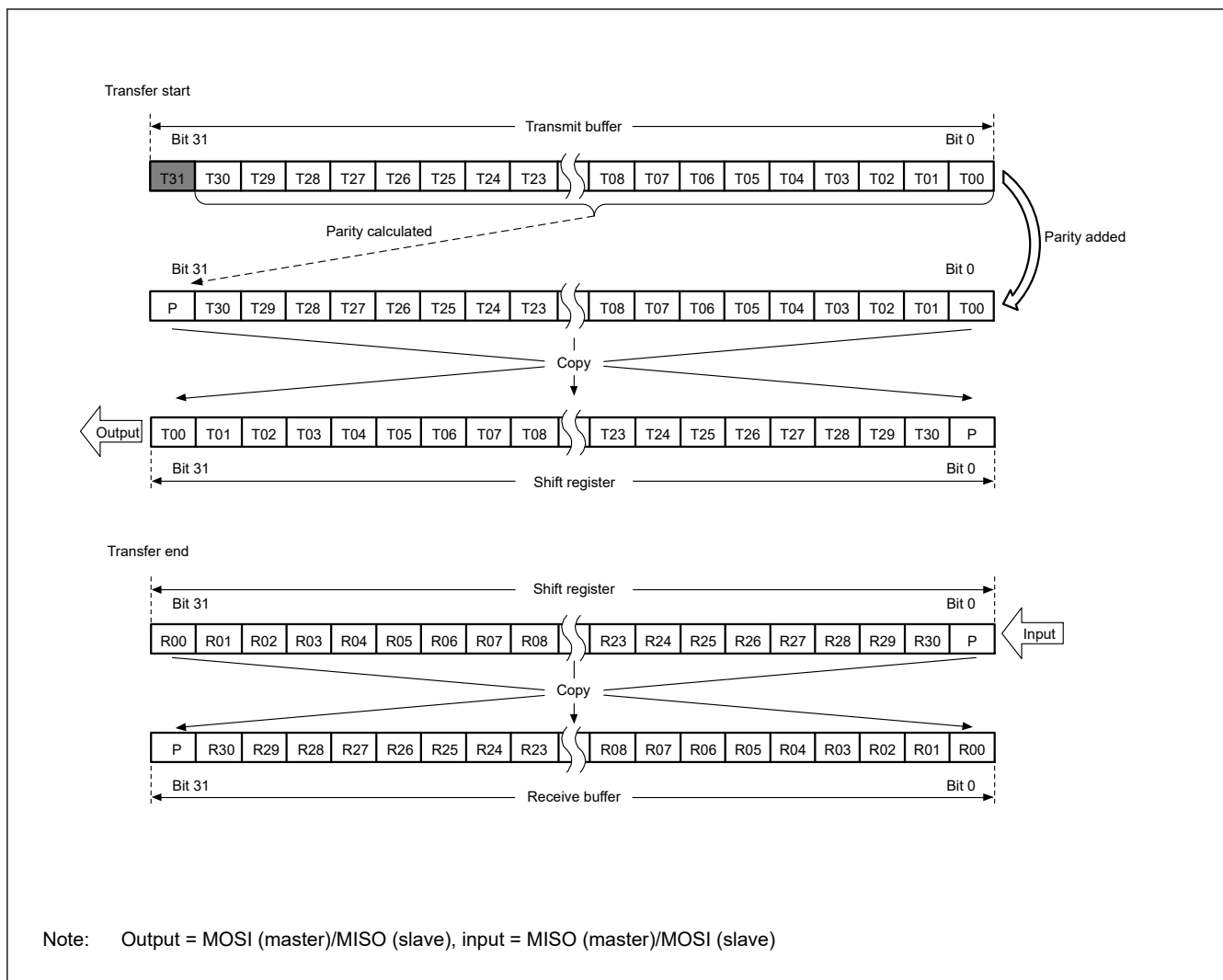


Figure 43.21 LSB-first transfer with 32-bit data and parity enabled

(4) LSB-first transfer with 24-bit data

Figure 43.22 shows the operation of the SPI Data Register (SPDR) and the shift register in a transfer with parity enabled, an SPI data length of 24 bits, and LSB-first selected.

In transmission, the value of the parity bit (P) is calculated from bits T22 to T0. This replaces the final bit, T23, and the whole value is copied to the shift register. Data is transmitted in the order T00, T01, ..., T22, and P.

In reception, received data is shifted in bit-by-bit through bit[8] of the shift register. When the R00 to P bits are collected after input of the required number of RSPCK cycles, the value in the shift register is copied to the receive buffer. On copying of data to the shift register, the data from R00 to P is checked for parity.

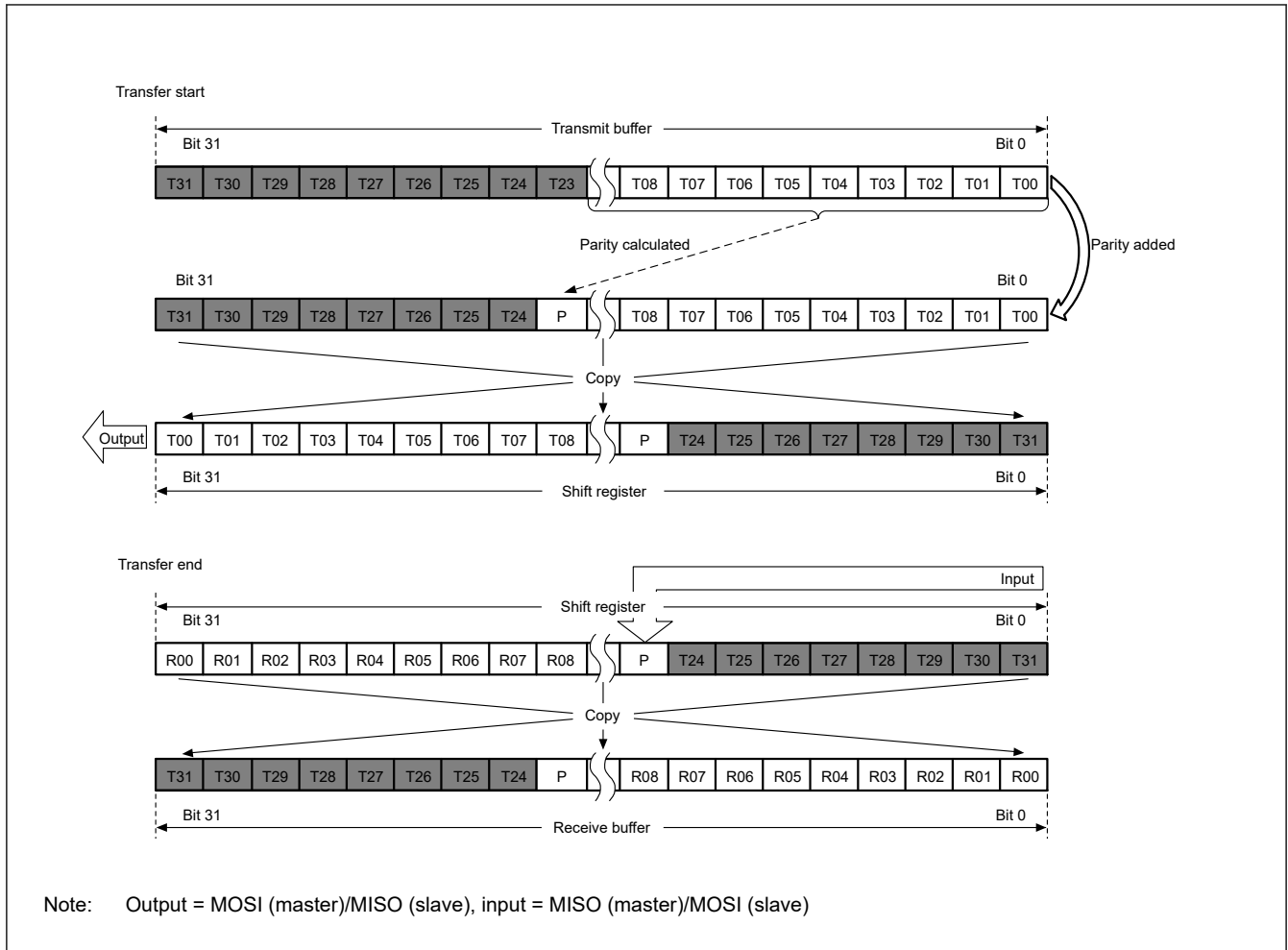


Figure 43.22 LSB-first transfer with 24-bit data and parity enabled

43.3.4.3 Byte Swap Transmission

When byte swapping is enabled, the data in the transmission buffer, swapped in 8-bit units, is copied to the shift register. Figure 43.23 shows the relationship between the SPDR (transmit buffer) and the shift register when transferring data with a 32-bit data length, using a combination of MSB / LSB first and with / without byte swap.

(1) MSB-first transfer. (When the byte swap is disabled.)

Data (Byte3 [T31 to T24] to Byte0 [T07 to T00]) in the transmit buffer are copied to the shift register.

Bit values in the shift register are shifted and transmitted in the order of T31 → T30 → ... → T00 as transmit data.

(2) MSB-first transfer. (When the byte swap is enabled.)

Byte values of the transmit buffer (Byte3 [T31 to T24] to Byte0 [T07 to T00]) are reversed in byte units and are copied to the shift register in the order of Byte0 [T07 to T00] to Byte3 [T31 to T24].

Bit values in the shift register are shifted and transmitted in the order of T07 → T06 → ... → T00 → T15 → T14 → ... → T08 → T23 → T22 → ... → T16 → T31 → T30 → ... → T24 as transmit data.

(3) LSB-first transfer. (When the byte swap is disabled.)

Bit values of the transmit buffer (Byte3 [T31 to T24] to Byte0 [T07 to T00]) are reversed in bit units and are copied to the shift register in the order of Byte0 [T00 to T07] to Byte3 [T24 to T31].

Bit values in the shift register are shifted and transmitted in the order of T00 → T01 → ... → T31 as transmit data.

(4) LSB-first transfer. (When the byte swap is enabled.)

Bit values of each byte of the transmit buffer (Byte3 [T31 to T24] to Byte0 [T07 to T00]) are reversed in bit units and are copied to the shift register in the order of Byte3 [T24 to T31] to Byte0 [T00 to T07].

Bit values in the shift register are shifted and transmitted in the order of T24 → T25 → ... → T31 → T16 → T17 → ... → T23 → T08 → T09 → ... → T15 → T00 → T01 → ... → T07 as transmit data.

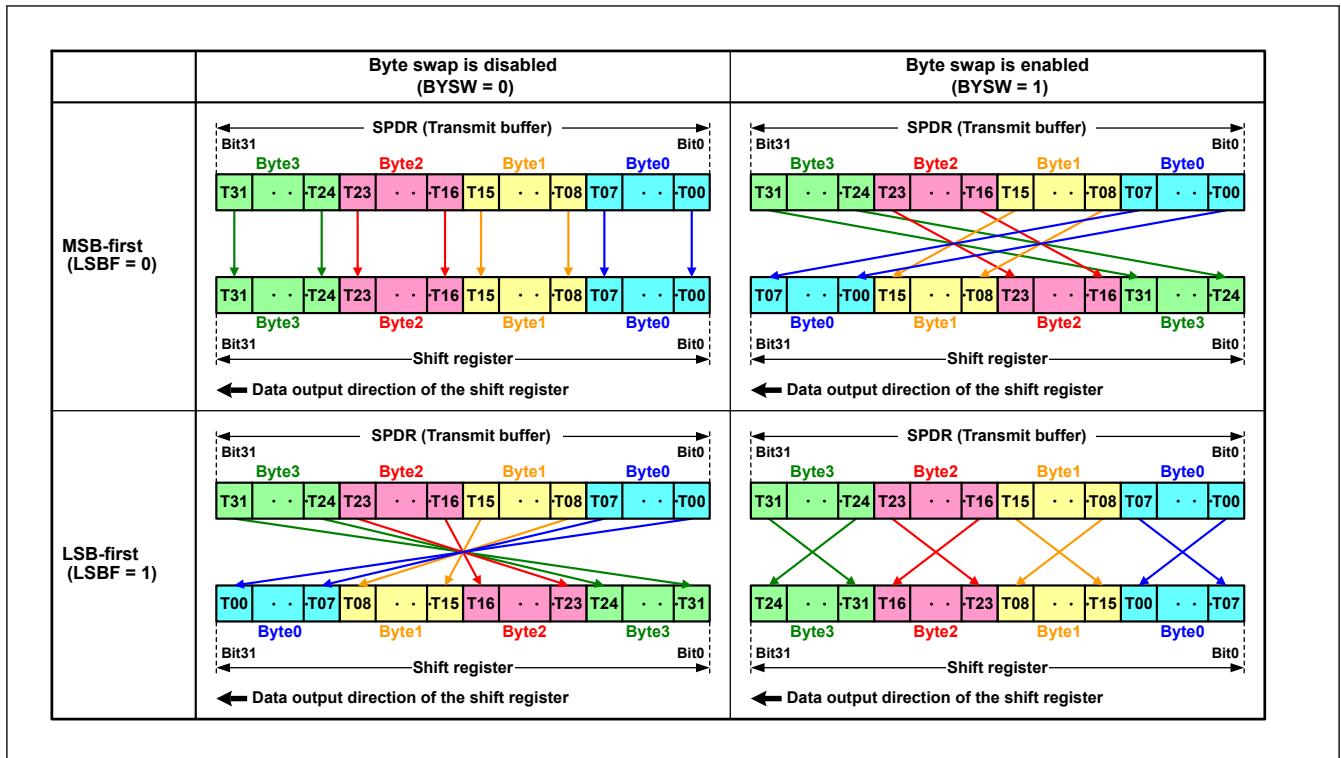


Figure 43.23 Byte swap with MSB/LSB transfer (32bit)

Figure 43.24 shows the relationship between the SPDR (transmit buffer) and the shift register when transferring data with a 16-bit data length, using a combination of MSB / LSB first and with / without byte swap.

1. MSB-first transfer. (When the byte swap is disabled.)
Data (Byte1 [T15 to T08] to Byte0 [T07 to T00]) in the transmit buffer are copied to the shift register in the order of Byte1 [T15 to T08] to Byte0 [T07 to T00], Byte1 [T15 to T08] to Byte0 [T07 to T00]. Bit values in the shift register are shifted and transmitted in the order of T15 → T14 → ... T00 as transmit data.
2. MSB-first transfer. (When the byte swap is enabled.)
Byte values of the transmit buffer (Byte1 [T15 to T08] to Byte0 [T07 to T00]) are reversed in byte units and are copied to the shift register in the order of Byte0 [T07 to T00] to Byte1 [T15 to T08], Byte0 [T07 to T00] to Byte1 [T15 to T08]. Bit values in the shift register are shifted and transmitted in the order of T07 → T06 → ... T00 → T15 → T14 → ... T08 as transmit data.
3. LSB-first transfer. (When the byte swap is disabled.)
Bit values of the transmit buffer (Byte1 [T15 to T08] to Byte0 [T07 to T00]) are reversed in bit units and are copied to the shift register in the order of Byte0 [T00 to T07] to Byte1 [T08 to T15], Byte0 [T00 to T07] to Byte1 [T08 to T15]. Bit values in the shift register are shifted and transmitted in the order of T00 → T01 → ... T15 as transmit data.
4. LSB-first transfer. (When the byte swap is enabled.)
Bit values of each byte of the transmit buffer (Byte1 [T15 to T08] to Byte0 [T07 to T00]) are reversed in bit units and are copied to the shift register in the order of Byte1 [T08 to T15] to Byte0 [T00 to T07], Byte1 [T08 to T15] to Byte0 [T00 to T07]. Bit values in the shift register are shifted and transmitted in the order of T08 → T09 → ... T15 → T00 → T01 → ... T07 as transmit data.

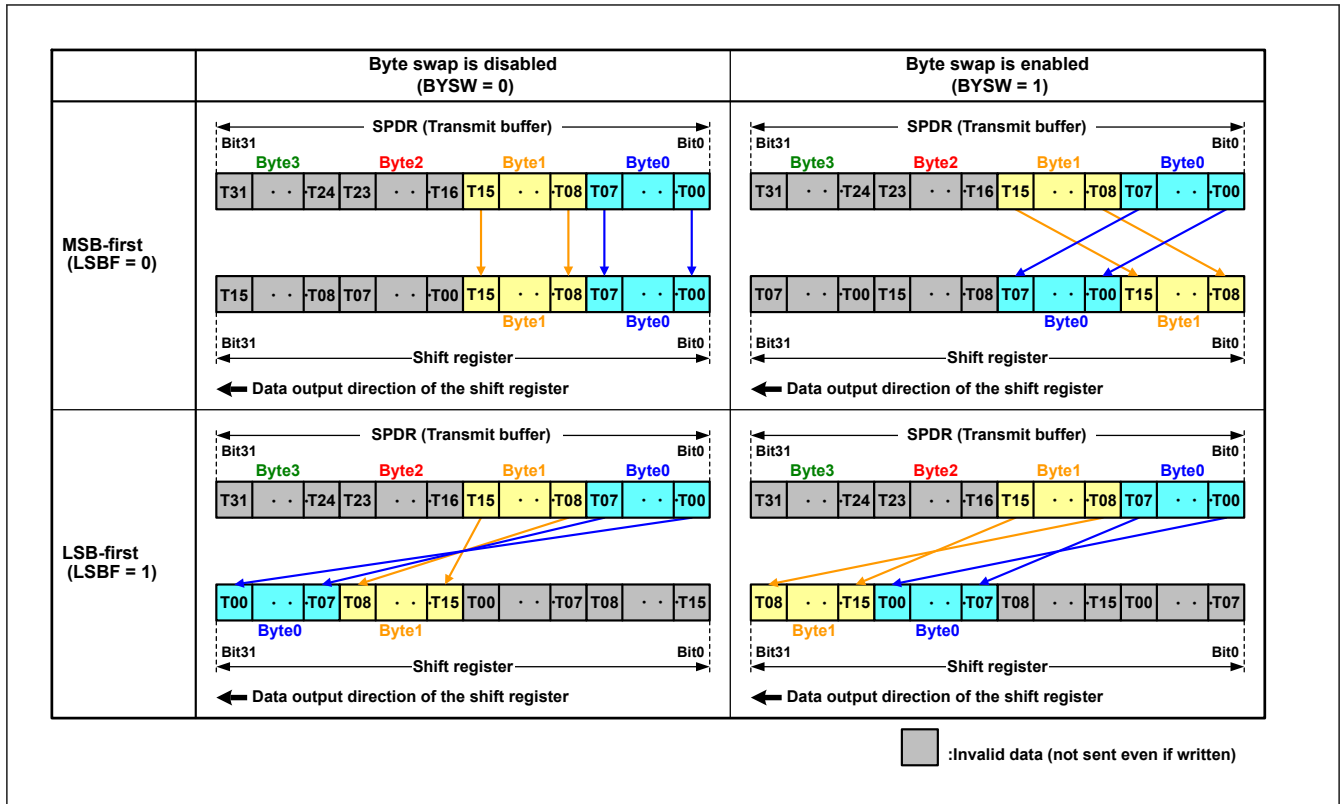


Figure 43.24 Byte swap with MSB/LSB transfer (16bit)

- Note:
1. When using the byte swap, set 16 bits or 32 bits to the data length (SPCMDm.SPB[4:0] setting). If setting the other length, the behavior is not guaranteed.
 2. When the byte swap is valid, set the parity function as invalid (SPCR.SPPE bit = 0). If setting the parity function as valid (SPPE bit = 1), the behavior is not guaranteed.
 3. Set SPDCR.BYSW bit, when SPCR.SPE bit is 0. If rewriting BYSW bit, when SPE bit is 1, the behavior after it is not guaranteed.

43.3.4.4 Byte Swap Reception

When byte swap is enabled, the data in the shift register, swapped in 8-bit units, is copied to the receive buffer. Figure 43.25 shows the relationship between the shift register and SPDR (reception buffer) when transferring data with a 32-bit data length, using a combination of MSB / LSB first and with / without byte swap.

(1) MSB-first transfer (When the byte swap is disabled)

The first received data (R31) is stored in bit 0 of the shift register, and received data is shifted in the order of R31 → R30 → ... → R00.

When necessary RSPCK cycles are input and data is stored from Byte3 [R31 to R24] to Byte0 [R07 to R00], the shift register value is copied to the receive buffer.

(2) MSB-first transfer (When the byte swap is enabled)

The first received data (R07) is stored in bit 0 of the shift register, and received data is shifted in the order of R07 → R06 → ... → R00 → R15 → R14 → ... → R08 → R23 → R22 → ... → R16 → R31 → R30 → ... → R24.

When necessary RSPCK cycles are input and data is stored from Byte0 [R07 to R00] to Byte3 [R31 to R24], byte values in the shift register are reversed in byte units and are copied to the receive buffer in the order of Byte3 [R31 to R24] to Byte0 [R07 to R00].

(3) LSB-first transfer (When the byte swap is disabled)

The first received data (R00) is stored in bit 0 of the shift register, and received data is shifted in the order of R00 → R01 → ... → R31.

When necessary RSPCK cycles are input and data is stored from Byte0 [R00 to R07] to Byte3 [R24 to R31], bit values in the shift register are reversed in bit units and are copied to the receive buffer in the order of Byte3 [R31 to R24] to Byte0 [R07 to R00].

(4) LSB-first transfer (When the byte swap is enabled)

The first received data (R24) is stored in bit 0 of the shift register, and received data is shifted in the order of R24 → R25 → ... → R31 → R16 → R17 → ... → R23 → R08 → R09 → ... → R15 → R00 → R01 → ... → R07.

When necessary RSPCK cycles are input and data is stored from Byte3 [R24 to R31] to Byte0 [R00 to R07], bit values of each byte in the shift register are reversed in bit units and are copied to the receive buffer in the order of Byte3 [R31 to R24] to Byte0 [R07 to R00].

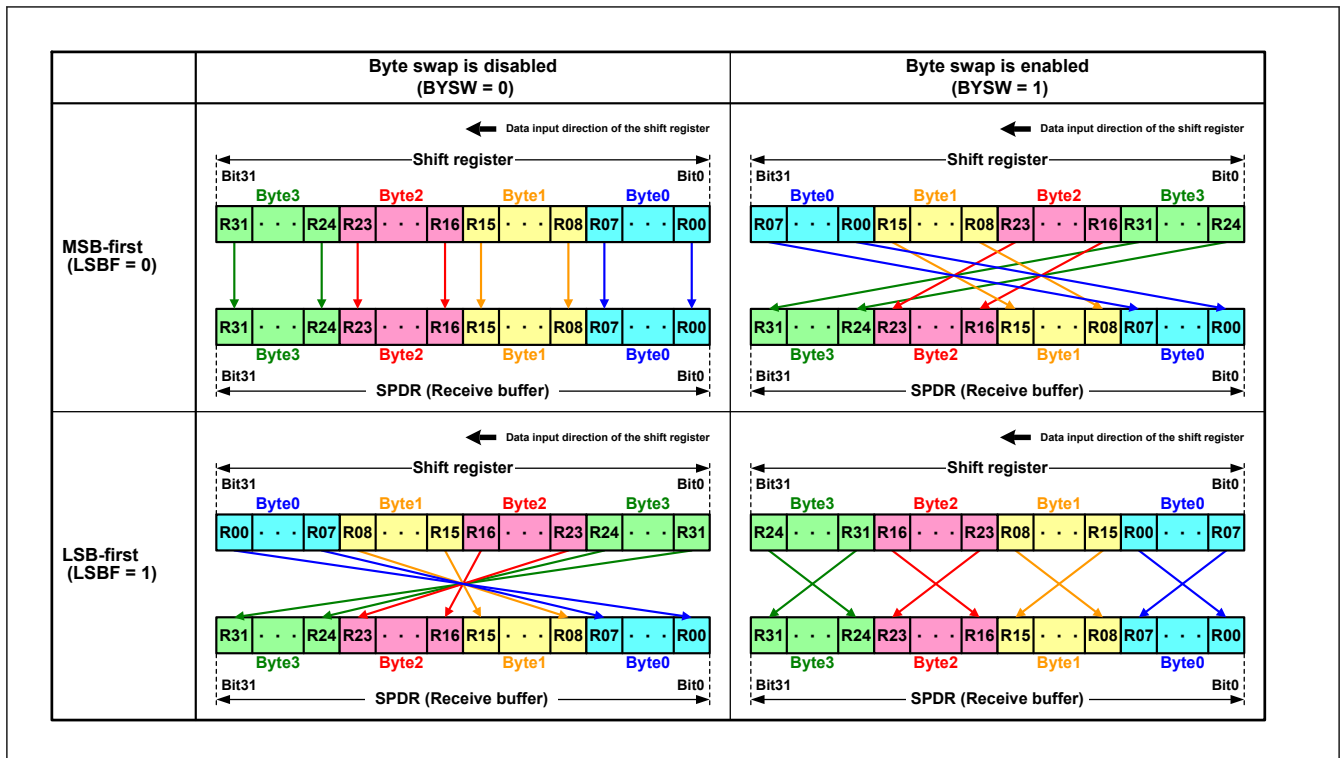


Figure 43.25 Byte swap with MSB/LSB transfer (32-bit)

Figure 43.26 shows the relationship between the shift register and SPDR (reception buffer) when transferring data with a 16-bit data length, using a combination of MSB / LSB first and with / without byte swap.

1. MSB-first transfer (when the byte swap is disabled)
The first received data (R15) is stored in bit 0 of the shift register, and received data is shifted in the order of R15 → R14 → ... R00. When necessary RSPCK cycles are input and data is stored from Byte3 [R31 to R24] to Byte0 [R07 to R00], the shift register value is copied to the receive buffer.
2. MSB-first transfer (when the byte swap is enabled)
The first received data (R07) is stored in bit 0 of the shift register, and received data is shifted in the order of R07 → R06 → ... R00 → R15 → R14 → ... R08. When necessary RSPCK cycles are input and data is stored from Byte0 [R07 to R00] to Byte1 [R15 to R08], byte values in the shift register are reversed in byte units and are copied to the receive buffer in the order of Byte3 [R31 to R24] to Byte0 [R07 to R00].
3. LSB-first transfer (when the byte swap is disabled)
The first received data (R00) is stored in bit 15 of the shift register, and received data is shifted in the order of R00 → R01 → ... R07 → R08 → R09 → ... R15. When necessary RSPCK cycles are input and data is stored from Byte0 [R00 to R07] to Byte1 [R08 to R15], bit values in the shift register are reversed in bit units and are copied to the receive buffer in the order of Byte3 [R31 to R24] to Byte0 [R07-R00].
4. LSB-first transfer (when the byte swap is enabled)
The first received data (R08) is stored in bit 15 of the shift register, and received data is shifted in the order of R08 → R09 → ... R15 → R00 → R01 → ... R07. When necessary RSPCK cycles are input and data is stored from Byte1 [R08

to R15] to Byte0 [R00 to R07], bit values of each byte in the shift register are reversed in bit units and are copied to the receive buffer in the order of Byte3 [R31 to R24] to Byte0 [R07 to R00].

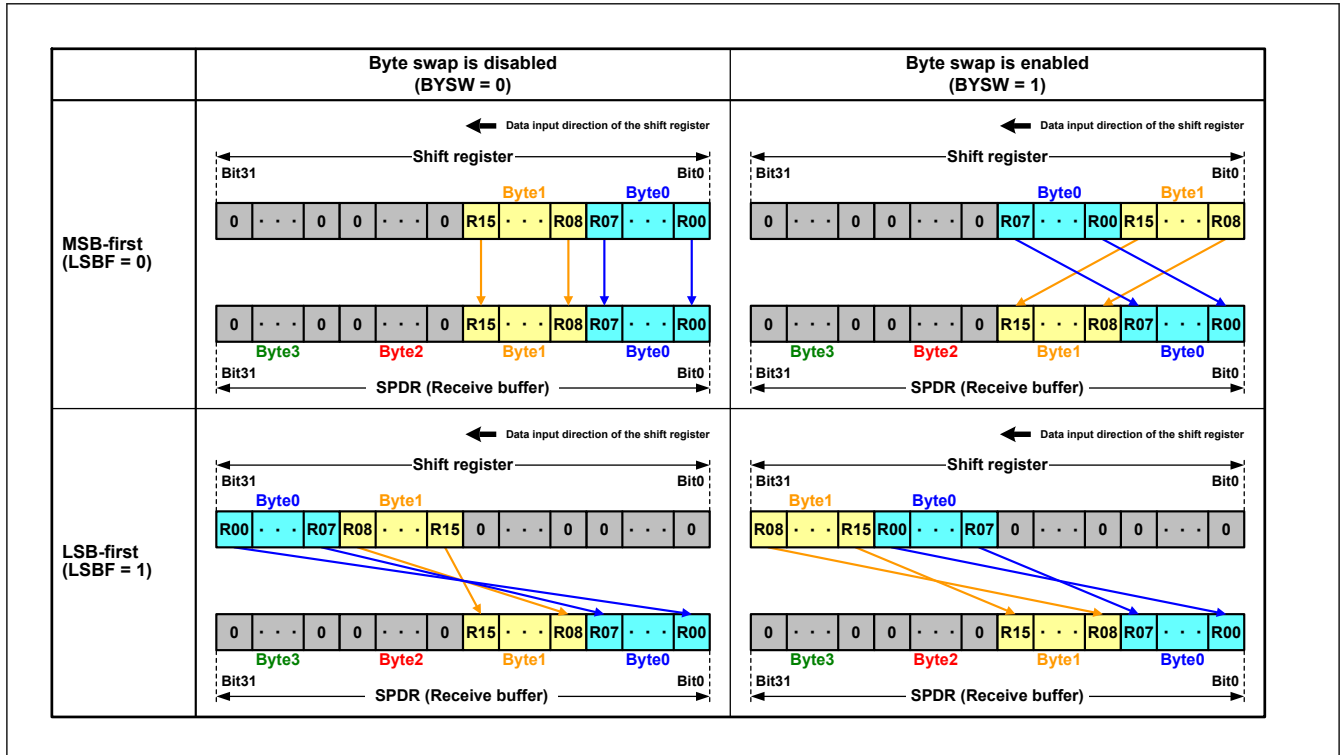


Figure 43.26 Byte swap with MSB/LSB transfer (16-bit)

- Note:
1. When using the byte swap, set 16 bits or 32 bits to the data length (SPCMDm.SPB[4:0] setting). If setting the other length, the behavior is not guaranteed.
 2. When the byte swap is valid, set the parity function as invalid (SPCR.SPPE bit = 0). If setting the parity function as valid (SPPE bit = 1), the behavior is not guaranteed.
 3. Set SPDCR.BYSW bit, when SPCR.SPE bit is 0. If rewriting BYSW bit, when SPE bit is 1, the behavior after it is not guaranteed.

43.3.5 Transfer Formats

43.3.5.1 When CPHA = 0

Figure 43.27 shows an example transfer format for the serial transfer of 8-bit data when the SPCMDm.CPHA bit is 0. Do not perform clock synchronous operation (SPCR.SPMS = 1) when the SPI operates in slave mode (SPCR.MSTR = 0) and the CPHA bit is 0. In Figure 43.27, RSPCKn (CPOL = 0) indicates the RSPCKn signal waveform when the SPCMDm.CPOL bit is 0, and RSPCKn (CPOL = 1) indicates the RSPCKn signal waveform when the CPOL bit is 1. The sampling timing represents the timing at which the SPI fetches serial transfer data into the shift register. The I/O directions of the signals depend on the SPI settings. For details, see section 43.3.2. Controlling the SPI Pins.

When the SPCMDm.CPHA bit is 0, the driving of valid data to the MOSIn and MISOOn signals begins at an SSLni signal assertion. The first RSPCKn signal change that occurs after the SSLni signal assertion becomes the first transfer data fetch. After this, data is sampled every 1 RSPCKn cycle. The change timing for MOSIn and MISOOn signals is 1/2 RSPCK cycles after the transfer data fetch timing. The CPOL bit setting does not affect the RSPCKn signal operation timing as it only affects the signal polarity.

t1 denotes the RSPCK delay, the period from an SSLni signal assertion to RSPCKn oscillation. t2 denotes the SSL negation delay, the period from the termination of RSPCKn oscillation to an SSLni signal negation. t3 denotes the next-access delay, the period in which SSLni signal assertion is suppressed for the next transfer after the end of serial transfer. t1, t2, and t3 are controlled by a master device running on the SPI system. For a description of t1, t2, and t3 when the SPI is in master mode, see section 43.3.12.1. Master Mode Operation.

[In the Motorola-SPI case]

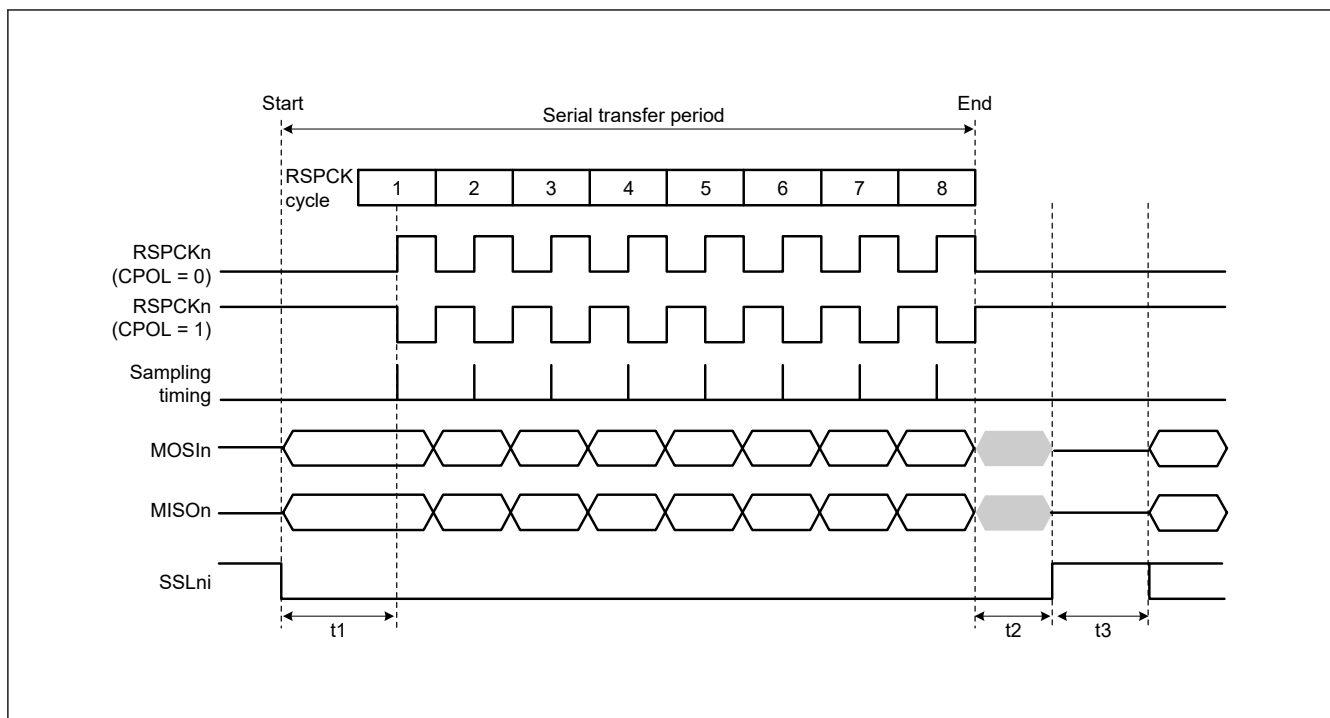


Figure 43.27 SPI transfer format when CPHA = 0, SPFRF = 0

[In TI-SSP case]

Not supported in CPHA = 0

43.3.5.2 When CPHA = 1

Figure 43.28 shows an example transfer format for the serial transfer of 8-bit data when the SPCMDm.CPHA bit is 1. However, when the SPCR.SPMS bit is 1, the SSLni signals are not used, and only the three signals RSPCKn, MOSIn, and MISO handle communications. In Figure 43.28, RSPCKn (CPOL = 0) indicates the RSPCKn signal waveform when the SPCMDm.CPOL bit is 0 and RSPCKn (CPOL = 1) indicates the RSPCKn signal waveform when the CPOL bit is 1. The sampling timing represents the timing at which the SPI fetches serial transfer data into the shift register. The I/O directions of the signals depend on the SPI mode (master or slave mode). For details, see [section 43.3.2. Controlling the SPI Pins](#).

When the SPCMDm.CPHA bit is 1, the driving of invalid data to the MISO signal begins at an SSLni signal assertion. The output of valid data to the MOSIn and MISO signals begins at the first RSPCKn signal change that occurs after the SSLni signal assertion. After this, data is updated every 1 RSPCK cycle. The transfer data fetch timing is 1/2 RSPCK cycles after the data update timing. The SPCMDm.CPOL bit setting does not affect the RSPCKn signal operation timing. It only affects the signal polarity.

t1, t2, and t3 are the same as those when CPHA = 0. For a description of t1, t2, and t3 when the SPI of the MCU is in master mode, see [section 43.3.12.1. Master Mode Operation](#).

[In the Motorola-SPI case]

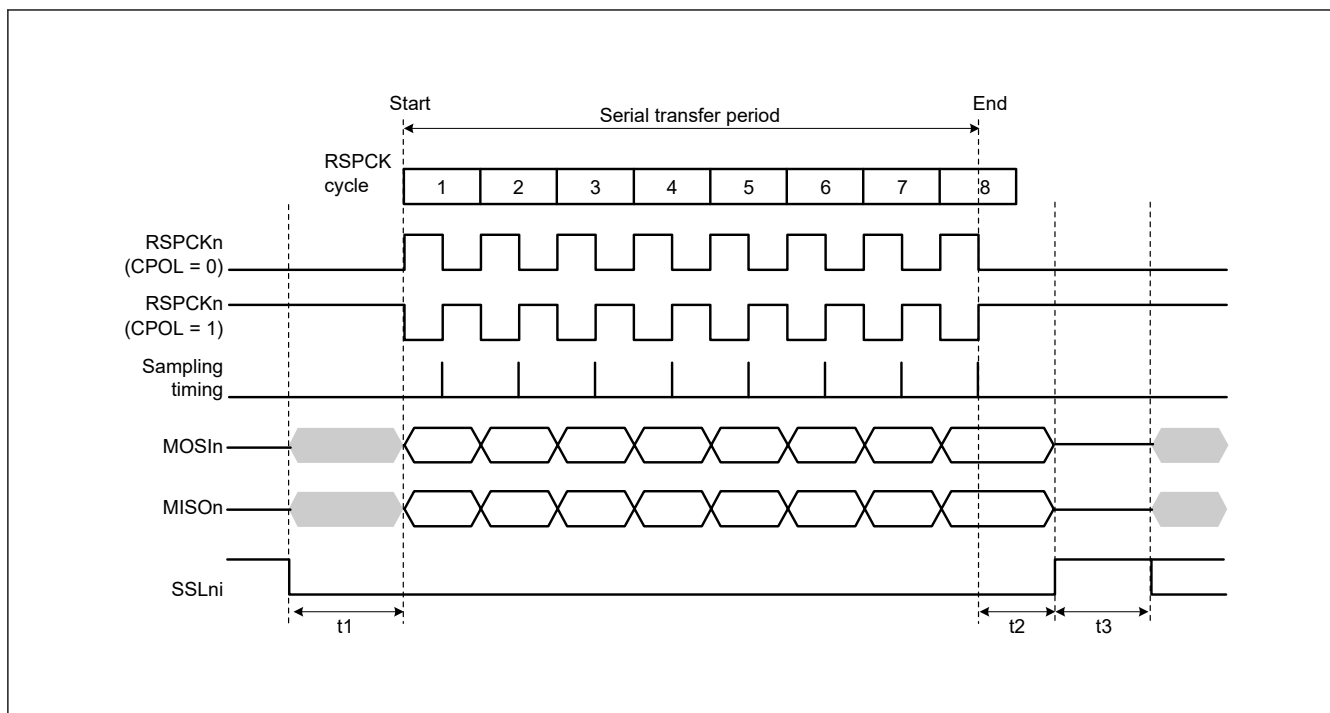


Figure 43.28 SPI transfer format when CPHA = 1, SPFRF = 0

[In the TI-SSP case]

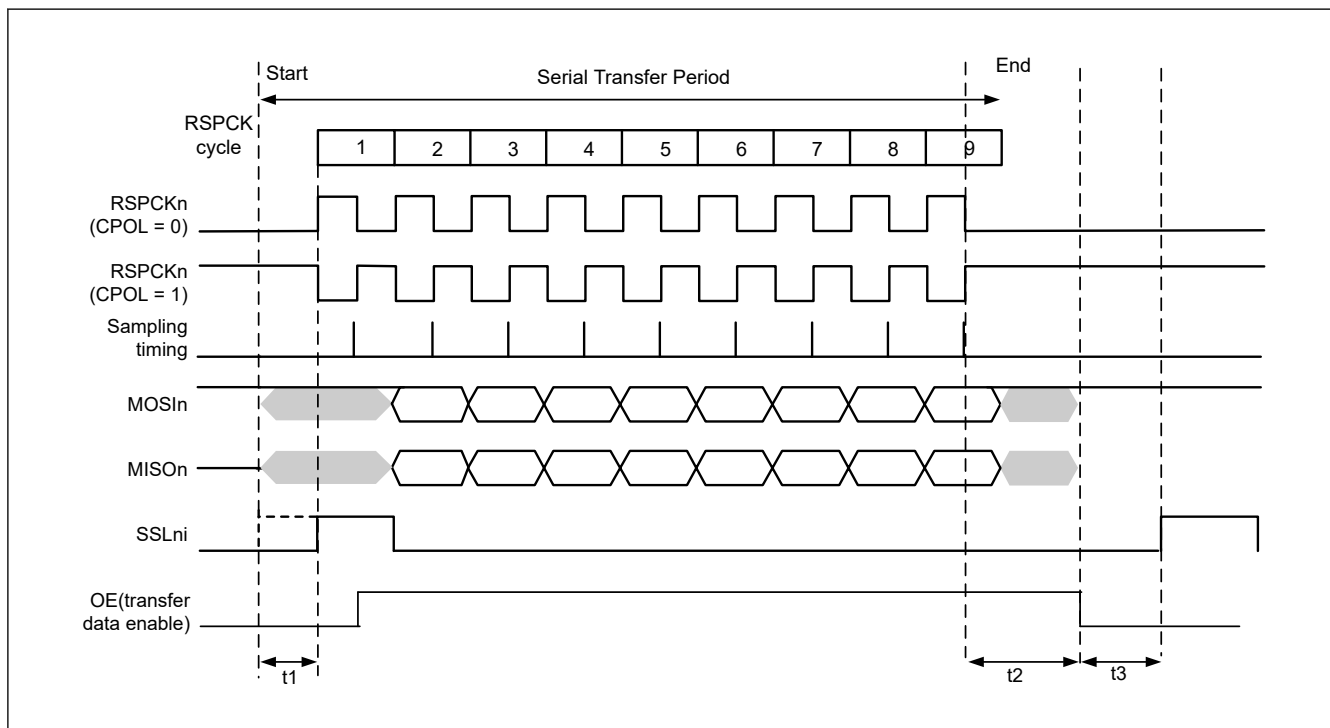


Figure 43.29 SPI transfer format when CPHA = 1, SPFRF = 1

43.3.6 Communication Operating Mode

Transmit-Receive serial communication, transmit-only operation, and Receive-only operation are selected by setting the Communication Mode Select bits (TXMD [1:0]) of the SPI control register (SPCR).

SPDR access described in [Figure 43.30](#), [Figure 43.31](#), [Figure 43.32](#) shows an access to the SPI data register (SPDR). W shows a write cycle.

43.3.6.1 Transmit-Receive Serial Communication (TXMD[1:0] = 00b)

Figure 43.30 shows an example of operation when the communication mode select bit (TXMD[1:0]) in the SPI control register (SPCR) is set to 00b. In the example, the SPI performs an 8-bit serial transfer when the SPDCR2.TTRG is 0, the SPDCR2.RTRG is FIFO stage - 1, the SPCMDm.CPHA bit is 1, and the SPCMDm.CPOL bit is 0. The numbers given for RSPCKn in the waveform represent the number of RSPCK cycles, such as the number of transferred bits.

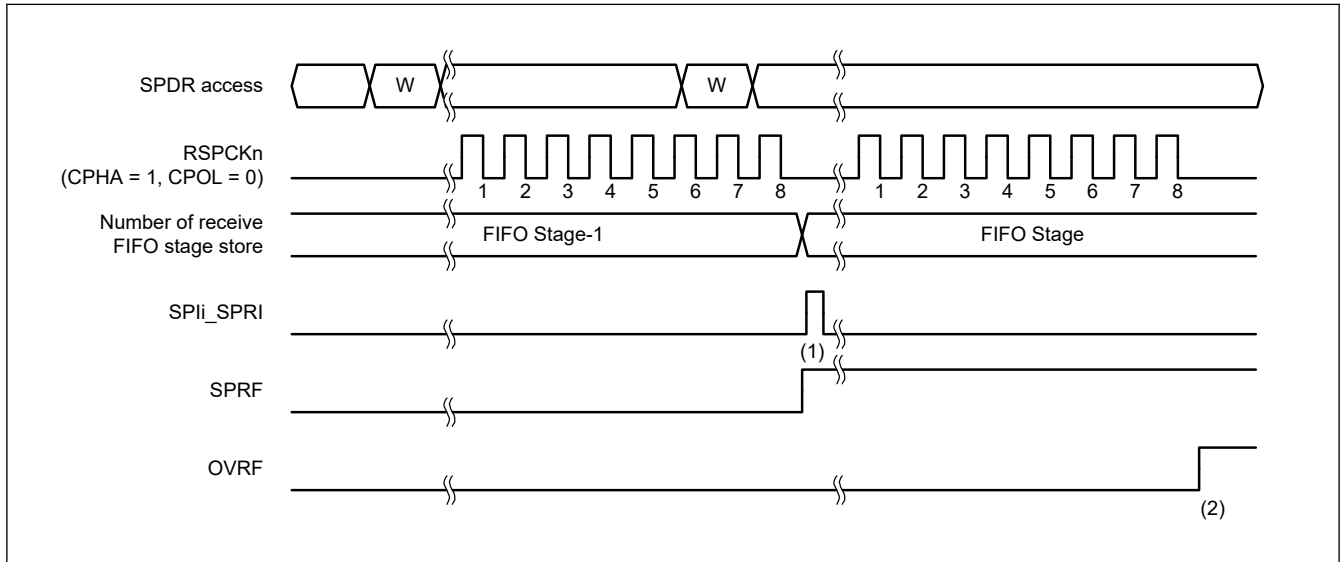


Figure 43.30 Operation example when SPCR.TXMD[1:0] = 00b

The operation of the flags at timings (1) and (2) in Figure 43.30 is as follows:

1. When serial transfer ends while the number of SPDR receive buffer store matches the number of frames set in SPDCR2.RTRG, the SPI generates a receive buffer full interrupt request (SPIi_SPRI), the SPI sets the SPSR.SPRF flag to 1, and copies the received data in the shift register to the receive buffer.
2. When serial transfer ends with data for the number of FIFO stages stored in the SPDR receive buffer, the SPI sets the SPSR.OVRF flag to 1, and discards the received data in the shift register. For details about the operation of the SPSR.OVRF flag, see [section 43.3.10.1. Overrun Errors](#).

In Transmit-Receive serial communication (TXMD[1:0] = 00b), transmit data is transmitted and receive data is received. Therefore, the SPRF flag and the OVRF flag are set to 1 at timings (1) and (2) respectively.

43.3.6.2 Transmit-Only Serial Communications (TXMD[1:0] = 01b)

Figure 43.31 shows an example of operation when the communication mode select bit (TXMD[1:0]) in the SPI control register (SPCR) is set to 01b. In this example, the SPI performs an 8-bit serial transfer when the SPDCR2.TTRG is 0, the SPDCR2.RTRG is 0, the SPCMDm.CPHA bit is 1, and the SPCMDm.CPOL bit is 0. The numbers given for RSPCKn in the waveform represent the number of RSPCK cycles, such as the number of transferred bits.

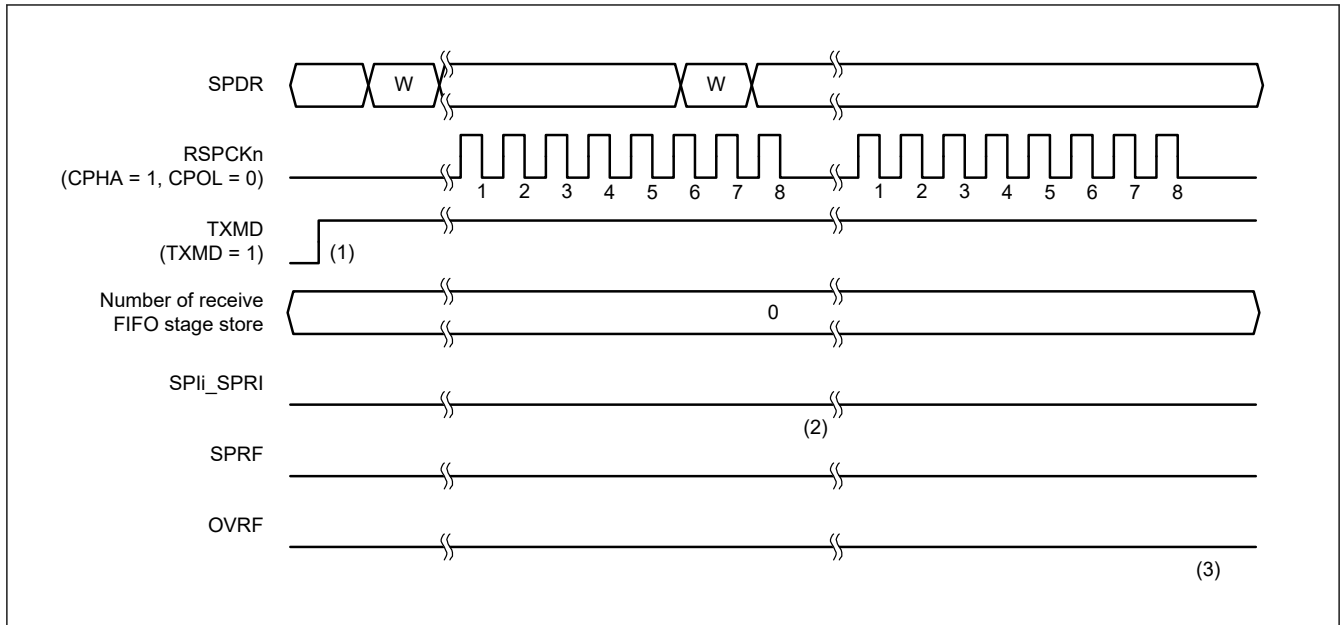


Figure 43.31 Operation example when SPCR.TXMD[1:0] = 01b

The operation of the flags at timings (1) to (3) in Figure 43.31 is as follows:

1. Make sure there is no data left in the receive buffer (the SPSR.SPRF flag is 0) and the SPSR.OVRF flag is 0 before entering transmit-only mode (SPCR.TXMD[1:0] = 01b).
2. When a serial transfer ends without receiving data in the receiving FIFO of SPDR, if the transmit-only mode is selected (SPCR.TXMD[1:0] = 01b), the SPSR.SPRF flag retains the value of 0, and the SPI does not copy the data in the shift register to the receive buffer.
3. Because the receive buffer of SPDR does not hold data that was received in the previous serial transfer, even when a serial transfer ends, the SPSR.OVRF flag retains the value of 0, and the data in the shift register is not copied to the receive buffer.

In transmit-only mode (SPCR.TXMD[1:0] = 01b), the SPI transmits data but does not receive data. Therefore, the SPSR.SPRF and SPSR.OVRF flags remain 0 at timings (1) to (3).

43.3.6.3 Receive-Only Serial Communication (TXMD[1:0] = 10b)

Figure 43.32 shows an example of operation when the communication mode select bit (TXMD[1]) in the SPI control register (SPCR) is set to 1. In this example, the SPI performs an 8-bit serial transfer when the SPDCR2.TTRG is FIFO stage - 1, the SPDCR2.RTRG is 0, the SPCMDm.CPHA bit is 1, and the SPCMDm.CPOL bit is 0. The numbers given for RSPCKn in the waveform represent the number of RSPCK cycles, such as the number of transferred bits.

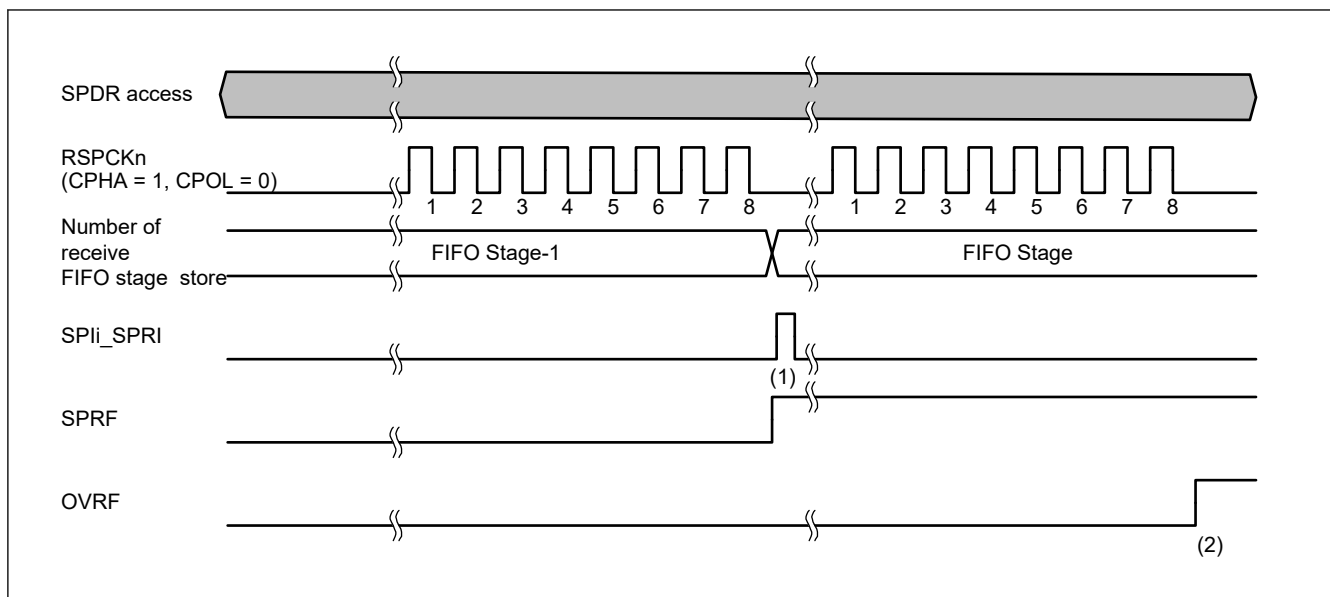


Figure 43.32 Example of Operation when SPCR.TXMD[1:0] = 10b

The following describes operation of flags at timings (1) and (2) in the figure above.

1. When serial transfer ends while the number of SPDR receive buffer store matches the number of frames set in SPDCR2.RTRG, the SPI generates a receive buffer full interrupt request (SPIi_SPRI), the SPI sets the SPSR.SPRF flag to 1, and copies the received data in the shift register to the receive buffer
2. When serial transfer ends with data for the number of FIFO stages stored in the SPDR receive buffer, the SPI sets the SPSR.OVRF flag to 1 and discards the received data in the shift register.

43.3.7 Transmit Buffer Empty and Receive Buffer Full Interrupts

Figure 43.33 show examples of operation of the transmit buffer empty interrupt (SPIi_SPTI ($i = 0, 1$)) and the receive buffer full interrupt (SPIi_SPRI). The SPDR register accesses shown in these figures indicate the conditions of access to the register, where W denotes a write cycle and R a read cycle. In Figure 43.33, the SPI performs an 8-bit serial transfer when SPCR.TXMD[1:0] bits are 00b, the SPDCR2.TTRG bit is 0, the SPDCR2.RTRG bit is 0, the SPCMDm.CPHA bit is 0, and the SPCMDm.CPOL bit is 0. The numbers given for RSPCKn in the waveform represent the number of RSPCK cycles, such as the number of transferred bits.

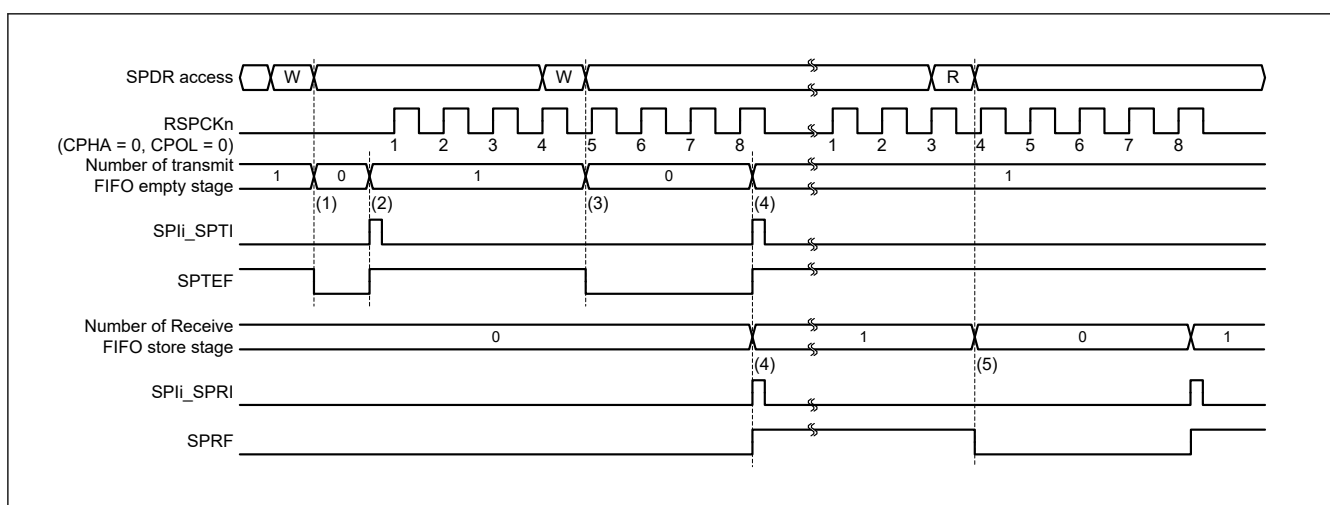


Figure 43.33 Operation example of the SPIi_SPTI and SPIi_SPRI interrupts when CPHA = 0 and CPOL = 0 in master mode

The operation of the SPI at timings (1) to (5) in Figure 43.33 is as follows:

1. When transmit data is written to SPDR with the transmit buffer of SPDR is before the next transfer data is set, the SPI writes data to the transmit buffer. When transmit data is written to SPDR in one processing routine using DTC / DMAC, the SPSR.SPTEF flag is cleared to 0 at the last access.
2. If the shift register is empty, the SPI copies the data in the transmit buffer to the shift register. At this time, if transmit FIFO empty stage number > TTRG value, then the SPI generates a transmit buffer empty interrupt request (SPli_SPTI), and sets the SPSR.SPTEF flag to 1. How a serial transfer is started depends on the SPI mode. For details, see [section 43.3. Operation](#), and [section 43.3.13. Clock Synchronous Operation](#).
3. When transmit data is written to SPDR either by the transmit buffer empty interrupt routine, or by the processing of the transmit buffer empty using the SPTEF flag, the SPI writes data to the transmit buffer. When the transmit data is written to SPDR in one processing routine using DTC / DMAC, the SPTEF flag is cleared to 0 at the last access. Because the data being transferred serially is stored in the shift register, the SPI does not copy the data in the transmit buffer to the shift register.
4. When the serial transfer ends with the receive buffer of SPDR > FIFO stage number, the SPI copies the receive data in the shift register to the receive buffer, generates a receive buffer full interrupt request (SPli_SPRI), and sets the SPRF flag to 1. Because the shift register becomes empty on completion of the serial transfer, if the next transfer data is set in the transmit FIFO before the serial transfer ended, the SPI sets the SPTEF flag to 1 and copies data in the transmit buffer to the shift register. Even when received data is not copied from the shift register to the receive buffer in an overrun error status, on completion of the serial transfer, the SPI determines that the shift register is empty, so data transfer from the transmit buffer to the shift register is enabled.
5. When SPDR is read either by the receive buffer full interrupt routine or processing of the receive buffer full interrupt using the SPRF flag, the receive data can be read. If the received data is read from SPDR in one processing routine using DTC / DMAC, the SPRF flag is cleared to 0 at the last access.

When transmit data is written to the SPDR register while no empty stages in the transmit FIFO, the SPI does not update data in the transmit buffer. When writing to SPDR, always use either a transmit buffer empty interrupt request or check the empty or processing of the transmit buffer empty interrupt using the SPTEF flag. To use a transmit buffer empty interrupt, set the SPTIE bit in SPCR to 1. If the SPI function is disabled (the SPCR.SPE bit is 0), set the SPTIE bit to 0.

When serial transfer ends while data is stored in the receive FIFO for the number of FIFO stages, the SPI does not copy data from the shift register to the receive buffer, and it detects an overrun error (see [section 43.3.10. Error Detection](#)). To prevent a receive data overrun error, read the received data using a receive buffer full interrupt request before the next serial transfer ends. To use an SPI receive buffer full interrupt, set the SPCR.SPRIE bit to 1.

Transmission and reception interrupts or the associated IELSRn.IR flags (where n is the interrupt vector number) in the ICU can be used to confirm the states of the transmit and receive buffers.

Similarly, the SPTEF and SPRF flags can be used to confirm the states of the transmit and receive buffers. See [section 14, Interrupt Controller Unit \(ICU\)](#) for the interrupt vector numbers.

43.3.8 Idle Interrupt

When the SPCP[2:0] of the SPI status register (SPSR) becomes 000b (start of sequence control), the IDLNF flag in the SPI status register (SPSR) is set to 1 and an idle interrupt request is made during master mode operation. An interrupt request is also made by clearing the SPCR.SPE bit to 0.

[In the Motorola-SPI case]

[Figure 43.34](#) shows an example of idle interrupt operation during normal operation.

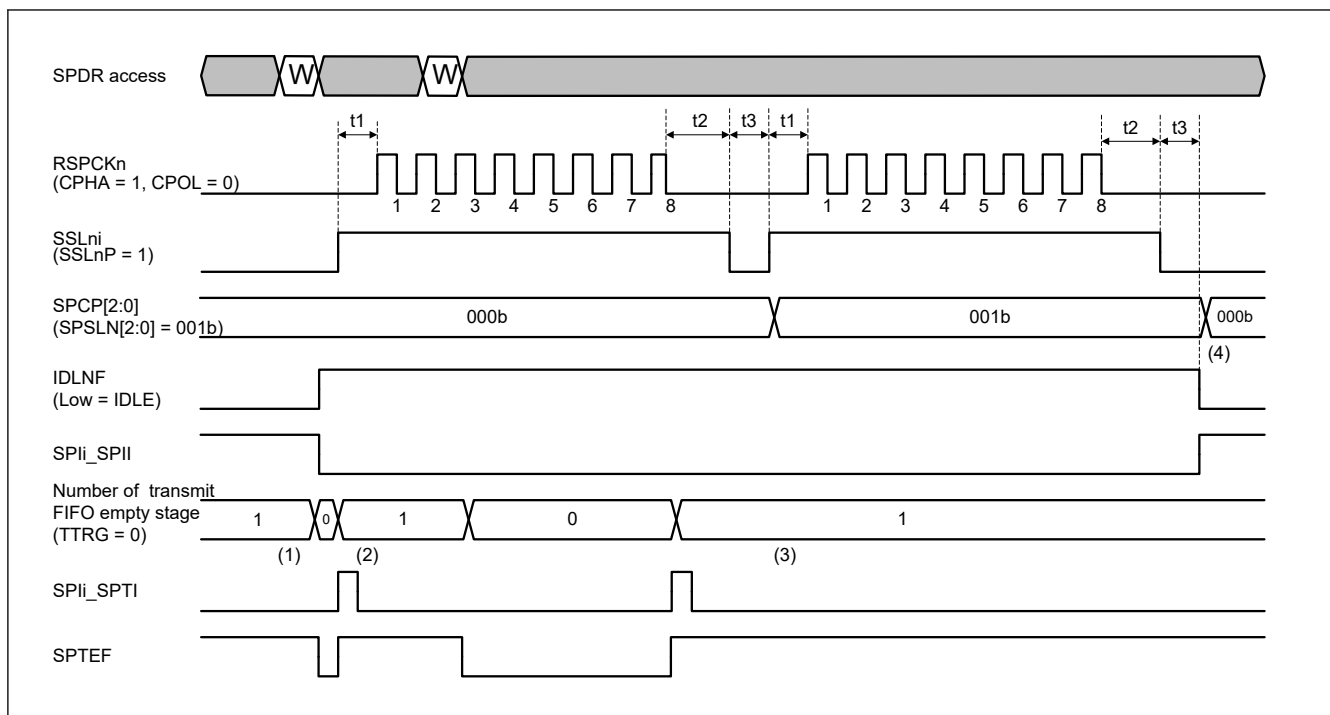


Figure 43.34 Example of Idle Interrupt Operation (Master mode / Motorola-SPI)

1. At the start of transmission, if the next transfer data is not set in the transmission buffer, the IDLNF flag is 0 (IDLE). Writing transmit data sets the IDLNF flag to 1 (BUSY). When the SPIIE bit in the SPI control register (SPCR) is set to 1 before transmit data is written, interrupt processing is required before transmission start. For this reason, set the SPIIE bit to 0 before starting transmission.
2. After transmission has started, the IDLNF flag remains 1 (BUSY) regardless of the transmit buffer state.
3. The SPCP[2:0] bits change the command to the next command at the end of t3 cycle. When the next command is not 000b, the IDLNF flag remains unchanged even when the next transmit data has not been written.
4. The IDLNF flag is cleared to 0 (IDLE) at the end of t3 cycle because the next command is 000b and the next transmit data is not present. When the SPIIE bit is 1 currently, an SPli_SPII (i = 0, 1) interrupt is output.

[In the TI-SSP case]

Figure 43.35 shows an example of idle interrupt operation during normal operation.

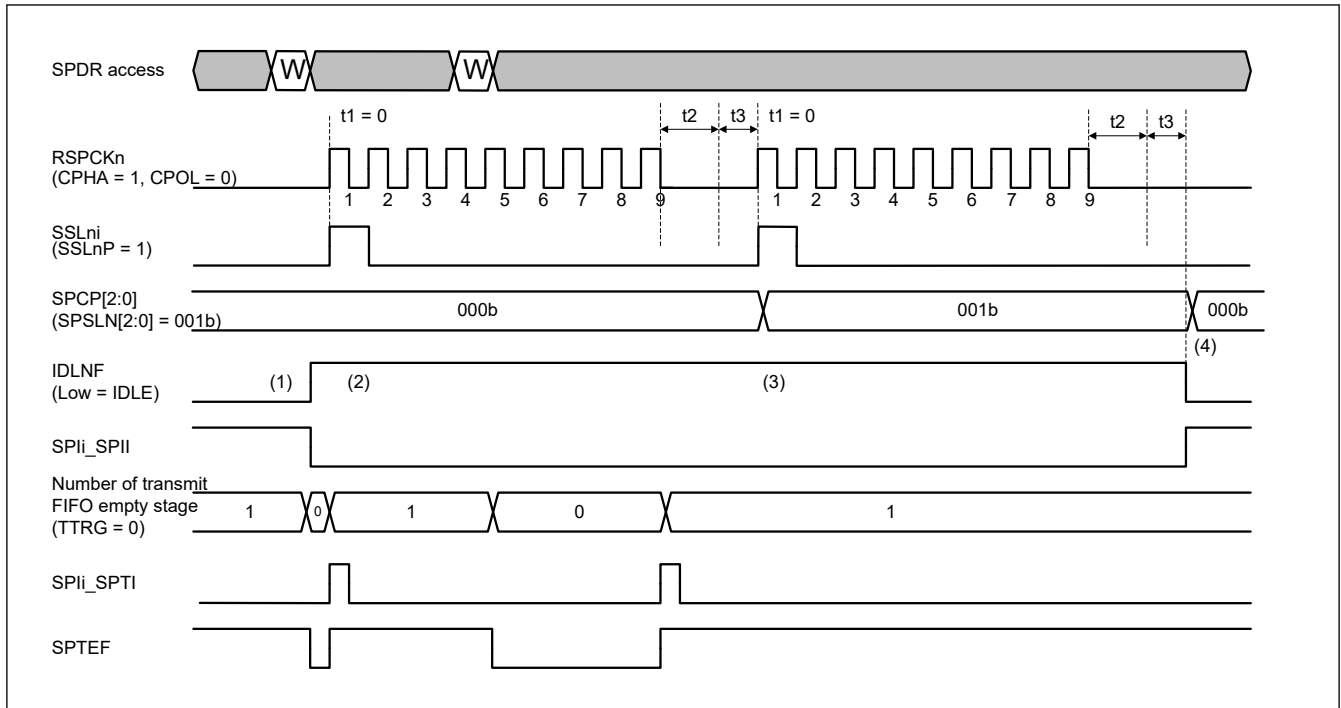


Figure 43.35 Example of Idle Interrupt Operation (Master mode / TI-SSP)

1. At the start of transmission, if the next transfer data is not set in the transmission buffer, the IDLNF flag is 0 (IDLE). Writing transmit data makes sets the IDLNF flag to 1 (BUSY). When the SPIIE bit in the SPI control register (SPCR) is set to 1 before transmit data is written, interrupt processing is required before transmission start. For this reason, set the SPIIE bit to 0 before starting transmission.
2. After transmission has started, the IDLNF flag remains 1 (BUSY) regardless of the transmit buffer state.
3. The SPCP[2:0] bits change the command to the next command at the end of t3 cycle. When the next command is not 000b, the IDLNF flag remains unchanged even when the next transmit data has not been written.
4. The IDLNF flag is cleared to 0 (IDLE) at the end of t3 cycle because the next command is 000b and the next transmit data is not present. When the SPIIE bit is 1 currently, an SPli_SPII interrupt is output.

43.3.9 Communication End Interrupt

43.3.9.1 Transmit-Receive/Transmit in Master Mode

See the description of the CENDF bit in [section 43.2.9. SPSR : SPI Status Register](#) for the setting / clearing conditions of the communication completion flag during Transmit-Receive/Transmit-only in Master Mode.

[In the Motorola-SPI case]

[Figure 43.36](#) shows an example of communication end interrupt operation during transmit-receive/transmit master mode.

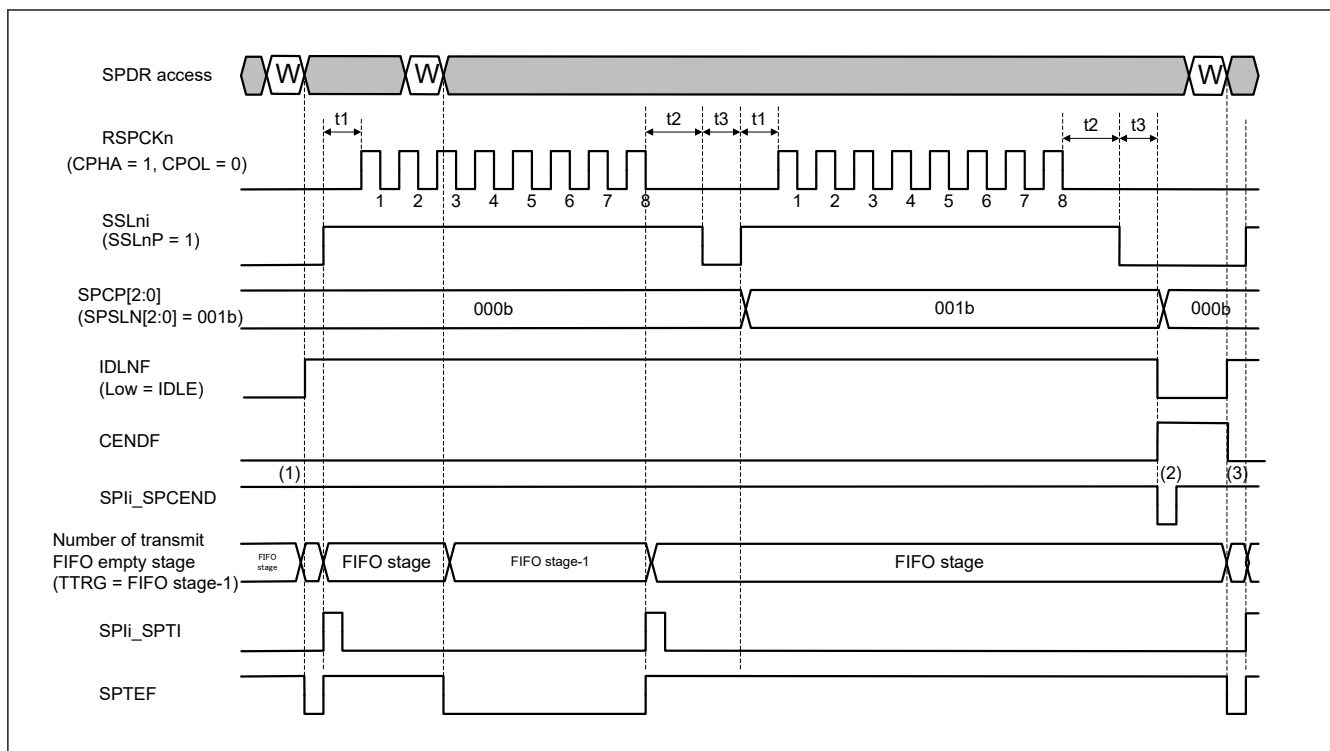


Figure 43.36 Example of Communication End Interrupt Operation (Transmit-Receive/Transmit Master mode/Motorola-SPI)

1. The CENDF flag is 0 and the level of SPIi_SPCEND (i = 0, 1) is 1 before communication start, and these have kept during communication.
2. The CENDF flag will be 1 (Communication End) at the end of t_3 cycle because the next command is 000b and there is no next transmit data, and then the SPIi_SPCEND interrupt output when the CENDIE bit is 1.
3. The CENDF flag is cleared when the next transmission data is written to the transmit buffer (SPTX). Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

[In the TI-SSP case]

Figure 43.37 shows an example of communication end interrupt operation during transmit-receive/transmit-only master mode.

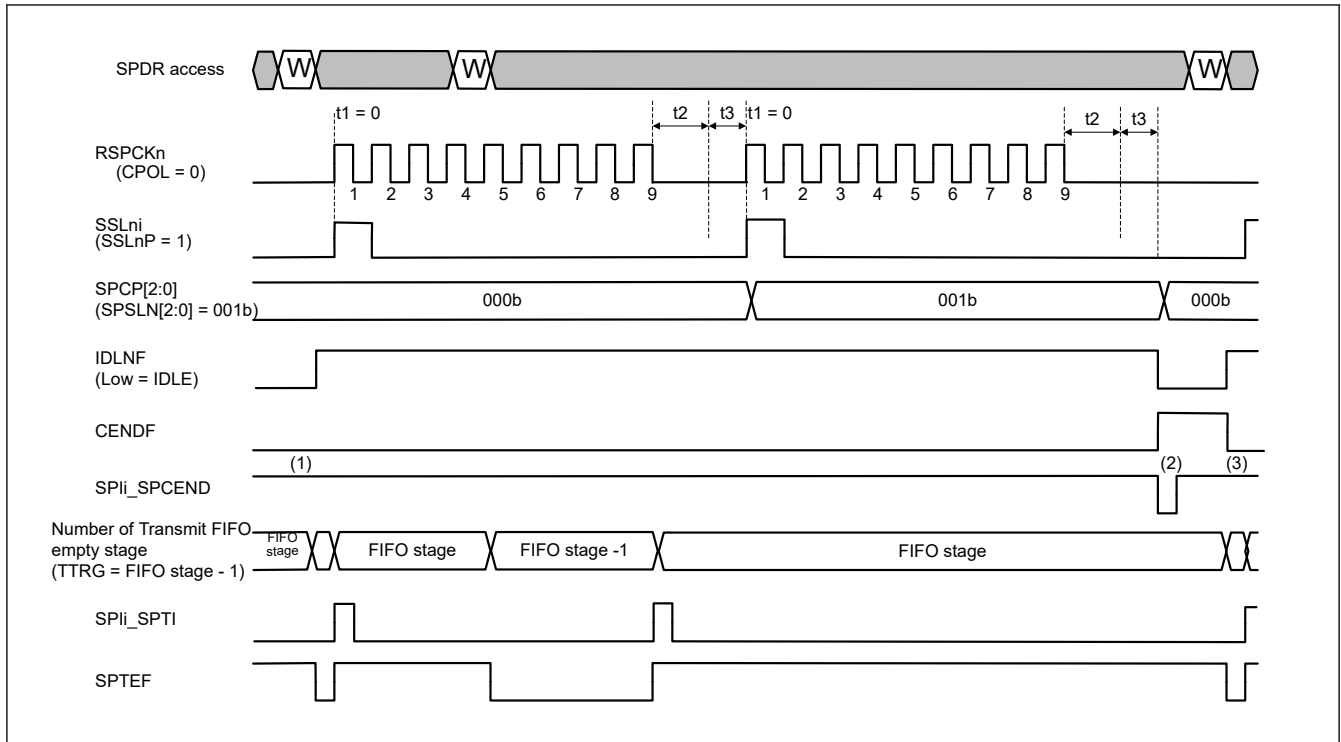


Figure 43.37 Example of Communication End Interrupt Operation (Transmit-Receive/Transmit-only Master mode/TI-SSP)

1. The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start, and these have kept during communication.
2. The CENDF flag will be 1 (Communication End) at the end of t3 cycle because the next command is 000b and there is no next transmit data, and then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
3. The CENDF flag is cleared when the next transmission data is written to the transmit buffer (SPTX). Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

In slave mode operation, the output timing of the communication end interrupt is different due to the value of the SPCR.SPMS bit (SPI mode select bit), and the clear timing of the communication end interrupt is different due to the communication mode (transmit-receive or transmit-only or receive-only).

43.3.9.2 Receive-only in Master Mode

See the description of the CENDF bit in [section 43.2.9. SPSR : SPI Status Register](#) for the setting / clearing conditions of the communication completion flag during Receive-only in Master Mode.

[Figure 43.38](#) shows an example of communication end interrupt operation during receive-only master mode at RMFM[4:0] = 0.

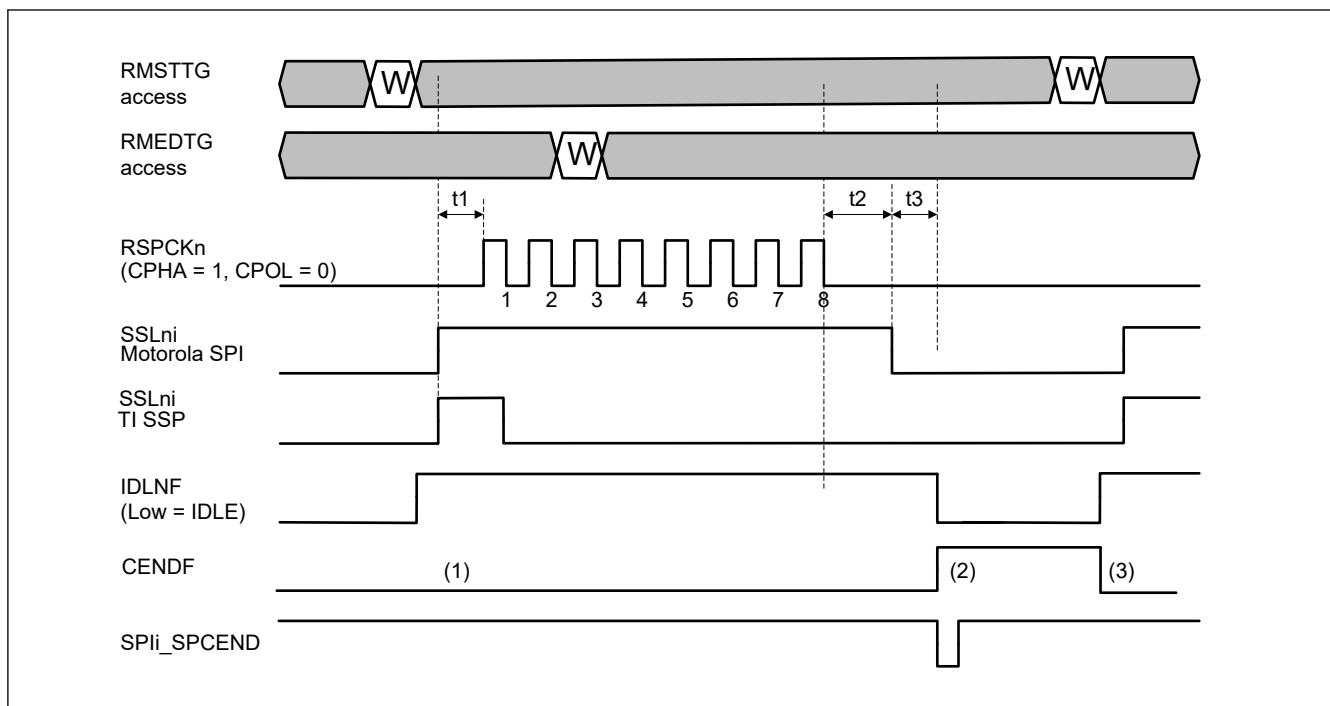


Figure 43.38 Example of Communication End Interrupt Operation (Receive-only Master mode / Motorola-SPI) at RMFM [4:0] = 0

1. The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start. These have kept during communication.
2. The CENDF flag will be 1 (Communication End) at the end of t3 cycle by writing 1 to RMEDTG during the communication frame. Then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
3. The CENDF flag is cleared when writing 1 to RMSTTG. Also when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

Figure 43.39 shows an example of communication end interrupt operation during receive-only master mode at RMFM[4:0] $\neq$ 0.

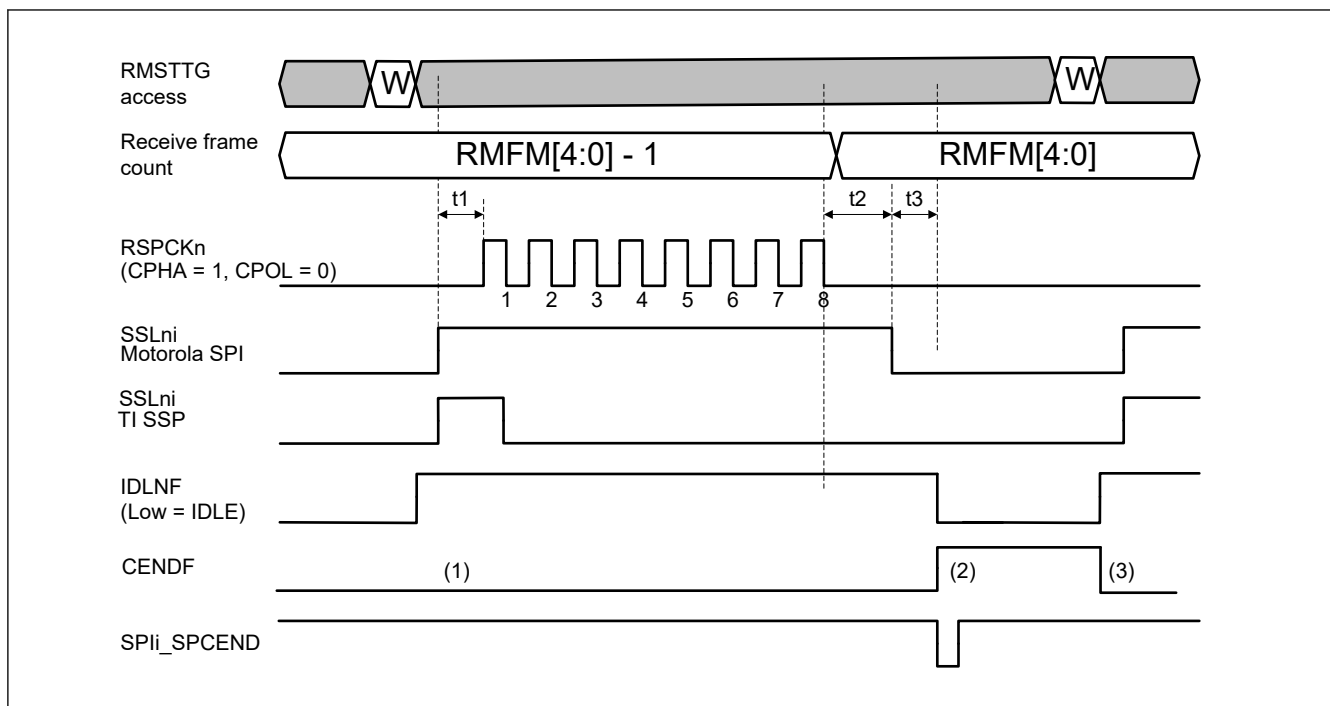


Figure 43.39 Example of Communication End Interrupt Operation (Receive-only Master mode / Motorola-SPI) at RMFM [4:0] ≠ 0

1. The CENDF flag is 0 and the level of SPli_SPCEND is 1 before communication start. These have kept during communication.
2. The CENDF flag will be 1 (Communication End) at the end of t3 cycle after receiving the number of frames set by RMFM[4:0]. Then the SPli_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
3. The CENDF flag is cleared when writing 1 to RMSTTG. Also when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

43.3.9.3 Transmit-Receive/Transmit in Slave Mode on SPI Operation (4-wire)

See the description of the CENDF bit in [section 43.2.9. SPSR : SPI Status Register](#) for the setting / clearing conditions of the communication completion flag during Transmit-Receive/Transmit-only in Slave Mode (4-wire).

[In the Motorola-SPI case]

[Figure 43.40](#) shows an example of communication end interrupt operation during transmit-receive/transmit slave mode on SPI operation.

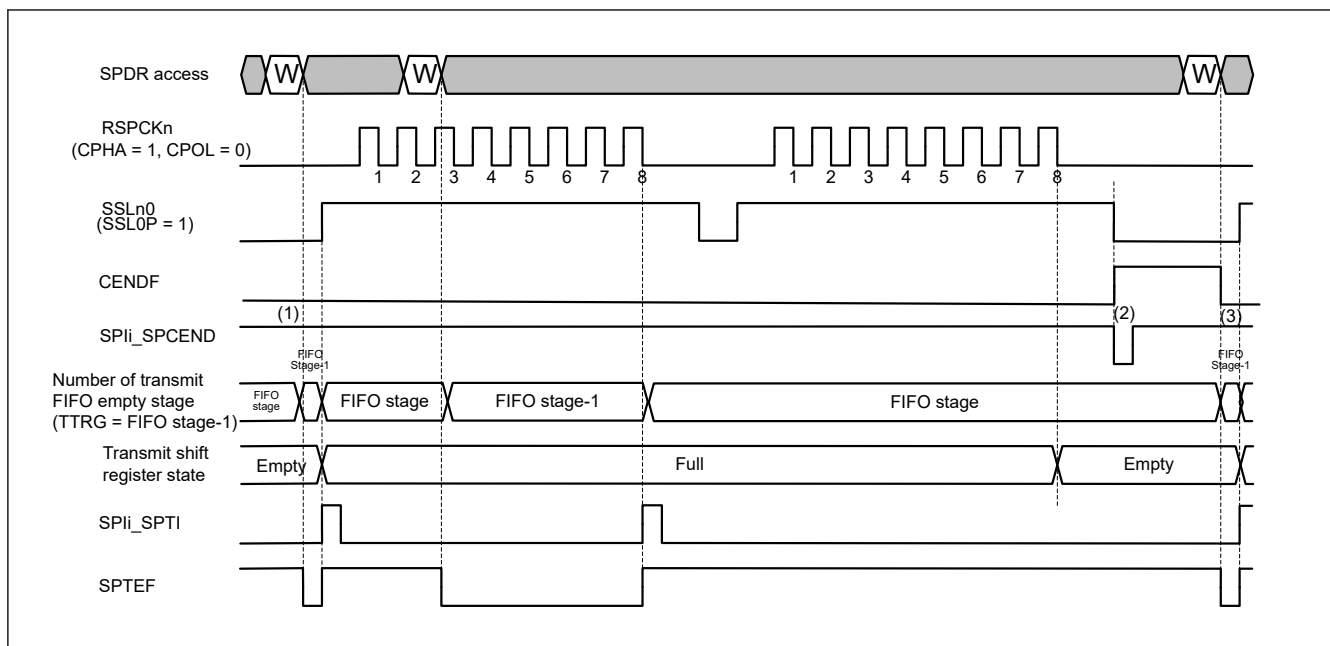


Figure 43.40 Example of Communication End Interrupt Operation (Transmit-Receive/Transmit Slave mode on SPI Operation/Motorola-SPI)

1. The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start, and these have kept during communication.
2. The CENDF flag will be 1 (Communication End) at the timing of SSLn0 negate when the next transfer data is not set in the transmit FIFO and the transmit shift register is empty. Then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
3. The CENDF flag is cleared when the next transmission data is written to the transmit buffer (SPTX). Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

[In the TI-SSP case]

Figure 43.41 shows an example of communication end interrupt operation during transmit-receive/transmit-only slave mode on SPI operation.

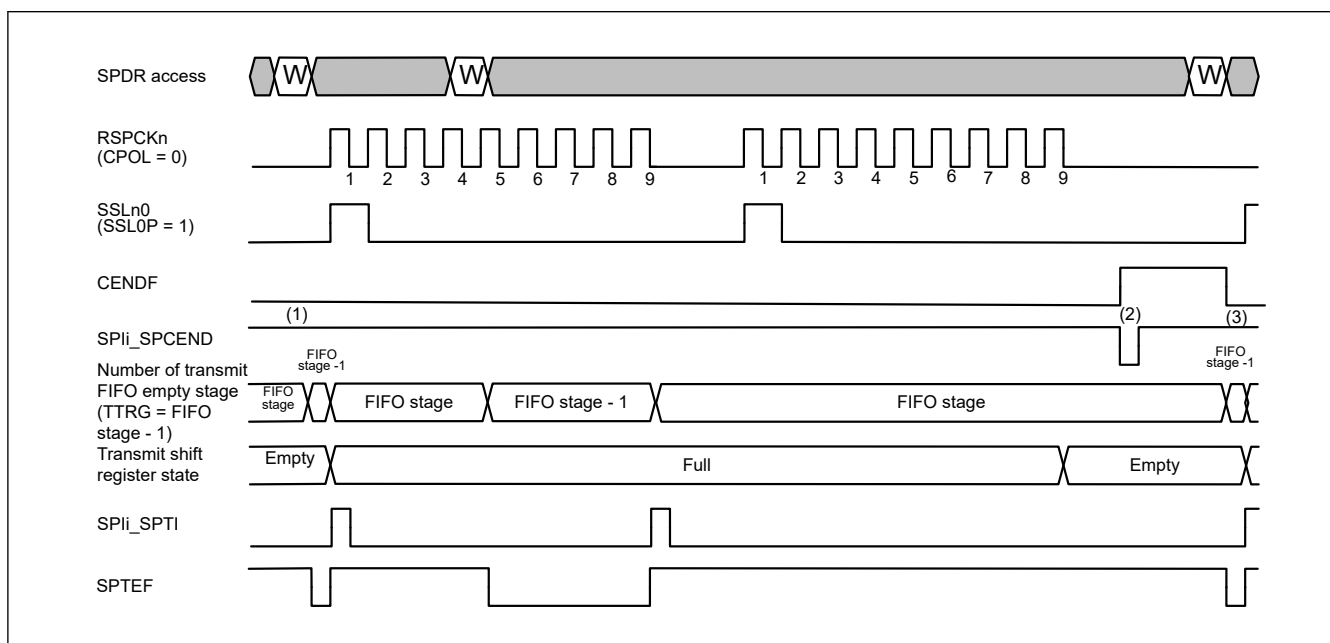


Figure 43.41 Example of Communication End Interrupt Operation (Transmit-Receive/Transmit-only Slave mode on SPI Operation / TI-SSP)

1. The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start, and these have kept during communication.
2. The CENDF flag will be 1 (Communication End) at the RSPCKn last data bit sampling when the next transfer data is not set in the transmit FIFO and the transmit shift register is empty. Then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
3. The CENDF flag is cleared when the next transmission data is written to the transmit buffer (SPTX). Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

43.3.9.4 Receive Only in Slave Mode on SPI Operation (4-wire)

See the description of the CENDF bit in [section 43.2.9. SPSR : SPI Status Register](#) for the setting / clearing conditions of the communication completion flag during Receive-only in Slave Mode (4-wire).

[In the Motorola-SPI case]

[Figure 43.42](#) shows an example of communication end interrupt operation during receive only slave mode on SPI operation (4-wire).

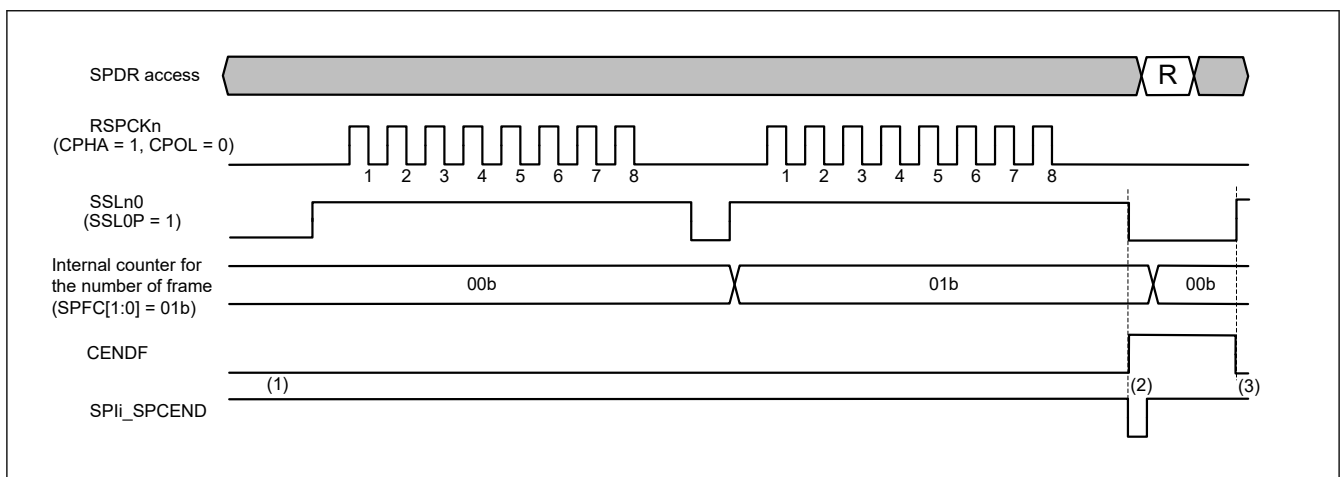


Figure 43.42 Example of Communication End Interrupt Operation (Receive only Slave mode on SPI Operation / Motorola-SPI)

1. The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start, and these have kept during communication.
2. After the frames for SPFC set value in the SPI data control register (SPDCR) are stored in the receive buffer, the CENDF flag becomes 1 (communication completed) at the timing of SSLn0 negation. Then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
3. The CENDF flag is cleared at the SSLn0 assert when the next transmission start. Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

[In the TI-SSP case]

[Figure 43.43](#) shows an example of communication end interrupt operation during receive only slave mode on SPI operation (4-wire).

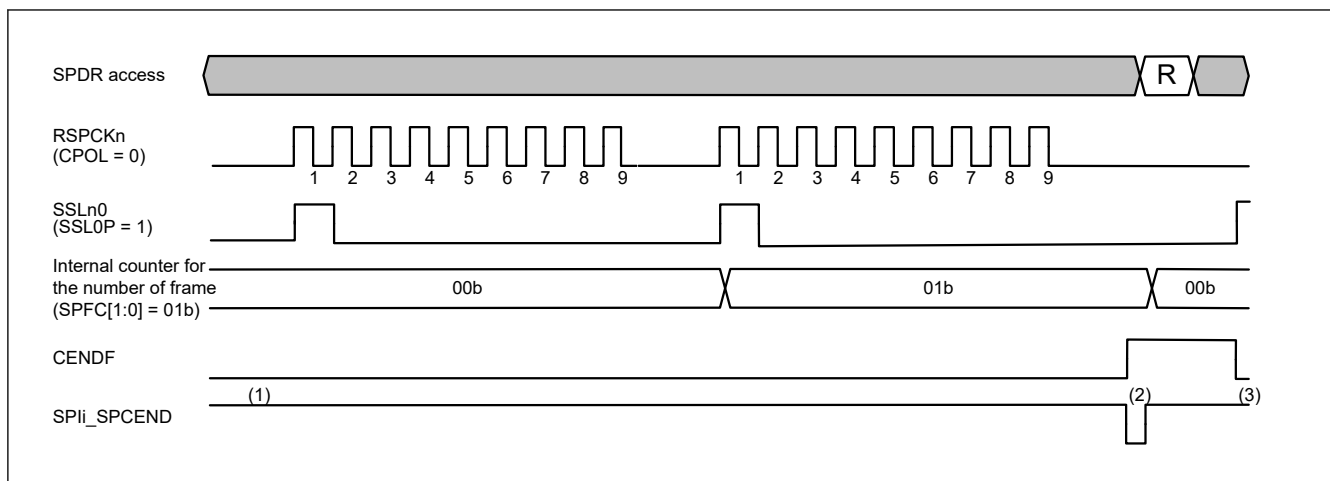


Figure 43.43 Example of Communication End Interrupt Operation (Receive-only Slave mode on SPI Operation / TI-SSP)

1. The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start, and these have kept during communication.
2. The CENDF flag will be 1 (Communication End) at the RSPCK last data bit sampling when the last frame transmission ends. Then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
3. The CENDF flag is cleared at the SSLn0 assert when the next transmission start. Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

43.3.9.5 Transmit-Receive/Transmit in Slave Mode on Clock Synchronous Operation (3-wire)

See the description of the CENDF bit in [section 43.2.9. SPSR : SPI Status Register](#) for the setting / clearing conditions of the communication completion flag during Transmit-Receive/Transmit-only in Slave Mode on Clock Synchronous (3-wire).

[Figure 43.44](#) shows an example of communication end interrupt operation during transmit-receive/transmit slave mode on clock synchronous operation (3-wire).

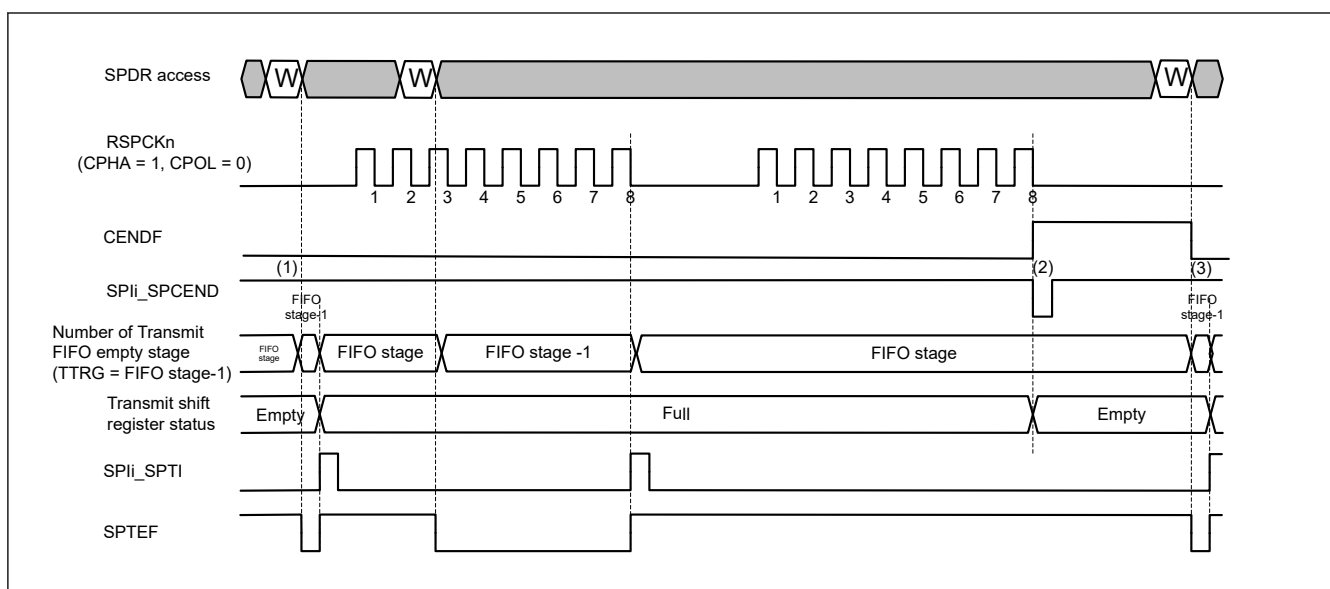


Figure 43.44 Example of Communication End Interrupt Operation (Transmit-Receive/Transmit Slave mode on Clock Synchronous Operation)

1. The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start, and these have kept during communication.

- When the next transfer data is not set in the transmit FIFO and the transmit shift register is empty, then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
- The CENDF flag is cleared when the next transmission data is written to the transmit buffer (SPTX). Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

43.3.9.6 Receive Only in Slave Mode on Clock Synchronous Operation (3-wire)

See the description of the CENDF bit in [section 43.2.9. SPSR : SPI Status Register](#) for the setting / clearing conditions of the communication completion flag during Receive -only in Slave Mode on Clock Synchronous (3-wire).

[Figure 43.45](#) shows an example of communication end interrupt operation during receive only slave mode on clock synchronous operation.

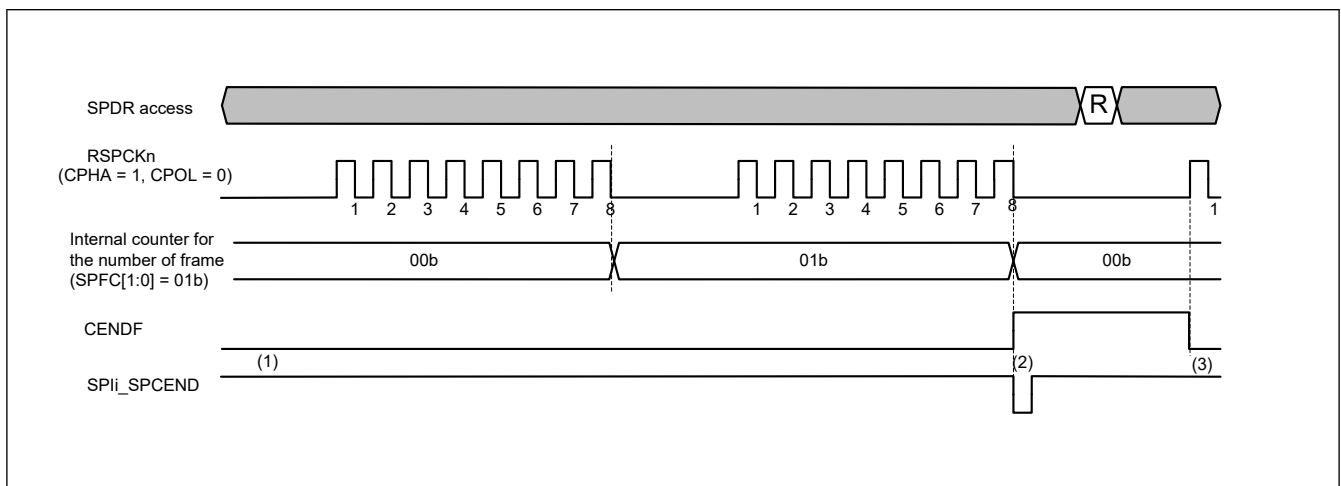


Figure 43.45 Example of Communication End Interrupt Operation (Receive-only Slave mode on Clock Synchronous Operation)

- The CENDF flag is 0 and the level of SPIi_SPCEND is 1 before communication start, and these have kept during communication.
- The CENDF flag is set to 1 (communication completed) at the timing of the last data bit sampling of RSPCKn in the last frame communication when the last frame of the SPI data control register (SPDCR) SPFC set value is received. Then the SPIi_SPCEND interrupt outputs with PCLK 1 cycle width if the CENDIE bit is 1.
- The CENDF flag is cleared at the first edge of RSPCKn for the next transmission. Or when 1 is written to the SPSRC.CENDFC bit, then the CENDF flag is 0.

43.3.9.7 Common Operation

In this chapter, the operation common to each mode / area option communication in [section 43.3.9.1. Transmit-Receive/ Transmit in Master Mode](#) to [section 43.3.9.6. Receive Only in Slave Mode on Clock Synchronous Operation \(3-wire\)](#) is explained. When the enable of SPI communication end interrupt (CENDIE) is 0, at the time of communication completion, a flag of communication end (CENDF) is set and an event of communication end is output, but no interrupt is output. However, if the enable of communication end interrupt (CENDIE) is set to 1 before clearing the flag of communication end (CENDF) while the enable of SPI function (SPE) is 1, the communication end interrupt (SPIi_SPCEND) is output.

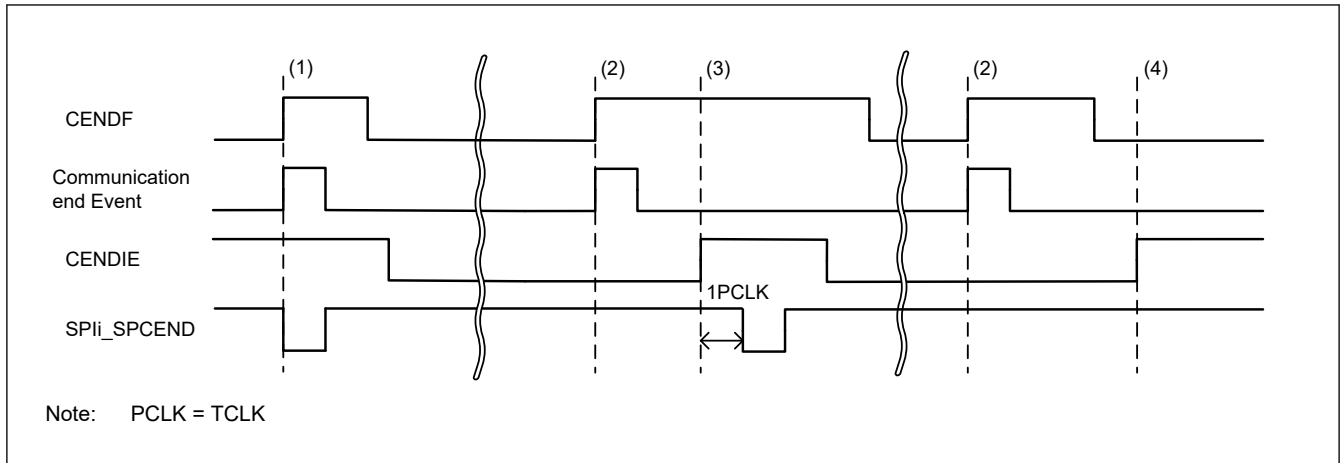


Figure 43.46 Example of Communication End Interrupt Operation (Enable control)

- When the enable of SPI communication end interrupt (CENDIE) is 1, at the time of communication completion, the following three are the same timing.
 - A flag of communication end (CENDF)
 - An event of communication end
 - The communication end interrupt (SPIi_SPCEND)
- When the enable of SPI communication end interrupt (CENDIE) is 0, at the time of communication completion, the following two are the same timing, but no interrupt.
 - A flag of communication end (CENDF)
 - An event of communication end
- After (2), if the enable of communication end interrupt (CENDIE) is set when the enable of SPI function (SPE) and the flag of communication end (CENDF) are 1, the communication end interrupt (SPIi_SPCEND) is output after 1 TCLK.
- After (2), even if the enable of communication end interrupt (CENDIE) is set when the enable of SPI function (SPE) or the flag of communication end (CENDF) is 0, the communication end interrupt (SPIi_SPCEND) is not output.

43.3.10 Error Detection

In normal SPI serial transfers, data written to the transmit buffer of SPDR is transmitted, and received data can be read from the receive buffer of SPDR. If access is made to SPDR, an abnormal transfer might occur, depending on the status of the transmit or receive buffer or the status of the SPI at the beginning or end of serial transfer.

If an abnormal transfer occurs, the SPI detects the event as an underrun error, overrun error, parity error, or mode fault error. [Table 43.9](#) lists the relationship between non-normal transfer operations and the SPI error detection function.

Table 43.9 Relationship between non-normal transfer operations and SPI error detection (1 of 2)

Operation	Occurrence condition	SPI operation	Error detection
1	SPDR is written while no empty stages in the transmit FIFO.	• The contents of the transmit buffer are kept • Write data is missing	None
2	SPDR is read while no data stored in receive FIFO.	The contents of the receive buffer and previously received data are output.	None
3	Serial transfer is started in slave mode when the SPI is not able to transmit data.	• Serial transfer is suspended • Transmit or receive data is missing • Driving of the MISO output signal is stopped • SPI function is disabled	Underrun error
4	Serial transfer ends when data is stored in the receive FIFO for the number of FIFO stages.	• Keeps the contents of the receive FIFO • Missing receive data	Overrun error

Table 43.9 Relationship between non-normal transfer operations and SPI error detection (2 of 2)

Operation	Occurrence condition	SPI operation	Error detection
5	An incorrect parity bit is received during full-duplex synchronous serial communication with the parity function enabled in following mode: • Transmit-receive master mode • Receive-only master mode • Transmit-receive slave mode • Receive-only slave mode	The parity error flag is asserted	Parity error
6	The SSLn0 input signal is asserted when the serial transfer is idle in multi-master mode.	• Driving of the RSPCKn, MOSIn, SSLn1 to SSLn3 output signals is stopped • SPI function is disabled	Mode fault error
7	The SSLn0 input signal is asserted during serial transfer in multi-master mode.	• Serial transfer is suspended • Transmit or receive data is missing • Driving of the RSPCKn, MOSIn, SSLn1 to SSLn3 output signals is stopped • SPI function is disabled	Mode fault error
8	[In the Motorola-SPI case] The SSLn0 input signal is negated during serial transfer in slave mode.	• Serial transfer is suspended • Transmit or receive data is missing • Driving of the MISOn output signal is stopped • SPI function is disabled	Mode fault error
9	[In the TI-SSP case] The SSLn0 input signal is asserted during serial transfer in slave mode.	• Serial transfer is suspended • Transmit or receive data is missing • Driving of the MISOn output signal is stopped • SPI function is disabled	Mode fault error
10	After data is stored in the receive FIFO with SPDRES = 1, the number of stored data is less than the threshold value and no receive data is written for the set value of SPDRC [7:0]	Assert the receive data ready flag	Receive data ready

In operation 1 described in [Table 43.9](#), the SPI does not detect an error. To prevent data omission during writes to SPDR, the writes to SPDR must be executed using a transmit buffer empty interrupt request (when the SPSR.SPTEF flag is 1).

Similarly, the SPI does not detect an error in operation 2. To prevent extraneous data from being read, SPDR read must be executed with an SPI receive buffer full interrupt request (when the SPSR.SPRF flag is 1).

For information on the other errors, see the following sections:

- Underrun error, indicated in operation 3, see [section 43.3.10.4. Underrun Errors](#)
- Overrun error, indicated in operation 4, see [section 43.3.10.1. Overrun Errors](#)
- Parity error, indicated in operation 5, see [section 43.3.10.2. Parity Errors](#)
- Mode fault error, indicated in operations 6 to 9, see [section 43.3.10.3. Mode Fault Errors](#)
- For the transmit and receive interrupts, see [section 43.3.7. Transmit Buffer Empty and Receive Buffer Full Interrupts](#).
- For the reception data ready in operations 10, see [section 43.3.10.5. Received Data Ready](#).

43.3.10.1 Overrun Errors

If a serial transfer ends when the receive buffer of SPDR is full, the SPI detects an overrun error and sets the SPSR.OVRF flag to 1. When the OVRF flag is 1, the SPI does not copy data from the shift register to the receive buffer, so the data prior to the error occurrence is retained in the receive buffer. To set the OVRF flag to 0, issue a system reset or 1 is written to the SPSRC.OVRFC bit.

[Figure 43.47](#) shows an example of operation of the OVRF and SPRF flags. The SPSRC and SPDR accesses shown in [Figure 43.47](#) indicate the condition of accesses to the SPSRC and SPDR register, where W denotes a write cycle, and R a read cycle. In this example, the SPI performs an 8-bit serial transfer when SPCMDm.CPHA bit is 1 and the SPCMDm.CPOL bit is 0. The numbers given for RSPCKn in the waveform represent the number of RSPCK cycles, such as the number of transferred bits.

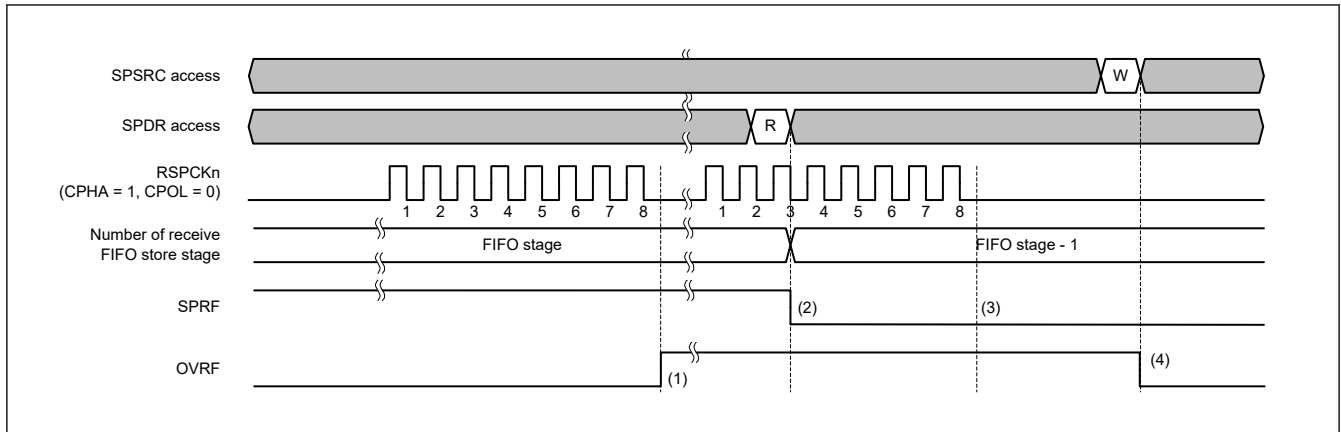


Figure 43.47 Operation example of the OVRF and SPRF flags

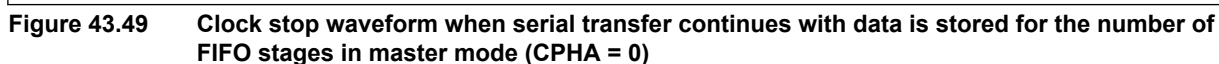
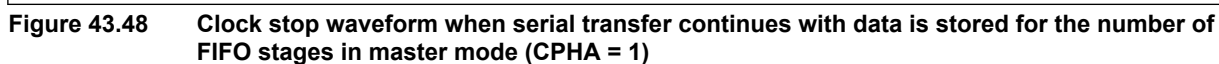
The operation of the flags at timings (1) to (4) in [Figure 43.47](#) is as follows:

1. When serial transfer ends while data is stored for the number of FIFO stages, the SPI detects an overrun error and sets the OVRF flag to 1. The SPI does not copy shift register data to the receive buffer. The SPI does not detect a parity error even when SPPE = 1. In master mode, the SPI copies the value of pointer to the SPI command register (SPCMDm) to the SPECMD[2:0] bits in the SPI status register (SPSR).
2. When SPDR is read, the SPI outputs the data in the receive buffer. At this time, the SPRF flag is cleared to 0 at the last access when the received data is read from SPDR in one processing routine using DTC / DMAC.
3. If the serial transfer ends with the OVRF flag set to 1 (overrun error occurred), the SPI does not copy data in the shift register to the receive buffer (the SPRF flag does not set to 1). A receive buffer full interrupt is not generated. Even when the SPPE bit is 1, parity errors are not detected. In an overrun error state when the SPI does not copy the received data from the shift register to the receive buffer, on termination of the serial transfer, the SPI determines that the shift register is empty. This enables data transfer from the transmit buffer to the shift register.
4. When 1 is written to the SPSRC.OVRFC bit, the SPSR.OVRF flag is cleared.

The occurrence of an overrun can be checked either by reading SPSR or by using an SPI error interrupt and reading SPSR. When executing a serial transfer, you must ensure that overrun errors are detected early, for example by reading SPSR immediately after SPDR is read.

If an overrun error occurs and the OVRF flag sets to 1, normal reception operations cannot be performed until the OVRF flag is set to 0.

When the RSPCK auto-stop function is enabled (SPCR.SCKASE = 1) in master mode, an overrun error does not occur. [Figure 43.48](#) and [Figure 43.49](#) show the clock stop waveform when a serial transfer continues while the receive buffer is full in master mode.



1. While data is stored in the receive FIFO for the number of FIFO stages, the RSPCK clock is deactivated and no overrun error occurs.
2. If SPDR is read while the clock is stopped, data in the receive buffer can be read. The RSPCK clock restarts.

R01UH1067EJ0130 Rev.1.30
Feb 27, 2026

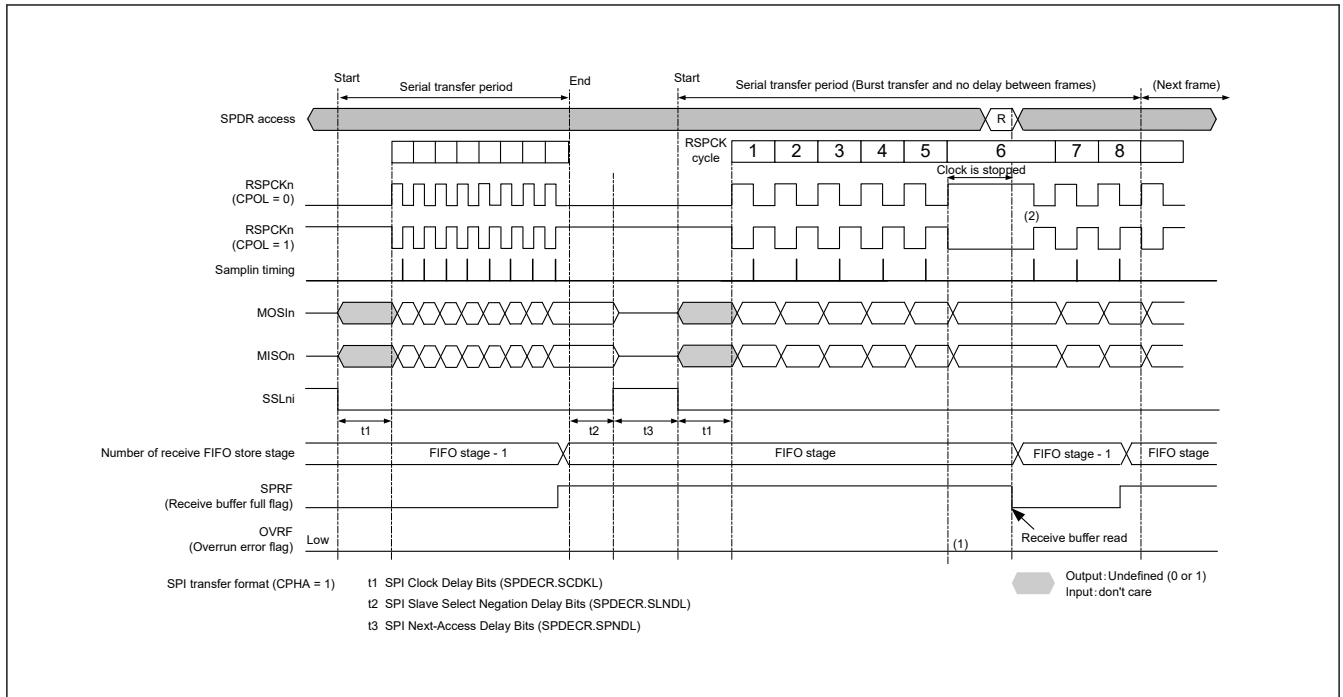


Figure 43.50 Clock Stop Waveform when Serial Transfer Continues in the Receive Buffer Full with data is stored for the number of FIFO stages (at burst transfer and no delay between frames CPHA = 1)

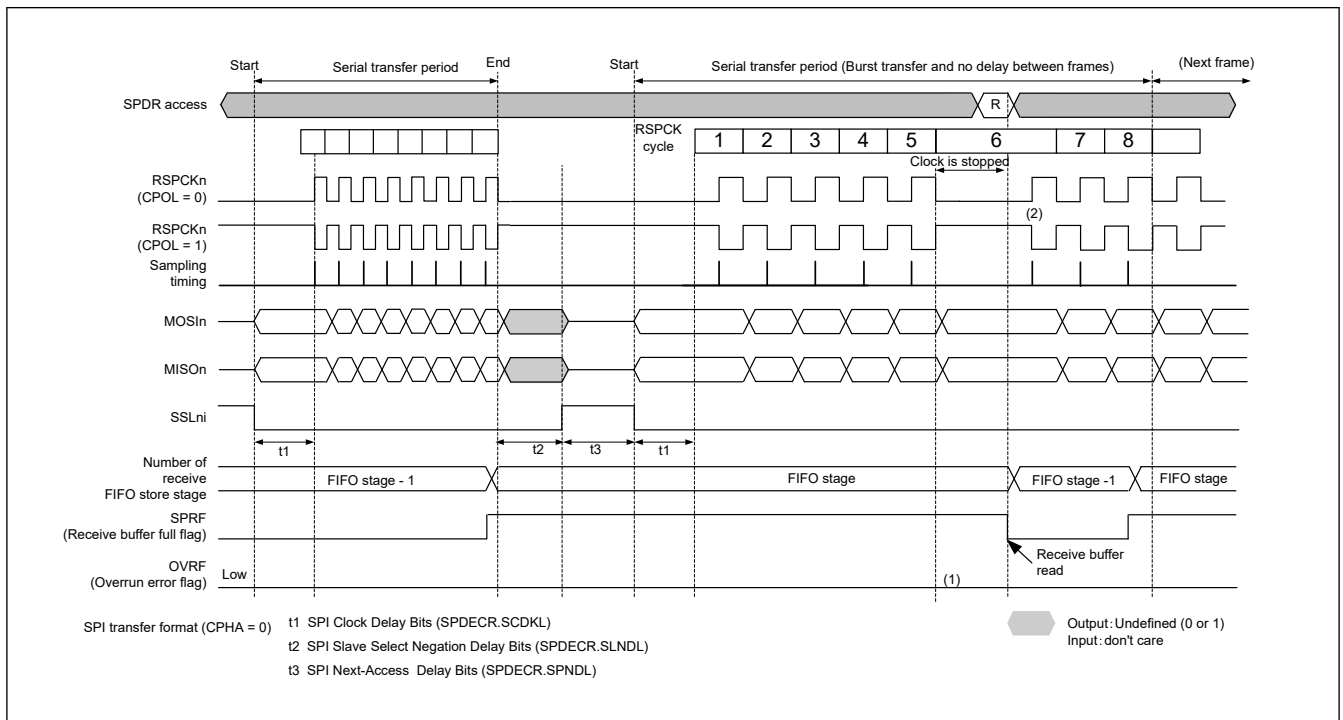


Figure 43.51 Clock Stop Waveform when Serial Transfer Continues with data is stored for the number of FIFO stages in Master Mode (at burst transfer and no delay between frames CPHA = 0)

The following describes operation of flags at timings (1) and (2) in the figure above.

1. While the data is stored for the number of FIFO stages, the RSPCK clock is deactivated and no overrun error occurs.
2. Receive buffer data can be read by reading SPDR during clock stop. After the receive buffer data has been read the RSPCK clock restarts.

43.3.10.2 Parity Errors

After transfer in transmit-receive or receive-only master mode, transmit-receive slave mode or receive only slave mode while the SPPE bit in the SPI control register (SPCR) is 1, the SPI checks occurrence of a parity error. When the SPI detects a parity error in received data, the PERF flag in the SPI status register (SPSR) is set to 1. While the OVRF flag is 1, the SPI does not copy shift register data to the receive buffer. Therefore, parity error in received data is not detected. To clear the PERF flag in SPSR to 0, issue a system reset or 1 is written to the SPSRC.PERFC bit.

Figure 43.52 shows an example of operation of the OVRF and PERF flags. The SPSR access shown in Figure 43.52 indicates the condition of access to the register, where W denotes a write cycle, and R a read cycle. In this example, full-duplex serial communication is performed while the SPCR.SPPE bit is 1. The SPI performs an 8-bit serial transfer when SPCMDm.CPHA bit is 1 and the SPCMDm.CPOL bit is 0. The numbers given for RSPCKn in the waveform represent the number of RSPCK cycles, such as the number of transferred bits.

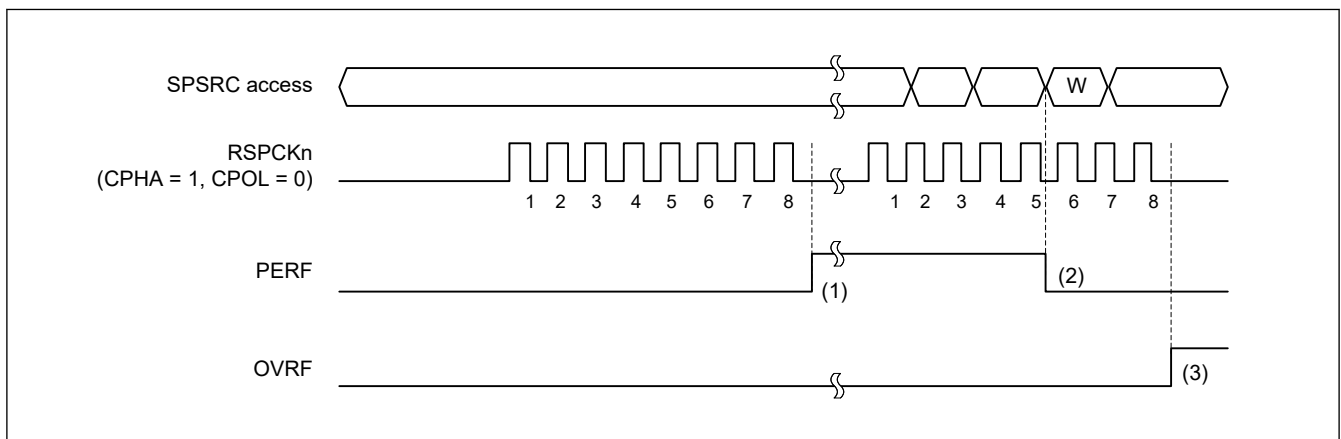


Figure 43.52 Operation example of the OVRF and PERF flags

The operation of the flags at timings (1) to (3) in Figure 43.52 is as follows:

1. When the SPI does not detect an overrun error and terminates the serial transfer, the SPI copies shift register data to the receive buffer. When the SPI checks the received data and detects a parity error at this time, the PERF flag is set to 1. In master mode, the SPI copies the value of pointer to the SPI command register (SPCMDm) to the SPECM[2:0] bits in the SPI Data control register 2 (SPDCR2).
2. When 1 is written to the SPSRC.PERFC bit, then clear the PERF flag.
3. When the SPI detects an overrun error and serial transfer is terminated, the data in the shift register is not copied to the receive buffer. The SPI does not perform parity error detection at this time.

Parity errors can be checked for by either reading the SPSR register or using an SPI error interrupt and reading the SPSR register. When executing a serial transfer, such checks are required to ensure early detection of parity errors. When the SPI is used in master mode, the pointer value to the SPCMDm register at the occurrence of the error can be checked by reading the SPDCR2.SPECM[2:0] bits.

43.3.10.3 Mode Fault Errors

The SPI operates in multi-master mode when the SPCR.MSTR bit is 1, the SPCR.SPMS bit is 0, and the SPCR.MODFEN bit is 1.

If the active level is input for the SSLn0 input signal of the SPI in multi-master mode, the SPI detects a mode fault error regardless of the status of the serial transfer, and sets the SPSR.MODF flag to 1.

On detecting the mode fault error, the SPI copies the value of the pointer to SPCMD to the SPECM[2:0] bits.

The active level of the SSLn0 signal is determined by the SPCR3.SSL0P bit.

When the MSTR bit is 0, the SPI operates in slave mode.

When the SPCR.MODFEN bit = 1 and the SPMS bit = 0 in slave mode, if the SSLn0 input signal is negated during the serial transfer period (from valid data drive start to final valid data latch), the SPI detects a mode fault error, while any of the following 2 conditions is met.

[In the Motorola-SPI case]

When the SSLn0 input signal is negated while serial data transfer.

[In the TI-SSP case]

When the SSLn0 input signal is asserted while serial data transfer. However, during burst transfer, no error is detected even if the SSLn0 input signal is asserted during the last bit of frame.

When the SPI detects a mode fault error, it stops driving output signals and clears the SPE bit in the SPCR register. When the SPE bit is cleared, the SPI function is disabled (as described in [section 43.3.12. SPI Operation](#)). In a multi-master configuration, the mastership can be released by stopping driving output signals and disabling the SPI function by using a mode fault error.

Whether a mode fault error is present can be checked by reading SPSR or by reading an SPI error interrupt and SPSR. To detect a mode fault error without using an SPI error interrupt, poll SPSR. When the SPI is used in master mode, the pointer value to SPCMD when an error is present can be checked by reading the SPECM[2:0] bits in SPSR.

While the MODF flag = 1, the SPI ignores writing 1 to the SPE bit. To enable the SPI function after a mode fault error is detected, clear the MODF flag to 0 without fail.

43.3.10.4 Underrun Errors

While the SPI is operating in slave mode (SPCR.MSTR bit = 0) and the communication mode select bit (TXMD[1:0]) in the SPI control register (SPCR) is set to 00b or 01b, if serial transfer is started before transmit data output is ready with the SPCR.SPE bit set to 1 (SPI function enabled), the SPI detects an underrun error and sets the SPSR.MODF and SPSR.UDRF flags to 1.

On detecting an underrun error, the SPI stops the driving of output signals and clears the SPCR.SPE bit to 0 (see [section 43.3.11. Initializing the SPI](#)).

The occurrence of underrun errors can be checked either by reading SPSR or by using an SPI error interrupt and reading SPSR. Detecting underrun errors without using the SPI error interrupt requires polling of SPSR.

When the MODF flag is 1, writing 1 to the SPE bit is ignored by the SPI. To enable the SPI function after the detection of an underrun error, the MODF flag must be set to 0.

43.3.10.5 Received Data Ready

When SPCR.TXMD[1:0] = 00b, 01b, or 11b, and SPCR2.SPDR[7:0] ≠ 0x00, after receiving data in the receive FIFO during communication (SPE = 1), SPSR.SPDRF flag is set to 1 when the received data is not stored even after the number of received FIFOs is equal to or less than ≤ the threshold value and the value set in SPDR[7:0] has elapsed.

When the receive data ready is detected, the interrupt and event link output can be selected as SPli_SPRI or SPli_SPEI with the SPCR.SPDRF bit.

[Figure 43.53](#) shows an example of reception data ready detection operation.

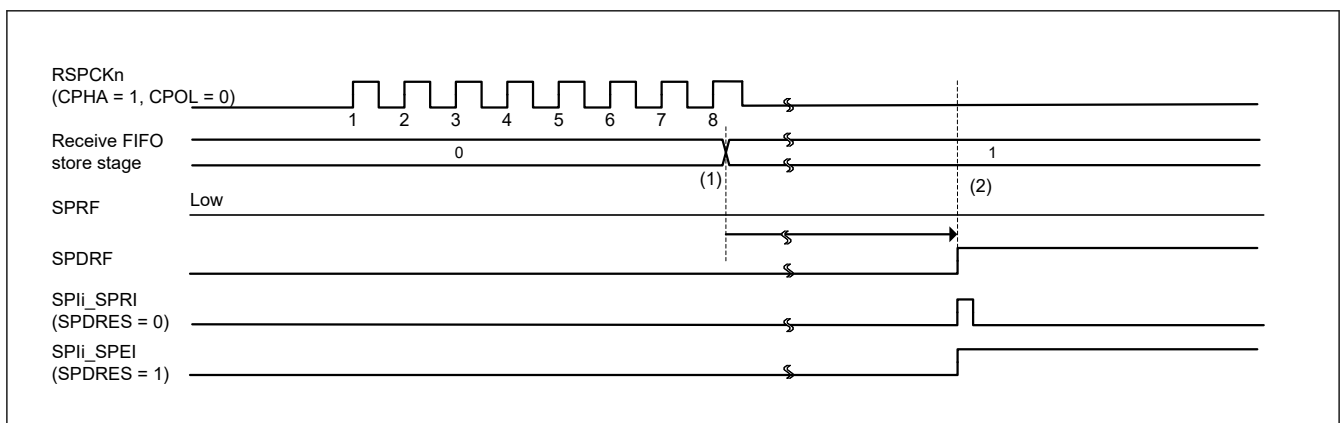


Figure 43.53 Received data ready

The following describes the operation at the timings indicated by (1) and (2) in the figure.

(1) Store the received data in the receive FIFO. SPRF is 0, because receive FIFO store stage $\leq$ number of frames set by SPDCR2.RTRG[1:0].

(2) Set SPDRF and assert SPI_i_SPRI or SPI_i_SPEI because there is no writing to the receive FIFO for the amount of SPDRFC [7:0] set from above (1).

43.3.11 Initializing the SPI

If 0 is written to the SPCR.SPE bit or if the SPI sets the SPE bit to 0 because it detected a mode fault error or an underrun error, the SPI disables the SPI function and initializes some of the module functions. When a system reset is generated, the SPI initializes all of the module functions. This section describes initialization by clearing of the SPCR.SPE bit and by a system reset.

43.3.11.1 Initialization by Clearing of the SPCR.SPE bit

When the SPCR.SPE bit is set to 0, the SPI initializes by:

- Suspending any serial transfer that is being executed
- Stopping the driving of output signals (Hi-Z) in slave mode
- Initializing the internal state of the SPI
- Initializing the transmit buffer of the SPI (the SPSR.SPTEF flag sets to 1)

Initialization by clearing of the SPE bit does not initialize the control bits of the SPI. For this reason, the SPI can be started in the same transfer mode in use prior to initialization when the SPE bit is set to 1 again.

The SPSR.CENDF, SPSR.SPRF, SPSR.OVRF, SPSR.MODF, SPSR.PERF, and SPSR.UDRF flags are not initialized, and the value of the SPDCR2.SPECM[2:0] and SPDCR2.SPCP[2:0] bits are not initialized. Therefore, even after the SPI is initialized, data from the receive buffer can be read to check the communication completion status and the error status during an SPI transfer.

The transmit buffer is initialized to an empty state (the SPSR.SPTEF flag sets to 1). Therefore, if the SPCR.SPTIE bit is set to 1 after SPI initialization, a transmit buffer empty interrupt is generated. To disable any transmit buffer empty interrupts when the SPI is initialized, write 0 to the SPTIE bit simultaneously while writing 0 to the SPE bit.

43.3.11.2 Initialization by System Reset

A system reset completely initializes the SPI by initializing all SPI control bits, status bits, and data registers, in addition to the requirements described in [section 43.3.11.1. Initialization by Clearing of the SPCR.SPE bit](#).

43.3.12 SPI Operation

43.3.12.1 Master Mode Operation

The only difference between single- and multi-master mode operation is the use of mode fault error detection (see [section 43.3.10. Error Detection](#)). In single-master mode, the SPI does not detect mode fault errors whereas in multi-master mode, it does. This section explains operations that are common to both modes.

(1) Starting a serial transfer

When data is written to the SPI data register (SPDR) while the next transfer data is not set in the transmit FIFO, the SPI updates the transmit buffer (SPTX_n, n = 0 to 3) data in SPDR. When the shift register is empty after the number of frames set in the SPDCR.SPFC[1:0] bits are written to the SPDR, the SPI copies data from the transmit buffer to the shift register and starts serial transfer. On copying transmit data to the shift register, the SPI changes the status of the shift register to full. On termination of the serial transfer, it changes the status of the shift register to empty. The status of the shift register cannot be referenced.

The polarity of the SSL_n output pins depends on the SPCR3.SSLnP (n = 0 to 3) bits settings. For details on the SPI transfer format, see [section 43.3.5. Transfer Formats](#).

(2) Terminating a serial transfer

[Except Receive-only in Master Mode]

After the SPI detects the RSPCKn edge corresponding to the final sampling timing regardless of the CPHA bit value in the SPI command register (SPCMD), the SPI terminates serial transfer. When the number of data stored in the receive FIFO is less than the number of FIFO stages, data is copied from the shift register to the receive buffer in the SPI data register (SPDR) after serial transfer.

The final sampling timing varies depending on the bit length of transfer data. In master mode, the SPI data length depends on the SPCMDm.SPB[4:0] bit settings. The polarity of the SSLni output pin depends on the SPCR3.SSLnP (n = 0 to 3) bits settings. For details on the SPI transfer format, see [section 43.3.5. Transfer Formats](#).

[Receive-only in Master Mode]

When any of the following 2 conditions is met, then SPI terminating the serial transfer.

- After the SPI detects the RSPCKn edge corresponding to the final sampling timing regardless of the CPHA bit value in the SPI command register (SPCMD), the SPI terminates serial transfer.
- When writing SPCR2.RMEDTG = 1 during the serial transfer period, SPI terminating the serial transfer.

When the number of data stored in the receive FIFO is less than the number of FIFO stages, data is copied from the shift register to the receive buffer in the SPI data register (SPDR) after serial transfer.

The final sampling timing varies depending on the transfer data bit length. The data length of the SPI in master mode depends on the set value of the SPB[4:0] bits in the SPI command register (SPCMD). The SSLni output signal polarity depends on the set SPI SSLi signal polarity bit (SPCR3.SSLiP) (i = 0 to 3) value. For details about the SPI transfer format, see [section 43.3.5. Transfer Formats](#).

(3) Sequence control

The transfer format in master mode is determined as follows.

The transfer format used in master mode is determined by the SPCR3, SPCMDm, and SPDECR registers.

The SPCR3.SPSSLN[2:0] bits determine the sequence configuration for serial transfers that are executed by the SPI in master mode. The following items are set in the SPCMDm register:

- SSLni pin output signal value
- MSB- or LSB-first
- Data length
- Some of the bit rate settings
- RSPCKn polarity and phase
- Whether SPDECR.SCKDL is to be referenced
- Whether SPDECR.SLNDL is to be referenced
- Whether SPDECR.SPNDL is to be referenced

SPCR3.SPBR holds some of the bit rate settings, including SPDECR.SCKDL (SPI clock delay), SPDECR.SLNDL (SSL negation delay), and SPDECR.SPNDL (next-access delay).

Based on the sequence length assigned in SPCR3.SPSSLN, the SPI makes up a sequence comprised of a part or all of the SPCMDm register. The SPI contains a pointer to the SPCMDm register that makes up the sequence. The value of this pointer can be checked by reading the SPDCR2.SPCP[2:0] bits. When the SPCR.SPE bit is set to 1 and the SPI function is enabled, the SPI loads the pointer to the commands in SPCMD0, and incorporates the SPCMD0 settings into the transfer format at the beginning of serial transfer. The SPI increments the pointer each time the next-access delay period for a data transfer ends. On completion of the serial transfer that corresponds to the final command in the sequence, the SPI sets the pointer to SPCMD0, and in this way the sequence is executed repeatedly.

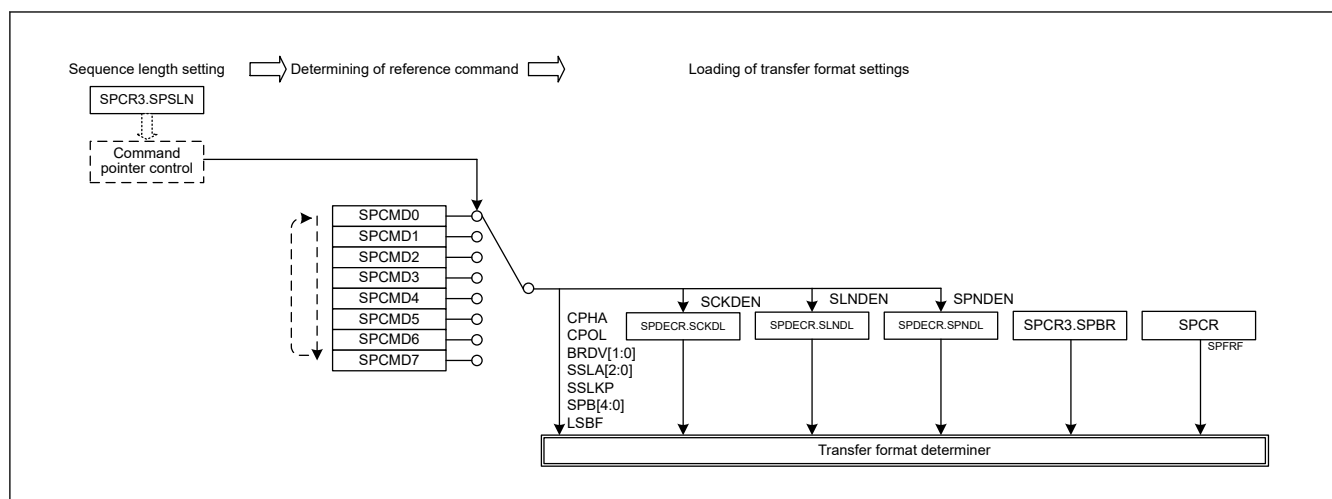


Figure 43.54 Procedure for determining the form of a serial transfer in master mode

In this section, a frame is the combination of the data in SPDR and the settings in SPCMDm.

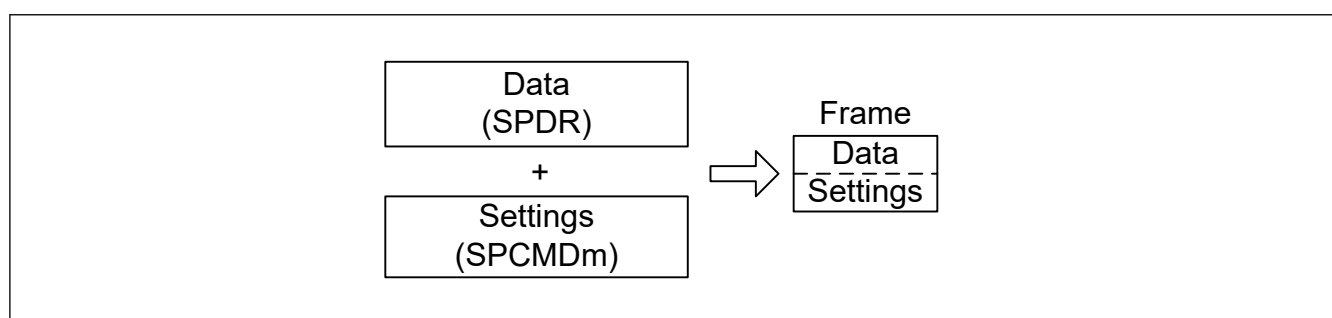


Figure 43.55 Conceptual diagram of frames

Figure 43.56 shows the correspondence between the commands and the transmit and receive buffers in the sequence of operations specified by the settings.

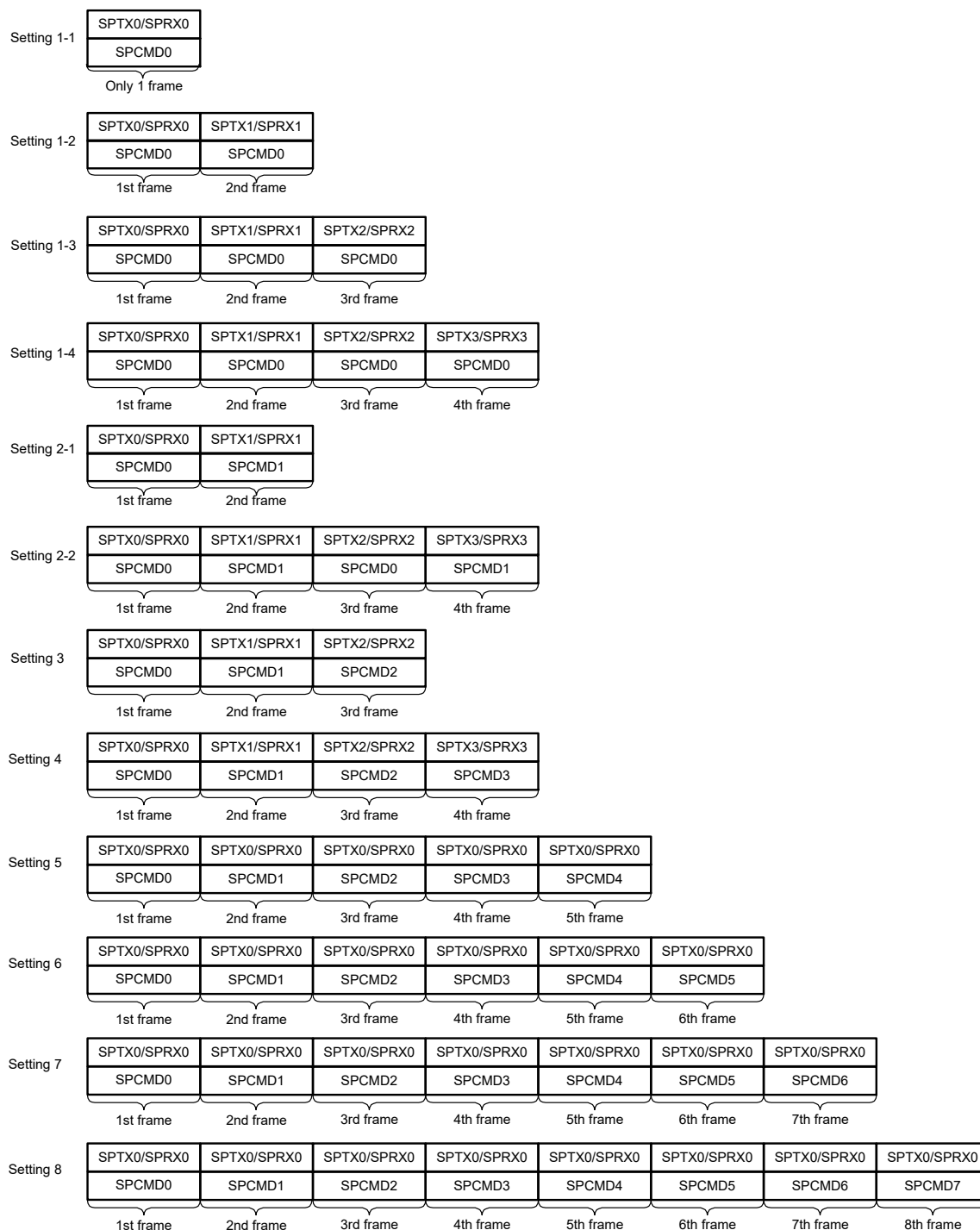


Figure 43.56 Correspondence between SPI Command Register and transmit and receive buffers in sequence operations

(4) Burst transfers

This section describes burst transfer during transmit-receive/transmit-only operation.

[In the Motorola-SPI case]

If the SPCMDm.SSLKP bit that the SPI references during the current serial transfer is 1, the SPI maintains the SSLni signal level during the serial transfer until the beginning of the SSLni signal assertion for the next serial transfer. If the SSLni signal level for the next serial transfer is the same as the SSLni signal level for the current serial transfer, the SPI can execute continuous serial transfers while keeping the SSLni signal assertion status (burst transfer).

- When Between Burst Transfer Frames Delay Select bit (BFDS) of SPI control register (SPCR) is 0.

Figure 43.57 shows an example of an SSLni signal operation for a burst transfer that is implemented using the SPCMD0 and SPCMD1 register settings. This section describes SPI operations (1) to (8) shown in Figure 43.57.

Note: The polarity of the SSLni output signal depends on the SPCR3.SSLnP (n = 0 to 3) bits settings.

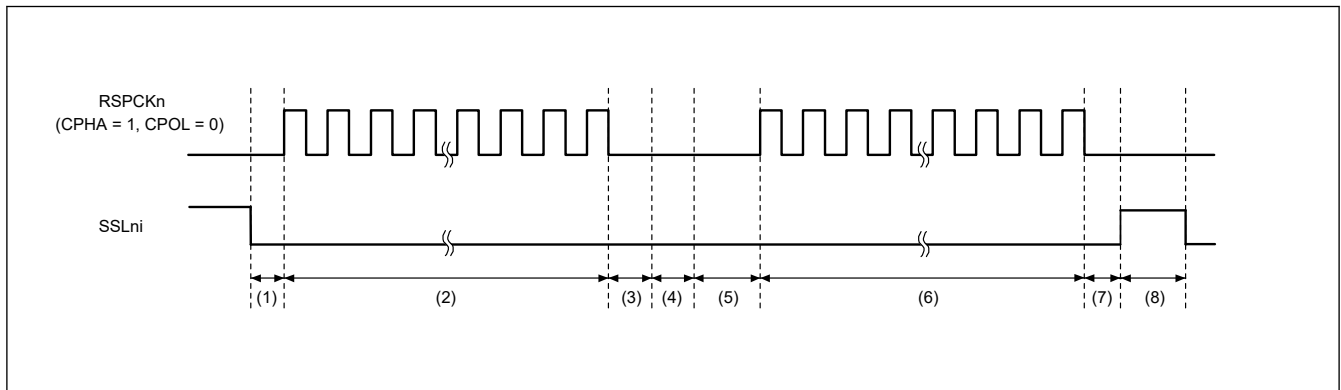


Figure 43.57 Example of burst transfer operation using the SSLKP bit (BFDS = 0, SPFRF = 0)

The SPI operation at times (1) to (8) in the figure is as follows:

1. Based on the SPCMD0 settings, the SPI asserts the SSLni signal and inserts RSPCK delays.
2. The SPI executes serial transfers in accordance with the SPCMD0 settings.
3. The SPI inserts an SSL negation delay.
4. Because the SPCMD0.SSLKP bit is 1, the SPI keeps the SSLni signal value specified in SPCMD0. This period additionally continues for 5 TCLK cycles (at minimum) that is the same as the next-access delay time of SPCMD0. If the shift register is empty after the passage of the minimum period, this period is sustained until the transmit data is stored in the shift register for the next transfer.
5. Based on the SPCMD1 settings, the SPI asserts the SSLni signal and inserts RSPCK delays.
6. The SPI executes serial transfers in accordance with the SPCMD1 settings.
7. Insert SSL negate delay.
8. Because the SPCMD1.SSLKP bit is 0, the SPI negates the SSLni signal. In addition, a next-access delay is inserted in accordance with SPCMD1.

If the SSLni signal output settings in the SPCMDm register where 1 is assigned to the SSLKP bit are different from the SSLni signal output settings in the SPCMDm register to be used in the next transfer, the SPI switches the SSLni signal status to SSLni signal assertion as shown in (5) in Figure 43.57. This corresponds to the command for the next transfer.

Note: If such an SSLni signal switching occurs, the slaves that drive the MISO signal compete, and collision of signal levels might occur.

The SPI in master mode references the SSLni signal operation within the module when the SSLKP bit is not used. When the SPCMDm.CPHA bit is 0, the SPI can accurately start serial transfers by using the SSLni signal assertion for the next transfer that is detected internally.

- When Between Burst Transfer Frames Delay Select bit (BFDS) of SPI control register (SPCR) is 1.

Figure 43.58 shows an example of SSLni signal operation when burst transfer is achieved by using the settings of SPCMD0 and SPCMD1. The following describes SPI operations of (1) to (6) shown in Figure 43.58. The SSLni output signal polarity depends on the set SPCR3.SSLnP (n = 0 to 3) value.

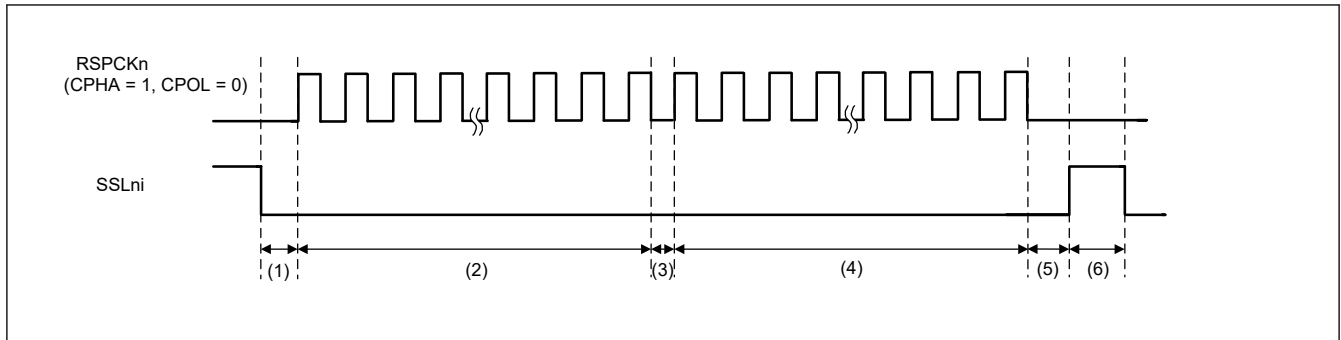


Figure 43.58 Example of Burst Transfer Operation Using SSLKP Bit (BFDS = 1, SPFRF = 0)

1. Assert the SSLNi signal and insert an RSPCK delay according to SPCMD0. The RSPCK delay is inserted only the first frame of burst transmission.
2. Perform serial transfer according to SPCMD0. Wait last clock until the next transmit data is stored in the shift register, if the shift register is empty during RSPCK negate period between frames.
3. The value of SSLNi signal according to SPCMD0 was hold, because the SPCMD0.SSLKP bit is 1. RSPCK negate period between frames is $0.5RSPCK$, if the shift register is not empty.
4. Perform serial transfer according to SPCMD1.
5. Insert SSLNi negate delay for the last frame.
6. The SSLNi signal is negated because the SSLKP bit in SPCMD1 is 0. Furthermore, the next-access delay is inserted according to SPCMD1.

[In the TI-SSP case]

SPI asserts the SSLNi signal for one cycle at the start of serial transfer.

Serial transfer can be executed continuously by asserting the SSLNi signal for one cycle at the start of the next serial transfer (burst transfer).

- When the SSLNi signal level holding bit (SSLKP) of the SPI command register (SPCMD) is 1 and the burst transfer frame delay selection bit (BFDS) of the SPI control register (SPCR) is 1, SPCMD0 to SPCMD1 are shown in [Figure 43.59](#). The following shows an example of SSLNi signal operation and serial data MISO_n/MOSI_n when burst transfer is realized using the settings. The SSLNi output signal polarity depends on the set SPI SSLi signal polarity bit (SPCR3.SSLiP) ($i = 0$ to 3) value.

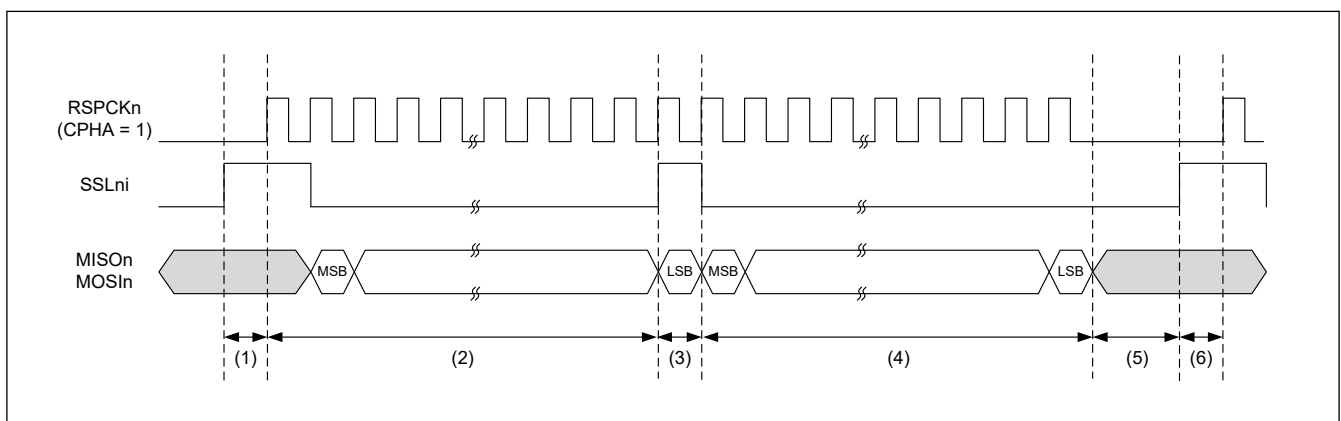


Figure 43.59 Example of Burst Transfer Operation (SPFRF = 1)

1. Assert the SSLNi signal and insert an RSPCK delay according to SPCMD0. The RSPCK delay is inserted only the first frame of burst transmission.
2. Perform serial transfer according to SPCMD0.
3. Final data transfer and SSLNi assertion are performed simultaneously. If the shift register is empty during the RSPCK negation period between frames, wait for the output of the last clock until the transmission data for the next transfer is stored in the shift register.

4. Perform serial transfer according to SPCMD1.
5. Insert OE negate delay for the last frame.
6. Insert the next access delay according to SPCMD1.

If the SSLni signal output setting in SPCMD with the SSLKP bit set to 1 differs from the SSLni signal output setting in SPCMD to be used for the next transfer, the SPI changes the SSLni signal state when the SSLni signal corresponding to the next-transfer command is asserted ((5)). Note that, if an SSLni signal change like this takes place, slaves that drive the MISO_n signal may conflict with each other, which may cause collision of signal level.

This section describes burst transfer during receive-only operation.

[In the Motorola-SPI case]

When the SSLKP bit in the SPI command register (SPCMD), which the SPI references in the current serial transfer, is 1, the SPI retains the SSLni signal level during serial transfer until the SSLni signal assertion of the next serial transfer starts. When the SSLni signal level in the next serial transfer is the same as the SSLni signal level in the current serial transfer, the SPI can continuously perform serial transfer while holding the SSLni signal assertion status (burst transfer).

- When Between Burst Transfer Frames Delay Select bit (BFDS) of SPI control register (SPCR) is 0.

Figure 43.60 shows an example of SSLni signal operation when burst transfer is achieved by using the settings of SPCMD0 and SPCMD1. The following describes SPI operations of (1) to (8) shown in Figure 43.60. The SSLni output signal polarity depends on the set SPI SSLni signal polarity bit (SPCR3.SSLiP) (i = 0 to 3) value.

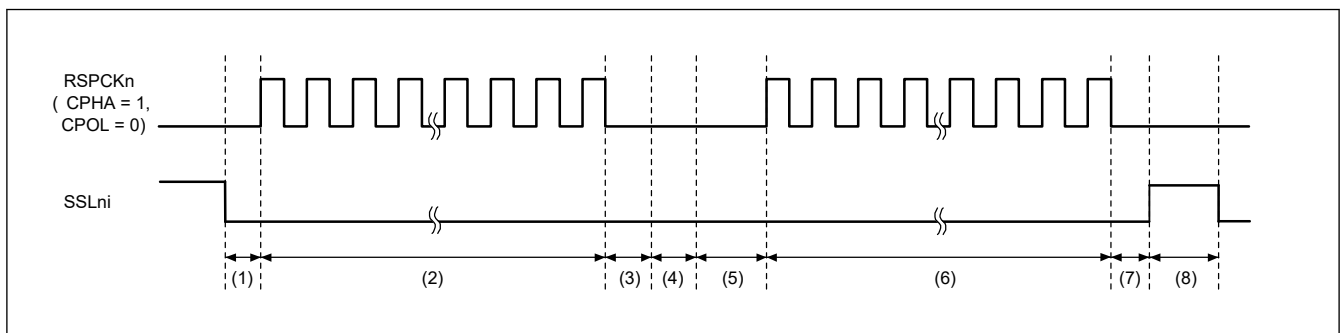


Figure 43.60 Example of Burst Transfer Operation Using SSLKP Bit (BFDS = 0, SPFRF = 0)

1. Assert the SSLni signal and insert an RSPCK delay according to SPCMD0.
2. Perform serial transfer according to SPCMD0.
3. Insert an SSLni negation delay.
4. The SSLni signal value in SPCMD0 is retained because the SSLKP bit in SPCMD0 is 1. This period additionally continues for 5 TCLK cycles (at minimum) that is the same as the next-access delay time of SPCMD0.
5. Assert the SSLni signal and insert an RSPCK delay according to SPCMD1.
6. Perform serial transfer according to SPCMD1.
7. Insert SSLni negate delay.
8. The SSLni signal is negated because the SSLKP bit in SPCMD1 is 0. Furthermore, the next-access delay is inserted according to SPCMD1.

If the SSLni signal output setting and the SSLni signal output setting between SPCMDs used for burst transfer are different, SPI switches the SSLni signal state when the SSLni signal corresponding to the next transfer command is asserted (5). Note that, if an SSLni signal change like this takes place, slaves that drive the MISO_n signal may conflict with each other, which may cause collision of signal level.

The SPI in master mode references the SSLni signal operation in the module when SSLKP is not used.

Even when the CPHA bit in SPCMD is 0, the SPI can accurately start serial transfer by using the next transfer SSLni signal assertion detected internally. For this reason, burst transfer in master mode is enabled regardless of the set CPHA bit value. (See section 43.3.11. Initializing the SPI.)

- When Between Burst Transfer Frames Delay Select bit (BFDS) of SPI control register (SPCR) is 1.

Figure 43.61 shows an example of SSLni signal operation when burst transfer is achieved by using the settings of SPCMD0 and SPCMD1. The following describes SPI operations of (1) to (6) shown in Figure 43.61. The SSLni output signal polarity depends on the set SPI SSLi signal polarity bit (SPCR3.SSLiP) (i = 0 to 3) value.

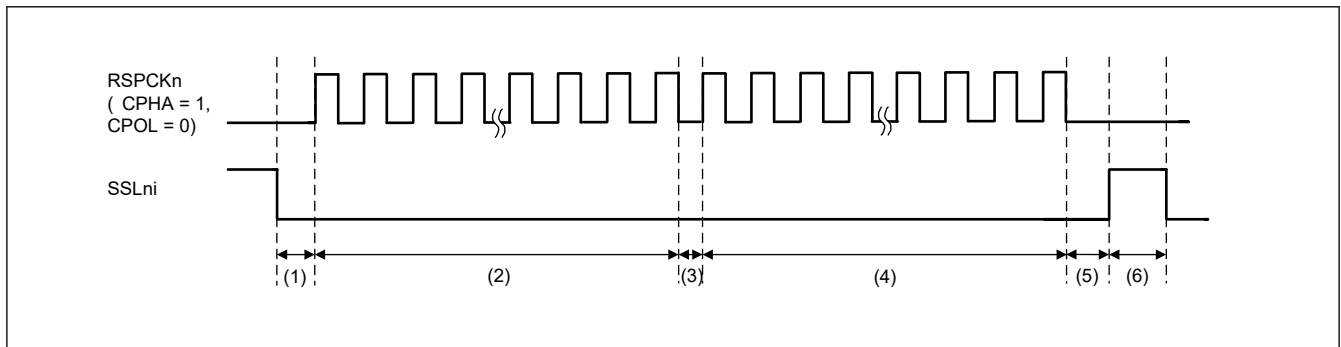


Figure 43.61 Example of Burst Transfer Operation Using SSLKP Bit (BFDS = 1, SPFRF = 0)

1. Assert the SSLni signal and insert an RSPCK delay according to SPCMD0.
The RSPCK delay is inserted only the first frame of burst transmission.
2. Perform serial transfer according to SPCMD0.
3. Since it is not the last frame, the SSLni signal value at SPCMD0 is retained. RSPCKn negation between frames is 0.5 RSPCKn for the next frame.
4. Perform serial transfer according to SPCMD1.
5. Insert SSLni negate delay for the last frame.
6. The SSLni signal is negated. Furthermore, the next-access delay is inserted according to SPCMD1.

Note: Last frame: Frame set by RMFM[4:0] bits when SPCR2.RMFM[4:0] ≠ 0x00
Or, a frame in which SPCR2.RMEDTG = 1 has been accepted.

[In the TI-SSP case]

SPI asserts the SSLni signal for one cycle at the start of serial transfer.

Serial transfer can be executed continuously by asserting the SSLni signal for one cycle at the start of the next serial transfer (burst transfer).

- When the SSLni signal level holding bit (SSLKP) of the SPI command register (SPCMD) is 1 and the burst transfer frame delay selection bit (BFDS) of the SPI control register (SPCR) is 1, SPCMD0 to SPCMD1 are shown in Figure 43.62. The following shows an example of SSLni signal operation and serial data MISO_n / MOSI_n when burst transfer is realized using the settings. The SSLni output signal polarity depends on the set SPI SSLi signal polarity bit (SPCR3.SSLiP) (i = 0 to 3) value.

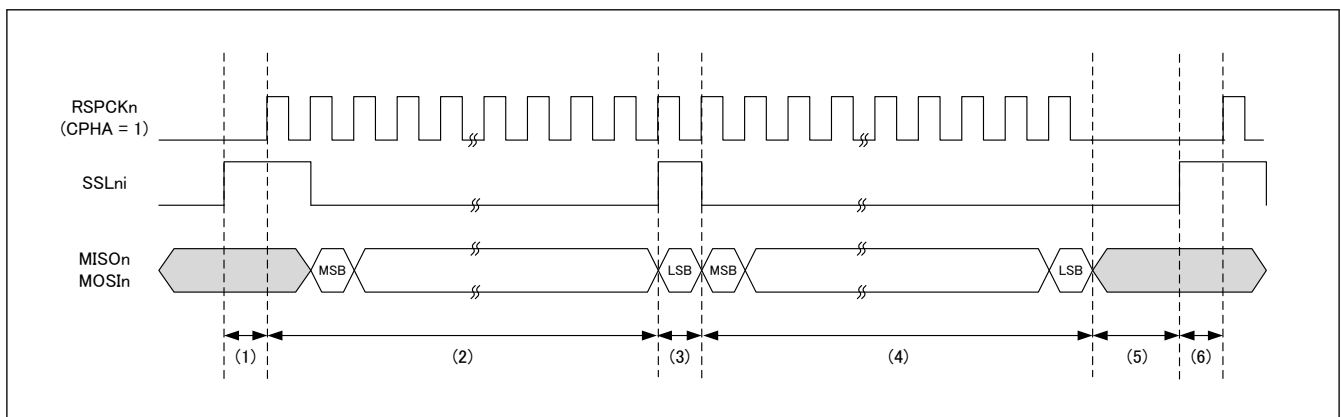


Figure 43.62 Example of Burst Transfer Operation (SPFRF = 1)

1. Assert the SSLni signal and insert an RSPCK delay according to SPCMD0.
The RSPCK delay is inserted only the first frame of burst transmission.

2. Perform serial transfer according to SPCMD0.
3. Final data transfer and SSLni assertion are performed simultaneously.
4. Perform serial transfer according to SPCMD1.
5. Insert OE negate delay for the last frame.
6. Insert the next access delay according to SPCMD1.

Note: Last frame: Frame set by RMFM[4:0] bits when SPCR2.RMFM[4:0] $\neq$ 0x00
Or, a frame in which SPCR2.RMEDTG = 1 has been accepted.

If the SSLni signal output setting between the SPCMDs used for burst transfer differs from the SSLni signal output setting, SPI switches the SSLni signal state when the SSLni signal corresponding to the next transfer command is asserted (5). Note that, if an SSLni signal change like this takes place, slaves that drive the MISO_n signal may conflict with each other, which may cause collision of signal level.

(5) RSPCK delay (t1)

The RSPCK delay value of the SPI in master mode depends on the SPCMDm.SCKDEN bit setting and the SPDECR.SCKDL[2:0] bits setting. The SPI determines the SPCMDm register to be referenced during a serial transfer by pointer control, and determines an RSPCK delay using the SPCMDm.SCKDEN bit and SPDECR.SCKDL[2:0] bits, as listed in [Table 43.10](#). For a definition of RSPCK delay, see [section 43.3.5. Transfer Formats](#).

RSPCK delay insert to only the first frame of burst transmission, when transmit without “Between Burst Transfer Frames Delay”. (The SPCMD.SSLKP bit is 1 and the SPCR.BFDS bit is 1.)

Table 43.10 Relationship between the SPCMDm.SCKDEN bit, SPDECR.SCKDL[2:0] bits, and RSPCK delay

SPCMDm.SCKDEN bit	SPDECR.SCKDL[2:0] bits	RSPCK delay	
		Motorola-SPI	TI-SSP
0	000b to 111b	1 RSPCK	0 RSPCK
1	000b	1 RSPCK	1 RSPCK
	001b	2 RSPCK	2 RSPCK
	010b	3 RSPCK	3 RSPCK
	011b	4 RSPCK	4 RSPCK
	100b	5 RSPCK	5 RSPCK
	101b	6 RSPCK	6 RSPCK
	110b	7 RSPCK	7 RSPCK
	111b	8 RSPCK	8 RSPCK

(6) SSL negation delay (t2)

The SSL negation delay value of the SPI in master mode depends on the SPCMDm.SLNDEN bit setting and the SPDECR.SLNDL[2:0] bits setting. The SPI determines the SPCMDm register to be referenced by pointer control during a serial transfer, and determines an SSL negation delay using the SPCMDm.SLNDEN bit and SPDECR.SLNDL[2:0] bits, as listed in [Table 43.11](#). For a definition of SSL negation delay, see [section 43.3.5. Transfer Formats](#).

An SSL negation delay is inserted to only the last frame of the burst transmission, that is, transmit without between burst transfer frames delay. (SPCMD.SSLKP bit is 1 and SPCR.BFDS bit is 1).

Table 43.11 Relationship between the SPCMDm.SLNDEN bit, SPDECR.SLNDL[2:0] bits, and SSL negation delay (1 of 2)

SPCMDm.SLNDEN bit	SPDECR.SLNDL[2:0] bits	SSL negation delay
0	000b to 111b	1 RSPCK

Table 43.11 Relationship between the SPCMDm.SLN DEN bit, SPDECR.SLNDL[2:0] bits, and SSL negation delay (2 of 2)

SPCMDm.SLN DEN bit	SPDECR.SLNDL[2:0] bits	SSL negation delay
1	000b	1 RSPCK
	001b	2 RSPCK
	010b	3 RSPCK
	011b	4 RSPCK
	100b	5 RSPCK
	101b	6 RSPCK
	110b	7 RSPCK
	111b	8 RSPCK

(7) Next-access delay (t3)

The next-access delay value of the SPI in master mode depends on the SPCMDm.SPNDEN bit setting and the SPDECR.SPNDL[2:0] bits setting. The SPI determines the SPCMDm register to be referenced during serial transfer by pointer control, and then determines a next-access delay during serial transfer using the SPCMDm.SPNDEN bit and SPDECR.SPNDL[2:0] bits, as listed in [Table 43.12](#). For a definition of next-access delay, see [section 43.3.5. Transfer Formats](#).

A next-Access delay is inserted to only the last frame of the burst transmission, that is, transmit without between burst transfer frames delay. (SPCMD.SSLKP bit is 1 and SPCR.BFDS bit is 1).

Table 43.12 Relationship between the SPCMDm.SPNDEN bit, SPDECR.SPNDL[2:0] bits, and next-access delay

SPCMDm.SPNDEN bit	SPDECR.SPNDL[2:0] bits	Next-access delay
0	000b to 111b	1 RSPCK + 5 TCLK
1	000b	1 RSPCK + 5 TCLK
	001b	2 RSPCK + 5 TCLK
	010b	3 RSPCK + 5 TCLK
	011b	4 RSPCK + 5 TCLK
	100b	5 RSPCK + 5 TCLK
	101b	6 RSPCK + 5 TCLK
	110b	7 RSPCK + 5 TCLK
	111b	8 RSPCK + 5 TCLK

(8) Initialization flow

[Figure 43.63](#) shows an example of SPI initialization flow when the SPI is in master mode. For information on how to set up the Interrupt Controller Unit (ICU), DMAC or DTC, and I/O ports, see the descriptions given in the individual blocks.

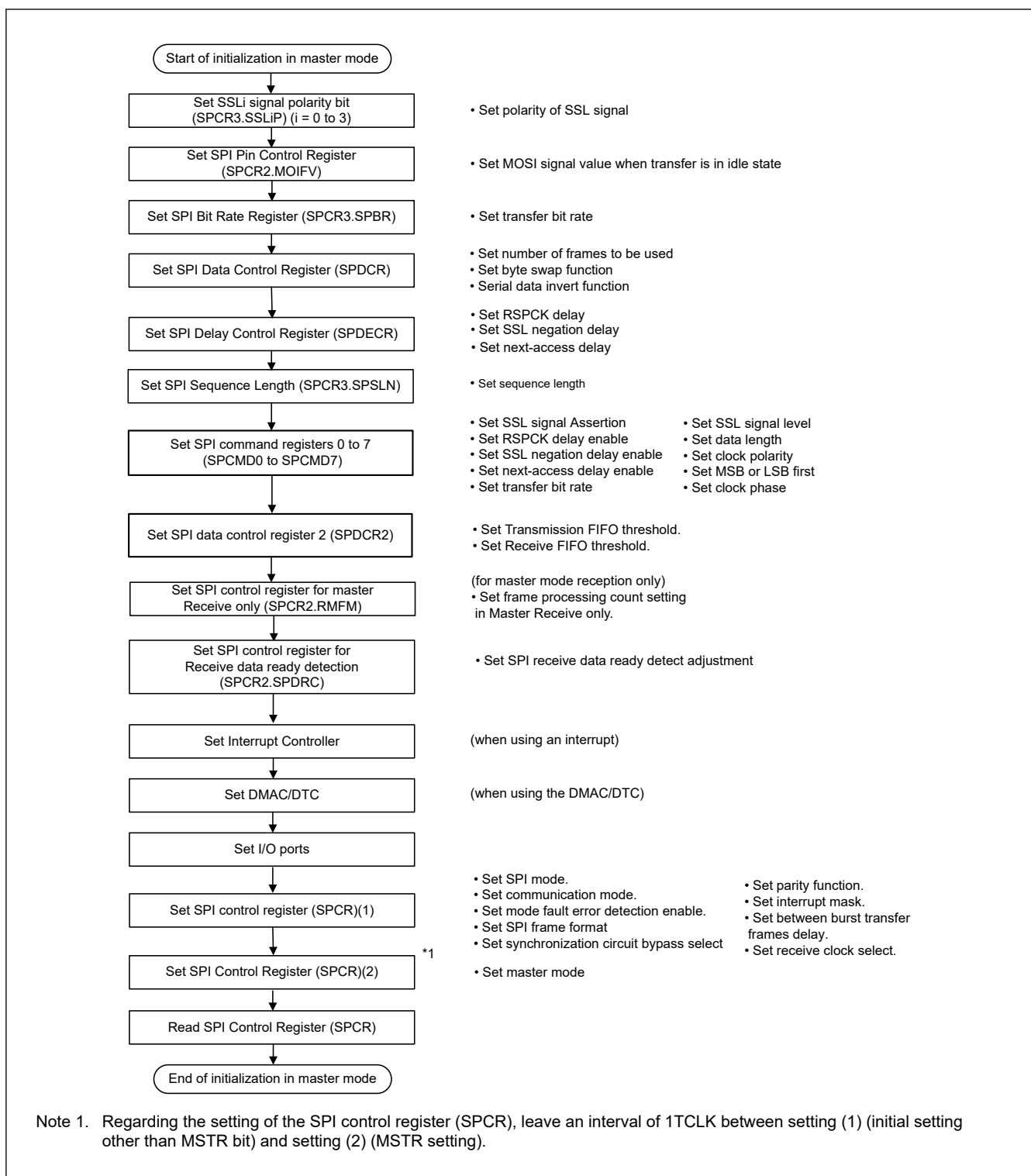


Figure 43.63 Example of initialization flow in master mode for SPI operation

(9) Software processing flow

Figure 43.64 to Figure 43.67 show examples of the software processing flow.

Transmit processing flow

When transmitting data, with the SPIi_SPII or SPIi_SPCEND interrupt enabled, the CPU is notified of the completion of data transmission after the last data write for transmission.

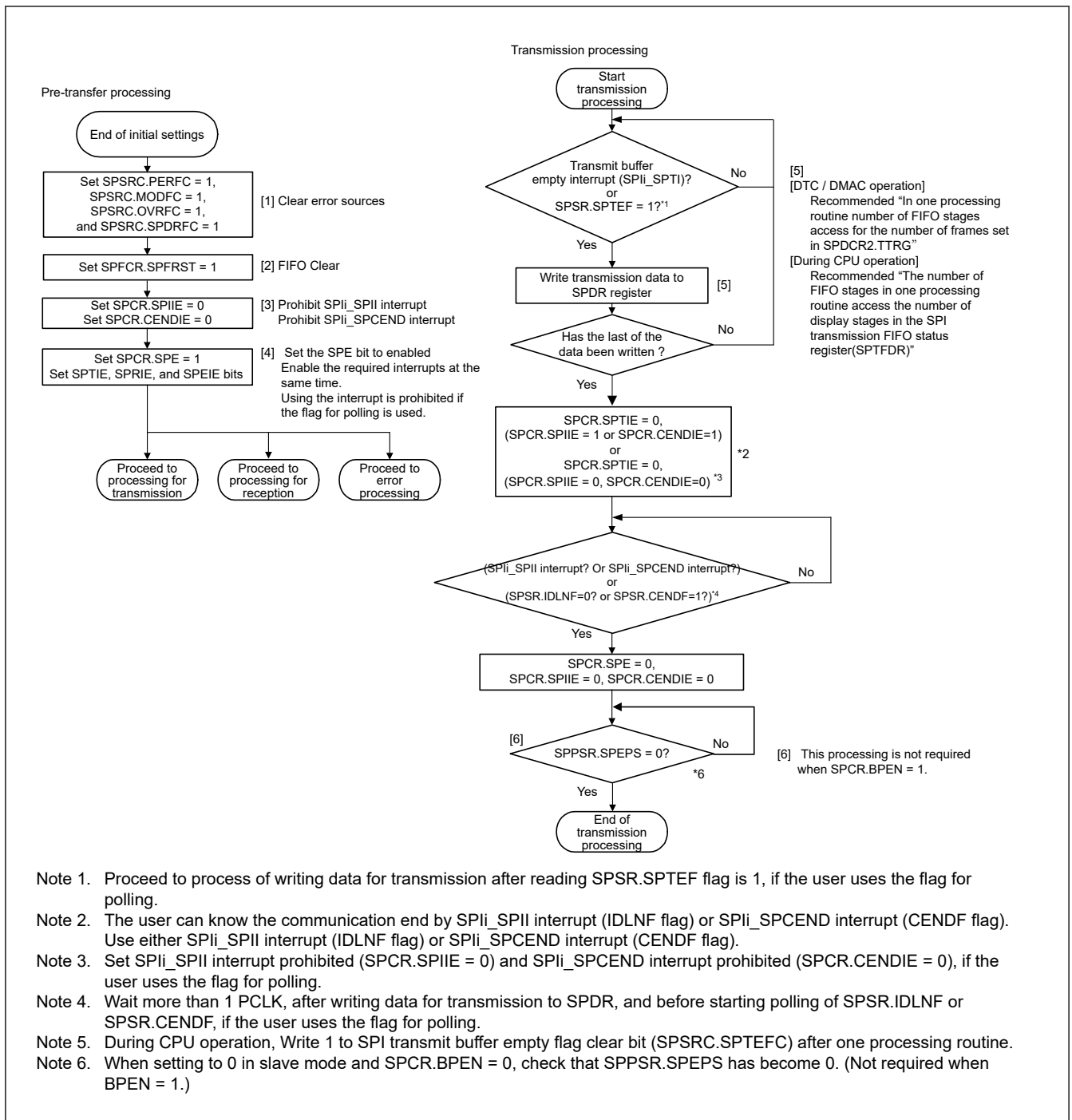


Figure 43.64 Transmission flow in master mode

Receive processing flow

The SPI has receive only operation in slave mode.

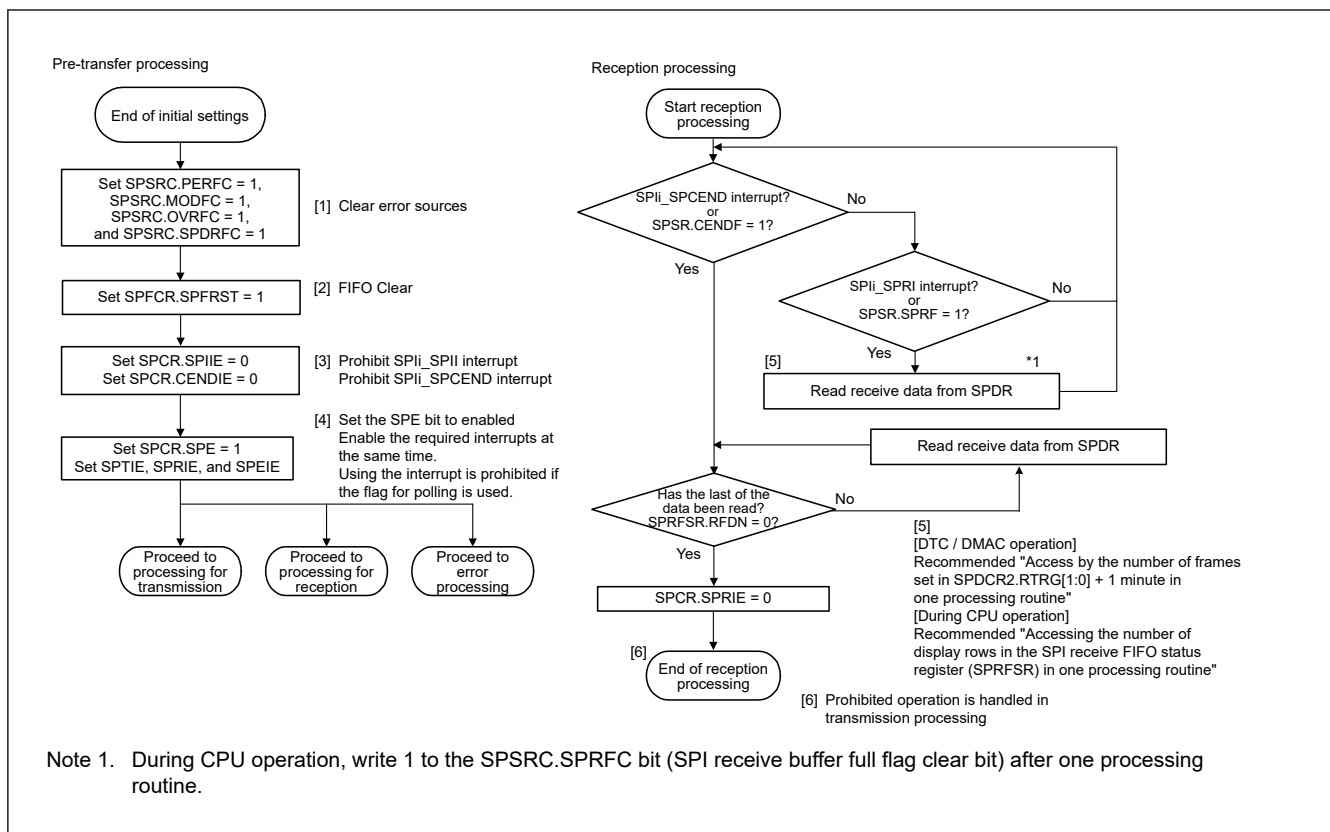


Figure 43.65 Reception flow in master mode

Receive only processing flow in master mode

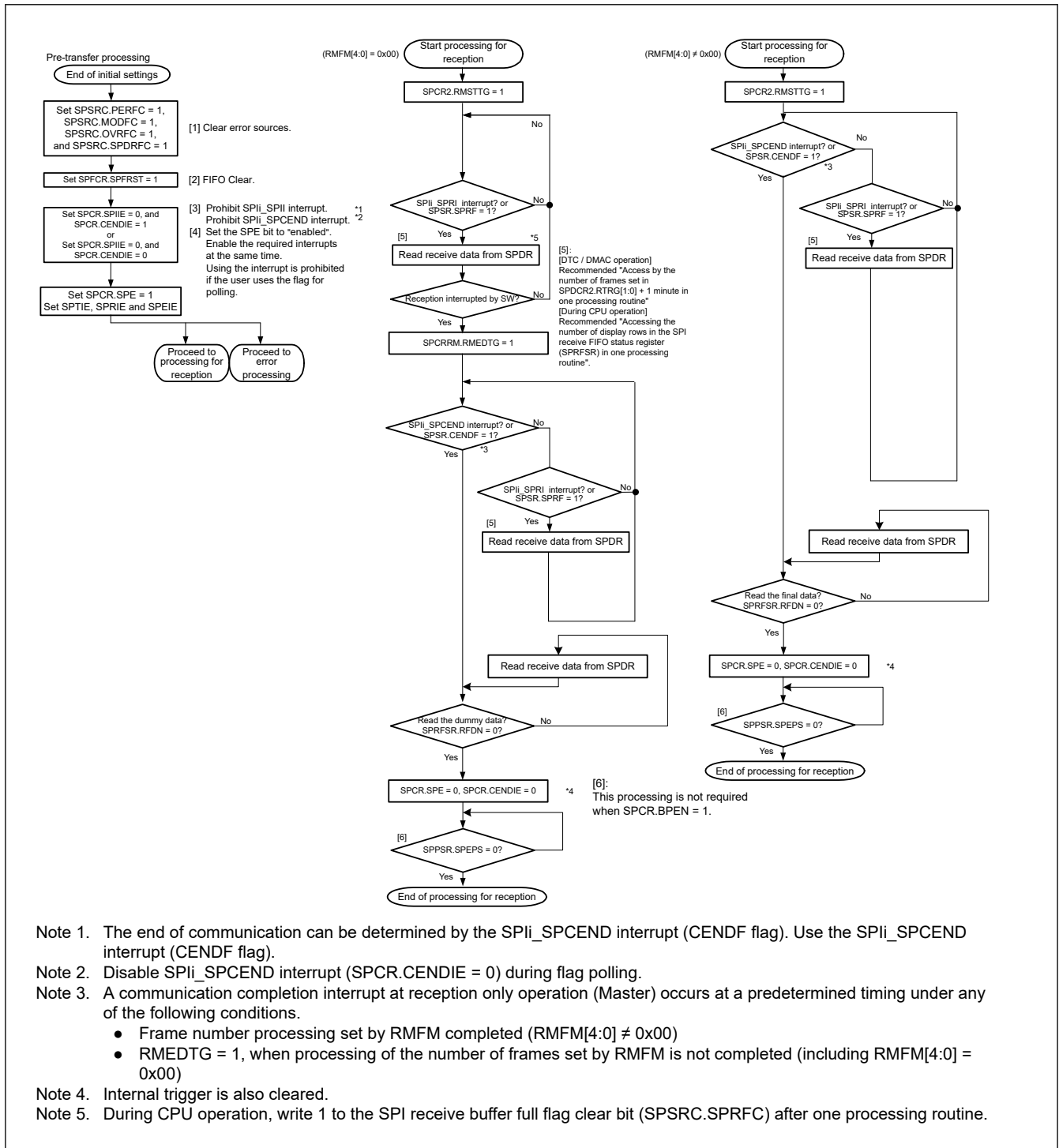


Figure 43.66 Software Processing Flowchart in Master Mode (Reception-only)

Error processing flow

The SPI detects the following errors:

- Mode fault error
- Underrun error
- Overrun error
- Parity error

When a mode fault error is generated, the SPCR.SPE bit is automatically cleared, stopping operations for transmission and reception. For errors from other sources, the SPCR.SPE bit is not cleared and operations for transmission and reception continue. Therefore, Renesas recommends clearing the SPCR.SPE bit to stop operations for errors other than mode fault errors. Not doing so leads to updating of the SPDCR2.SPECM[2:0] bits.

When an error is detected using an interrupt, clear the ICU.IELSRn.IR flag in the error processing routine. If this is not done, the ICU.IELSRn.IR flag might continue to indicate the SPIi_SPTI or SPIi_SPRI interrupt request. If the SPIi_SPRI interrupt request is indicated, read the receive buffer and initialize the sequencer in the SPI.

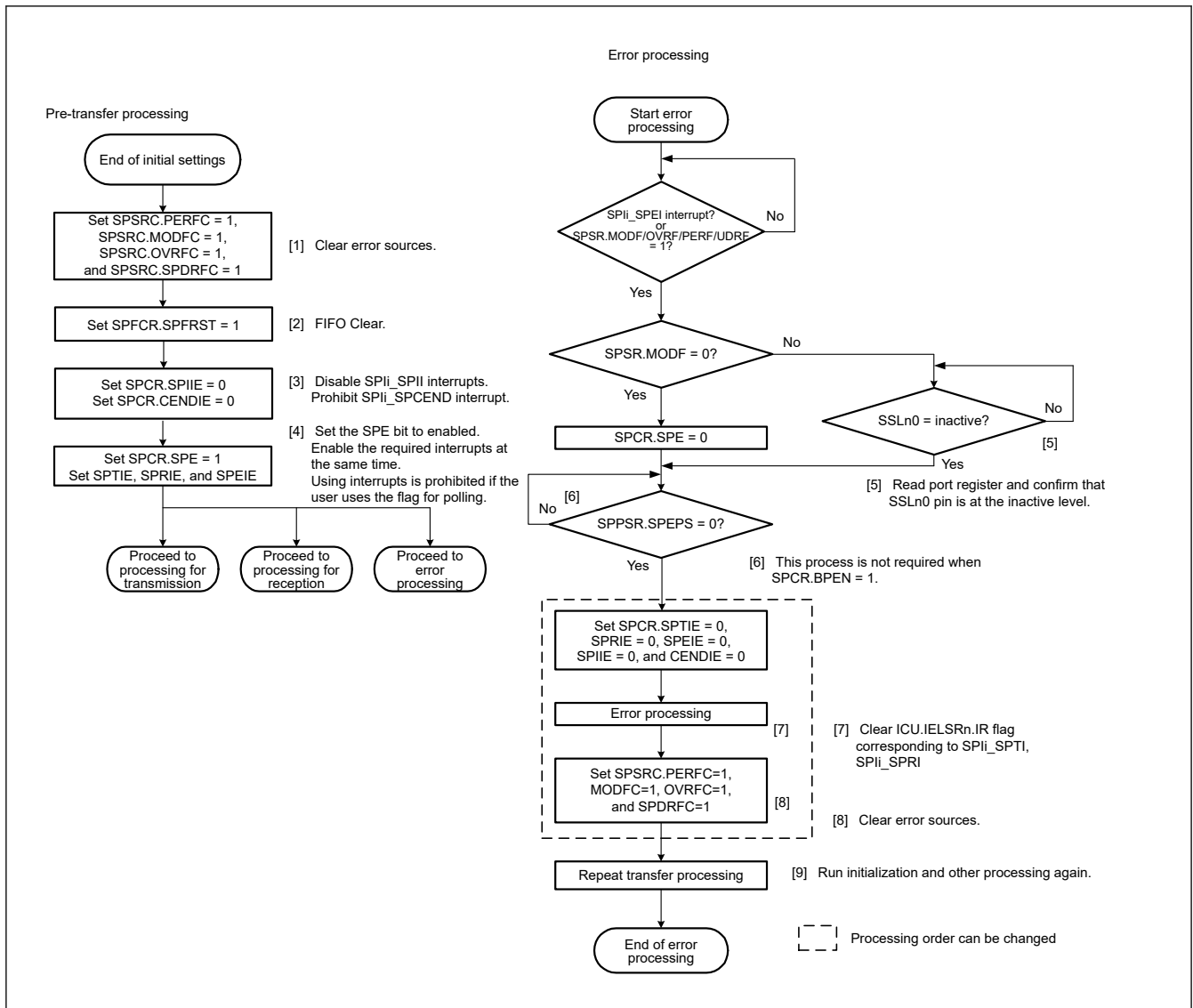


Figure 43.67 Error processing flow in master mode

43.3.12.2 Slave Mode Operation

(1) Starting a serial transfer

When the SPCMD0.CPHA bit is 0, if the SPI detects an SSLn0 input signal assertion, it must drive valid data to the MISO_n output signal. For this reason, when the CPHA bit is 0, the assertion of the SSLn0 input signal triggers the start of a serial transfer.

When the CPHA bit is 1, if the SPI detects the first RSPCK_n edge in an SSLn0 signal asserted condition, it must drive valid data to the MISO_n output signal. For this reason, when the CPHA bit is 1, the first RSPCK_n edge in an SSLn0 signal asserted condition triggers the start of a serial transfer.

Regardless of the CPHA bit setting, the SPI drives the MISO_n output signal on SSLn0 signal assertion. The data that is output by the SPI is either valid or invalid, depending on the CPHA bit setting.

For details on the SPI transfer format, see [section 43.3.5. Transfer Formats](#). The polarity of the SSLn0 input signal depends on the SPCR3.SSL0P setting.

(2) Terminating a serial transfer

Regardless of the SPCMD0.CPHA bit setting, the SPI terminates the serial transfer after detecting an RSPCKn edge corresponding to the final sampling timing. When the number of data stored in the receive FIFO is less than the number of FIFO stages, on termination of serial transfer, the SPI copies received data from the shift register to the receive buffer of the SPDR register. On termination of a serial transfer, the SPI changes the status of the shift register to empty, regardless of the receive buffer state. A mode fault error occurs if the SPI detects an SSLn0 input signal negation from the beginning of serial transfer to the end of serial transfer (see [section 43.3.10. Error Detection](#)).

The final sampling timing changes depending on the bit length of transfer data. In slave mode, the SPI data length is determined by the SPCMD0.SPB[4:0] bits setting. The polarity of the SSLn0 input signal is determined by the SPCR3.SSL0P bit setting. For details on the SPI transfer format, see [section 43.3.5. Transfer Formats](#).

(3) Notes on single-slave operations

[In the Motorola-SPI case]

If the SPCMD0.CPHA bit is 0, the SPI starts serial transfers when it detects the assertion edge for an SSLn0 input signal. In the configuration shown in [Figure 43.8](#), if the SPI is used in single-slave mode, the SSLn0 signal is fixed at an active state. Therefore, when the CPHA bit is set to 0, the SPI cannot correctly start a serial transfer. For the SPI to correctly execute transmit and receive operations in slave mode when the SSLn0 input signal is fixed at an active state, the CPHA bit must be set to 1. Do not fix the SSLn0 input signal if there is a requirement for setting the CPHA bit to 0.

[In the TI-SSP case]

When SPI is used as a single slave in the configuration shown in the [Figure 43.8](#), the SSLn0 input signal is always fixed to the inactive state, so the SPI cannot start the serial transfer.

When using a single slave, use the configuration shown in the example in [Figure 43.7](#).

(4) Burst transfer

[In the Motorola-SPI case]

If the SPCMD0.CPHA bit is 1, continuous serial transfer (burst transfer) can be executed while retaining the assertion state for the SSLn0 input signal. When the CPHA bit is 1, the serial transfer period is the period from the first RSPCKn edge to the sampling timing for the reception of the final bit in an SSLn0 signal active state. Even when the SSLn0 input signal remains at the active level, the SPI can accommodate burst transfers, because it can detect the start of an access.

When the CPHA bit is 0, the second and subsequent serial transfers during burst transfer cannot be executed correctly.

[In the TI-SSP case]

In serial transfer, data transfer starts after the SSLn0 input signal is asserted for RSPCK 1 cycle. Since frame transfer starts from the SSLn0 input signal, SSLn0 must be asserted between frames.

(5) Initialization flow

[Figure 43.68](#) shows an example of initialization flow for SPI operation when the SPI is in slave mode. For a description of how to set up the ICU, DMAC or DTC, and I/O ports, see the descriptions given in the individual blocks.

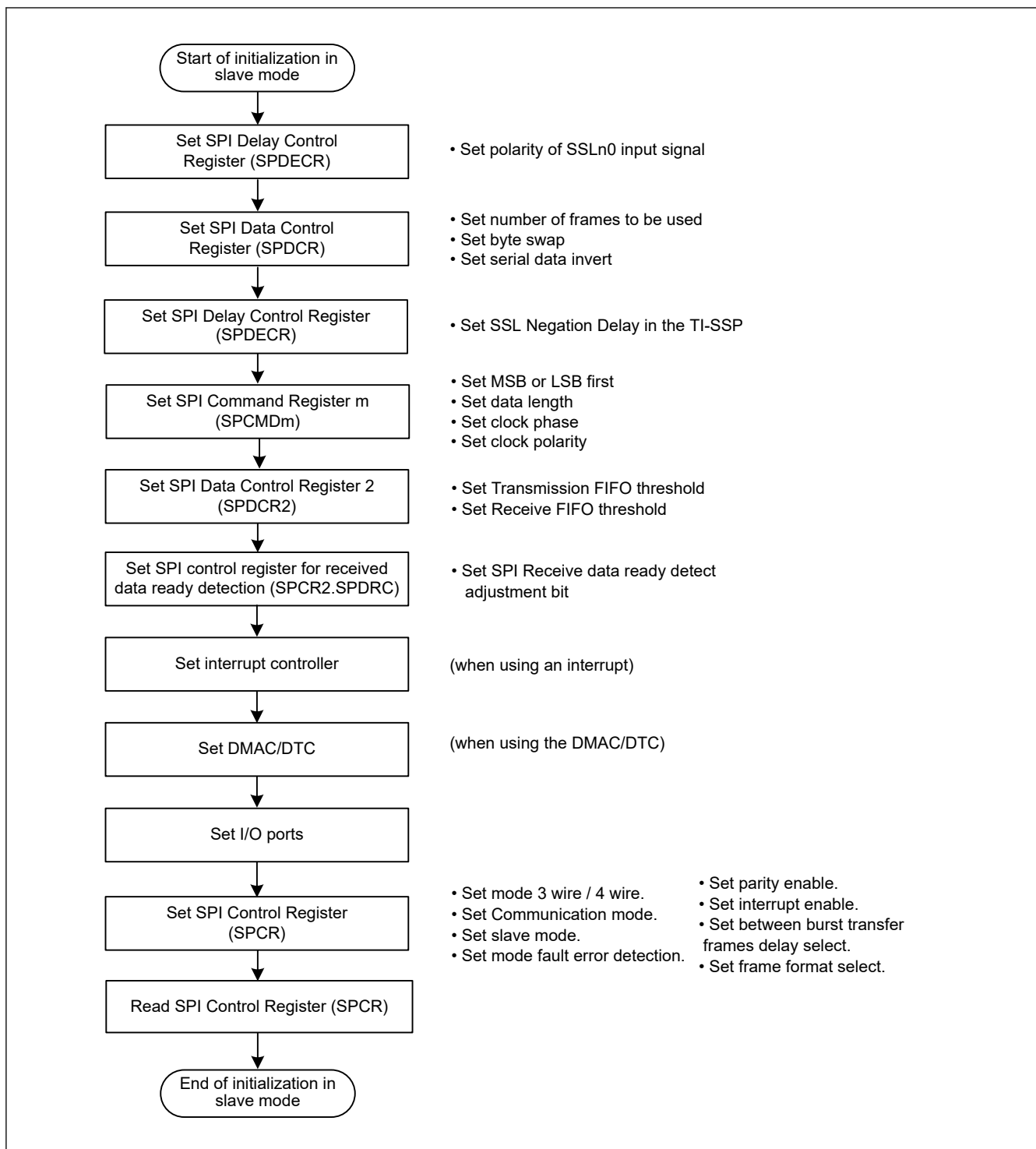


Figure 43.68 Example initialization flow in slave mode for SPI operation

(6) Software processing flow

Figure 43.69 to Figure 43.72 show examples of the flow of software processing.

Transmit processing flow

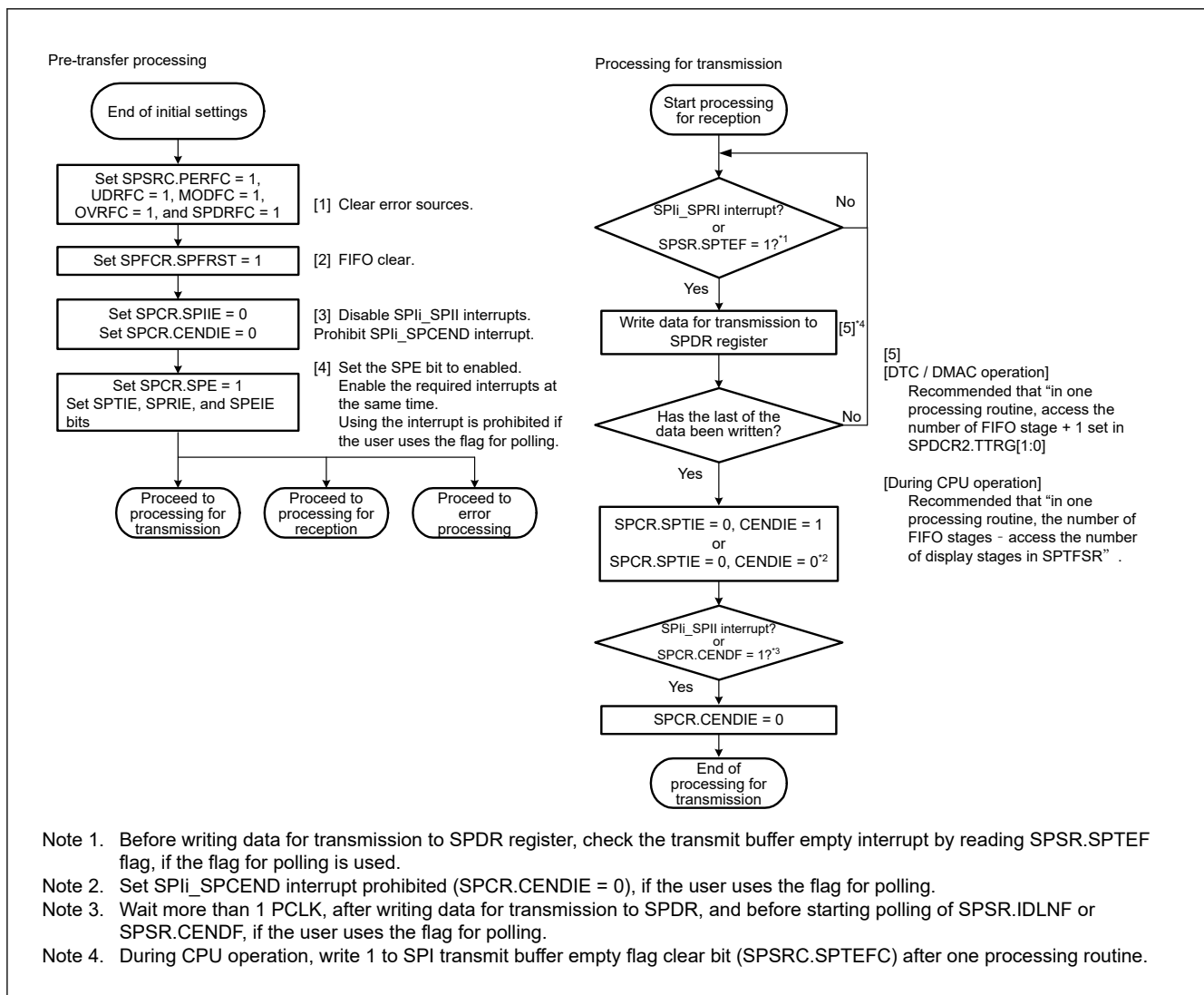


Figure 43.69 Transmission flow in slave mode

Receive processing flow

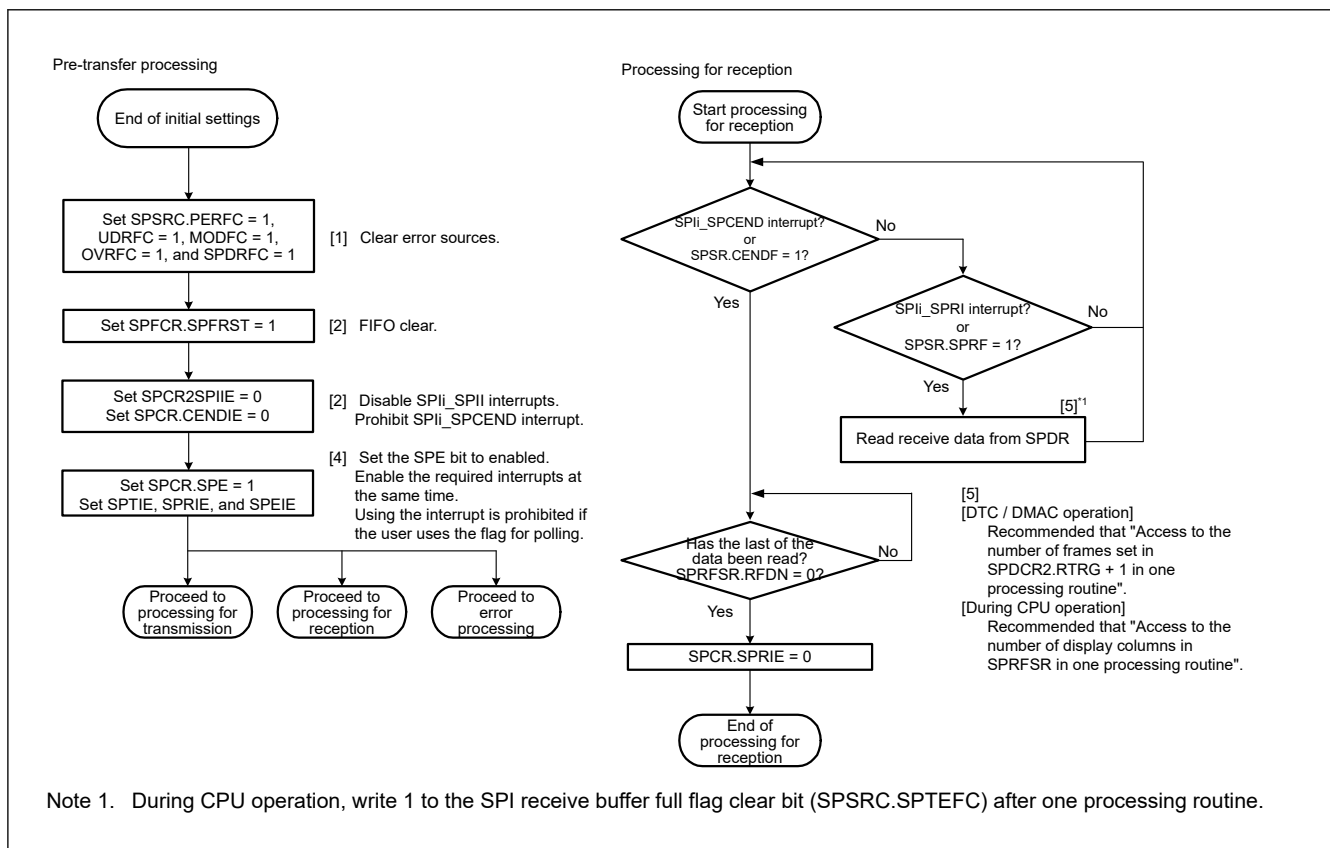


Figure 43.70 Reception flow in slave mode

Master Reception-only processing flow

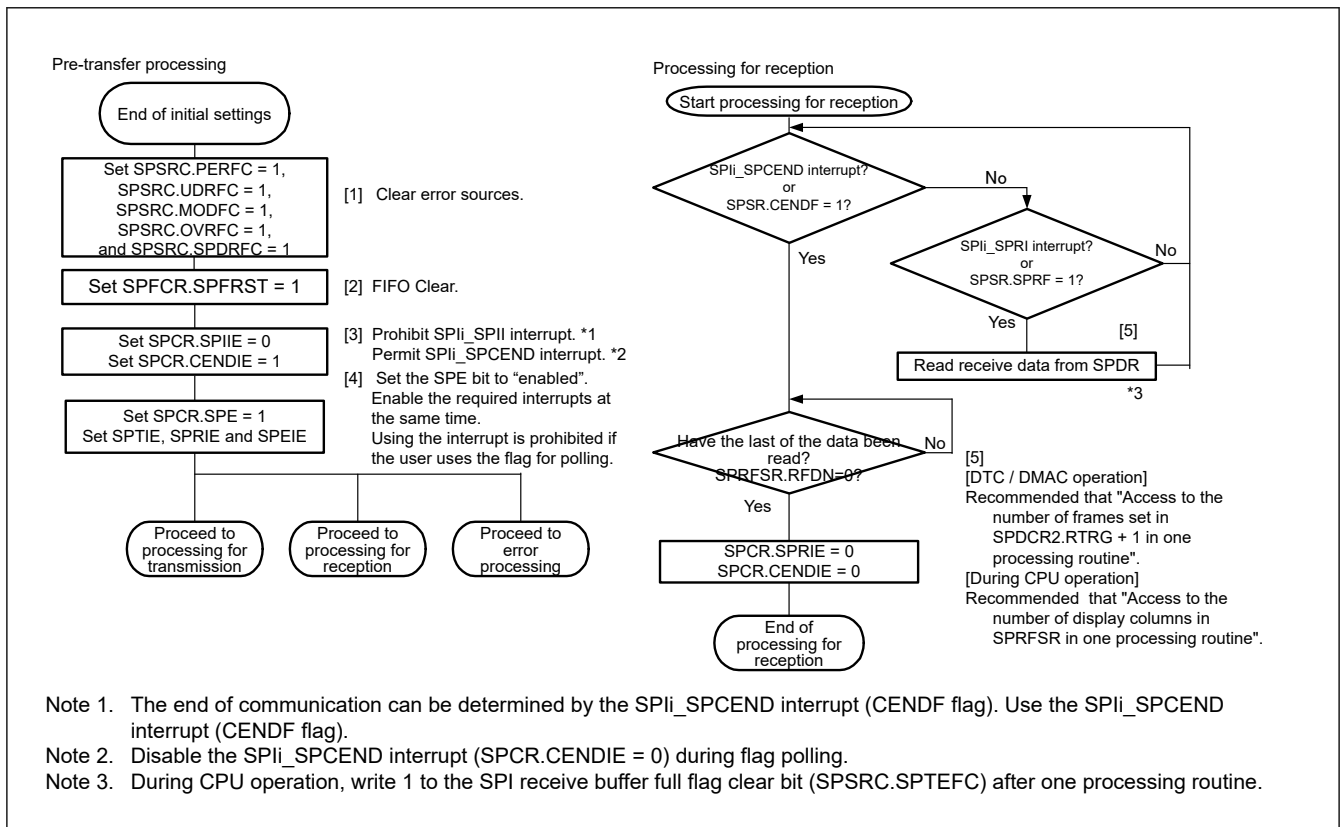


Figure 43.71 Software Processing Flowchart in Master Mode (Reception-only)

Error processing flow

In slave mode operation, even when a mode fault error is generated, the SPSR.MODF flag can be cleared regardless of the state of the SSLn0 pin.

When an error is detected by using an interrupt, clear the ICU.IELSRn.IR flag in the error processing routine. If this is not done, the ICU.IELSRn.IR flag might continue to indicate the SPIi_SPTI or SPIi_SPRI interrupt request. If the SPIi_SPRI interrupt request is indicated, read the receive buffer and initialize the sequencer in the SPI.

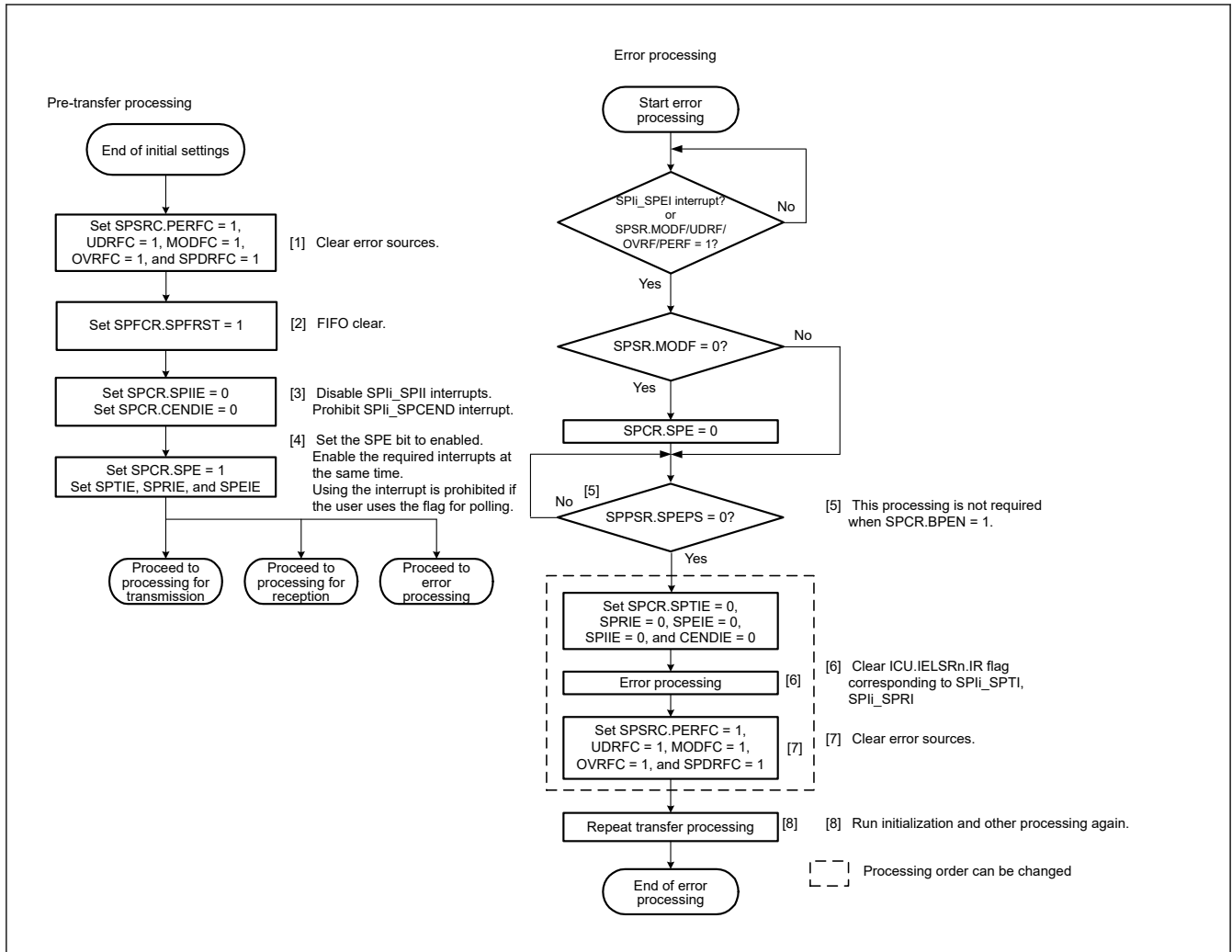


Figure 43.72 Error processing flow for slave mode

43.3.13 Clock Synchronous Operation

Setting the SPCR.SPMS bit to 1 selects clock synchronous operation of the SPI. In clock synchronous operation, the SSLni pin is not used, and the RSPCKn, MOSIn, and MISO pins handle communications. All SSLni pins are available as I/O port pins.

Although clock synchronous operation does not require the use of the SSLni pin, operation of the module is the same as in SPI operation. In both master mode and slave mode operations, communications can be performed with the same flow as in SPI operation. However, mode fault errors are not detected, because the SSLni pin is not used.

Additionally, do not perform operation if clock synchronous operation is enabled when the SPCMDm.CPHA bit is set to 0 in slave mode (SPCR.MSTR = 0).

43.3.13.1 Master Mode Operation

(1) Starting serial transfer

When data is written to the SPI data register (SPDR) while the next transfer data is not set in the transmit FIFO, the SPI updates the transmit buffer (SPTXn, n = 0 to 3) data in SPDR. While the shift register is empty, the SPI copies transmit buffer data to the shift register to start serial transfer. After the SPI copies transmit data to the shift register, it changes the shift register status to full. Upon completion of serial transfer, the SPI changes the shift register status to empty. The shift register status cannot be monitored.

For details about the SPI transfer format, see [section 43.3.5. Transfer Formats](#). In clock synchronous operation, however, the SSLn0 output signal is not used for communication.

(2) Terminating serial transfer

The SPI terminates the serial transfer after transmitting an RSPCKn edge corresponding to the sampling timing. If the number of data stored in the receive FIFO < the number of FIFO stages, on termination of serial transfer, the SPI copies data from the shift register to the receive buffer of the SPI Data Register (SPDR).

The final sampling timing varies depending on the bit length of transfer data. In master mode, the SPI data length depends on the SPCMDm.SPB[4:0] bits setting. Transfer in clock synchronous operation is conducted without the SSLn0 output signal. For details on the SPI transfer format, see [section 43.3.5. Transfer Formats](#).

(3) Sequence control

The transfer format used in master mode is determined by the SPCR3, SPCMDm, and SPDECR registers. Although the SSLn signals are not output in clock synchronous operation, these settings are valid.

The SPCR3.SPSSLN[2:0] bits determine the sequence configuration for serial transfers that are executed by the SPI in master mode. The following parameters are specified in the SPCMDm register:

- SSLn output signal value
- MSB or LSB first
- Data length
- Some of the bit rate settings
- RSPCKn polarity and phase
- Whether SPDECR.SCKDL is to be referenced
- Whether SPDECR.SLNDL is to be referenced
- Whether SPDECR.SPNDL is to be referenced

SPCR3.SPBR holds some of the bit rate settings such as SPDECR.SCKDL, an SPI clock delay value, SPDECR.SLNDL, an SSL negation delay, and SPDECR.SPNDL, a next-access delay value.

Based on the sequence length that is assigned to SPCR3, the SPI makes up a sequence comprised of a part or all of SPCMDm register. The SPI contains a pointer to the SPCMDm register that makes up the sequence. The value of this pointer can be checked by reading the SPDCR2.SPCP[2:0] bits. When the SPCR.SPE bit is set to 1 and the SPI function is enabled, the SPI loads the pointer to the commands in SPCMD0 register, and incorporates the SPCMD0 register setting into the transfer format at the beginning of serial transfer. The SPI increments the pointer each time the next-access delay period for a data transfer ends. On completion of the serial transfer that corresponds to the final command comprising the sequence, the SPI sets the pointer to the SPCMD0 register, and in this manner the sequence is executed repeatedly.

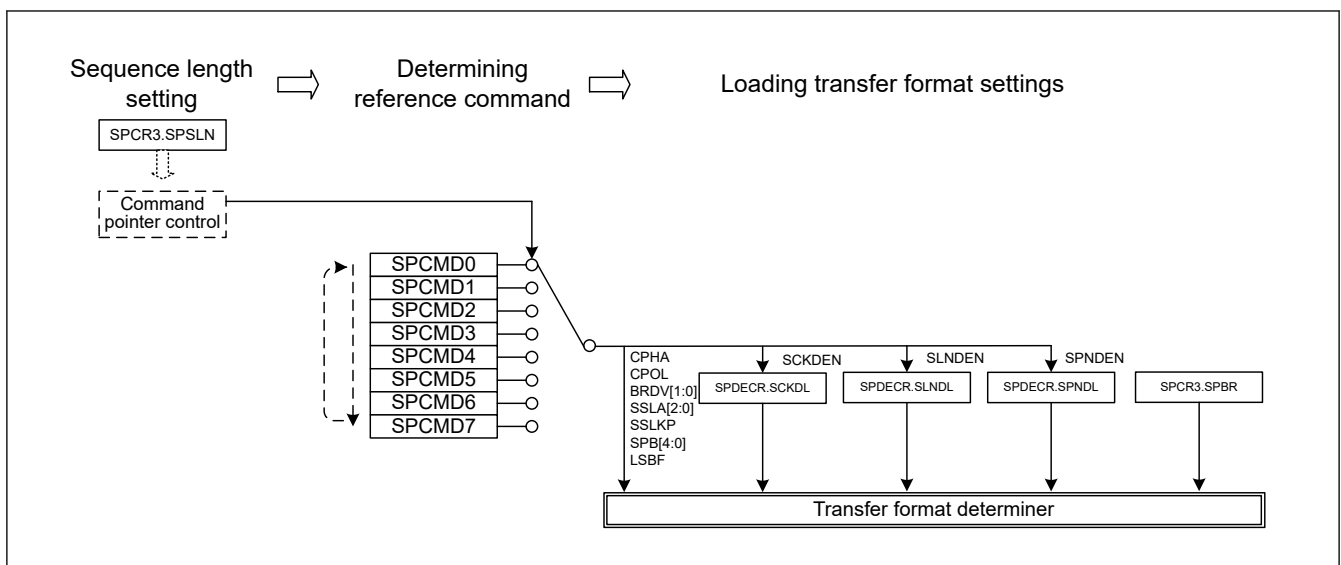


Figure 43.73 Procedure for determining the form of serial transmission in master mode

In this section, a frame is the combination of the data (SPDR) and the settings (SPCMDm).

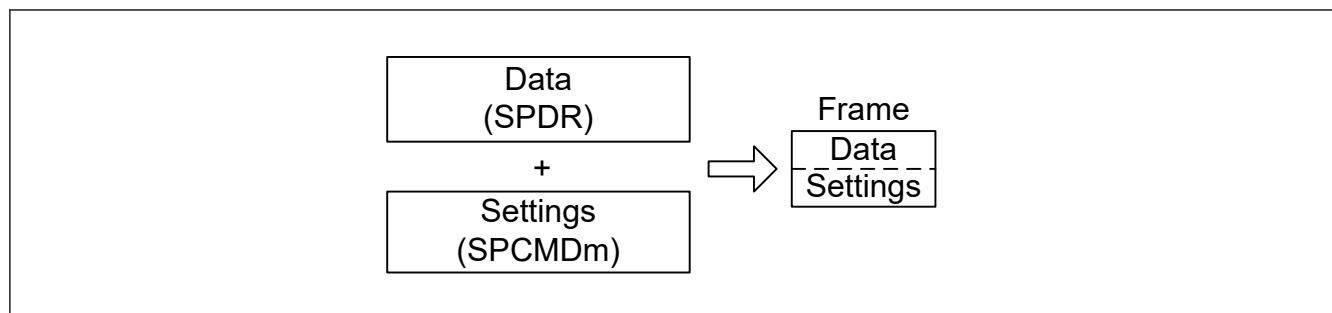


Figure 43.74 Conceptual diagram of frames

[Figure 43.75](#) shows the relationship between the command and the transmit and receive buffers in the sequence of operations specified by the settings.

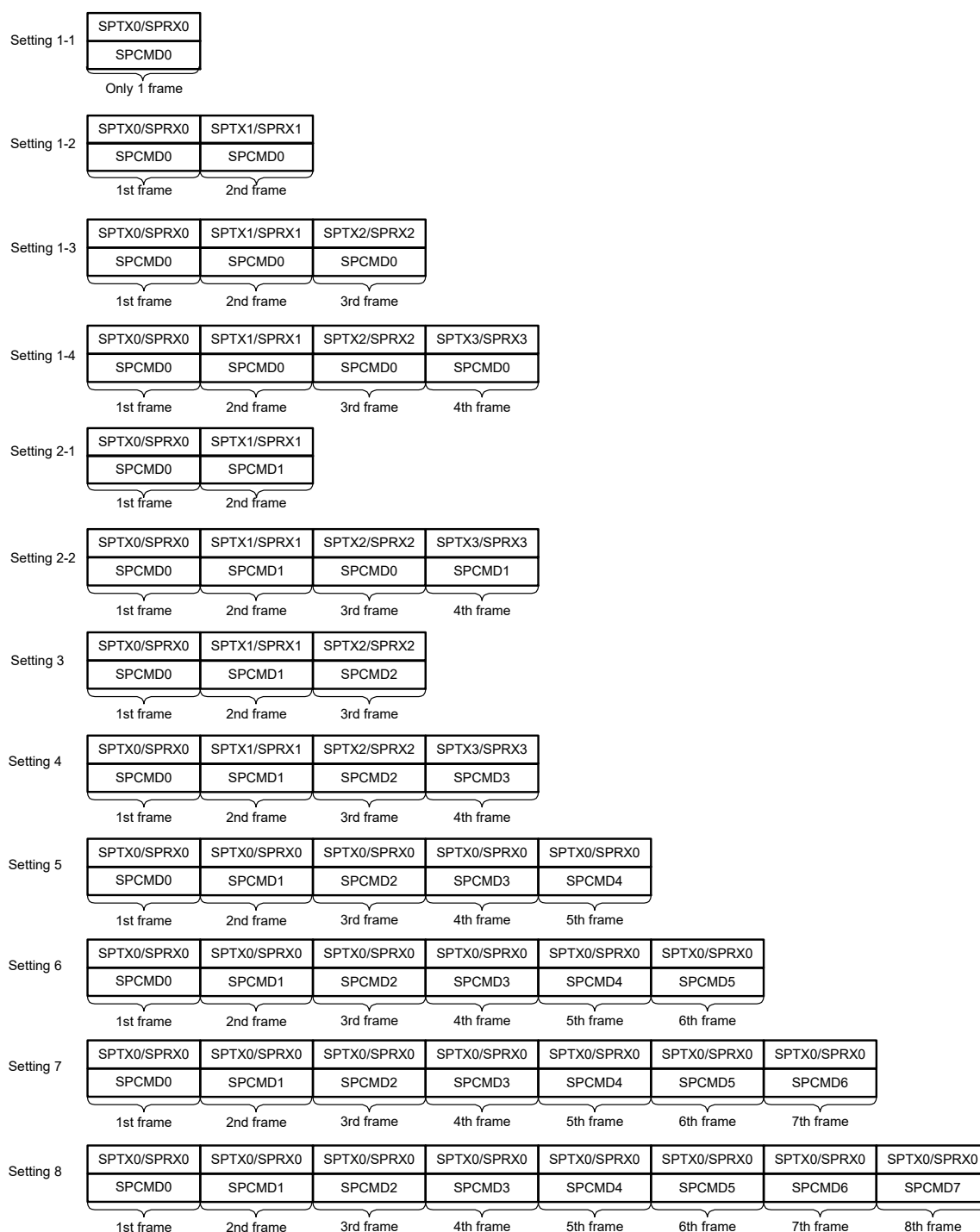


Figure 43.75 Correspondence between SPI Command Register and transmit and receive buffers in sequence operations

(4) Initialization flow

Figure 43.76 shows an example of initialization flow for clock synchronous operation when the SPI is used in master mode. For information on how to set up the ICU, DMAC or DTC, and I/O ports, see the descriptions given in the individual blocks.

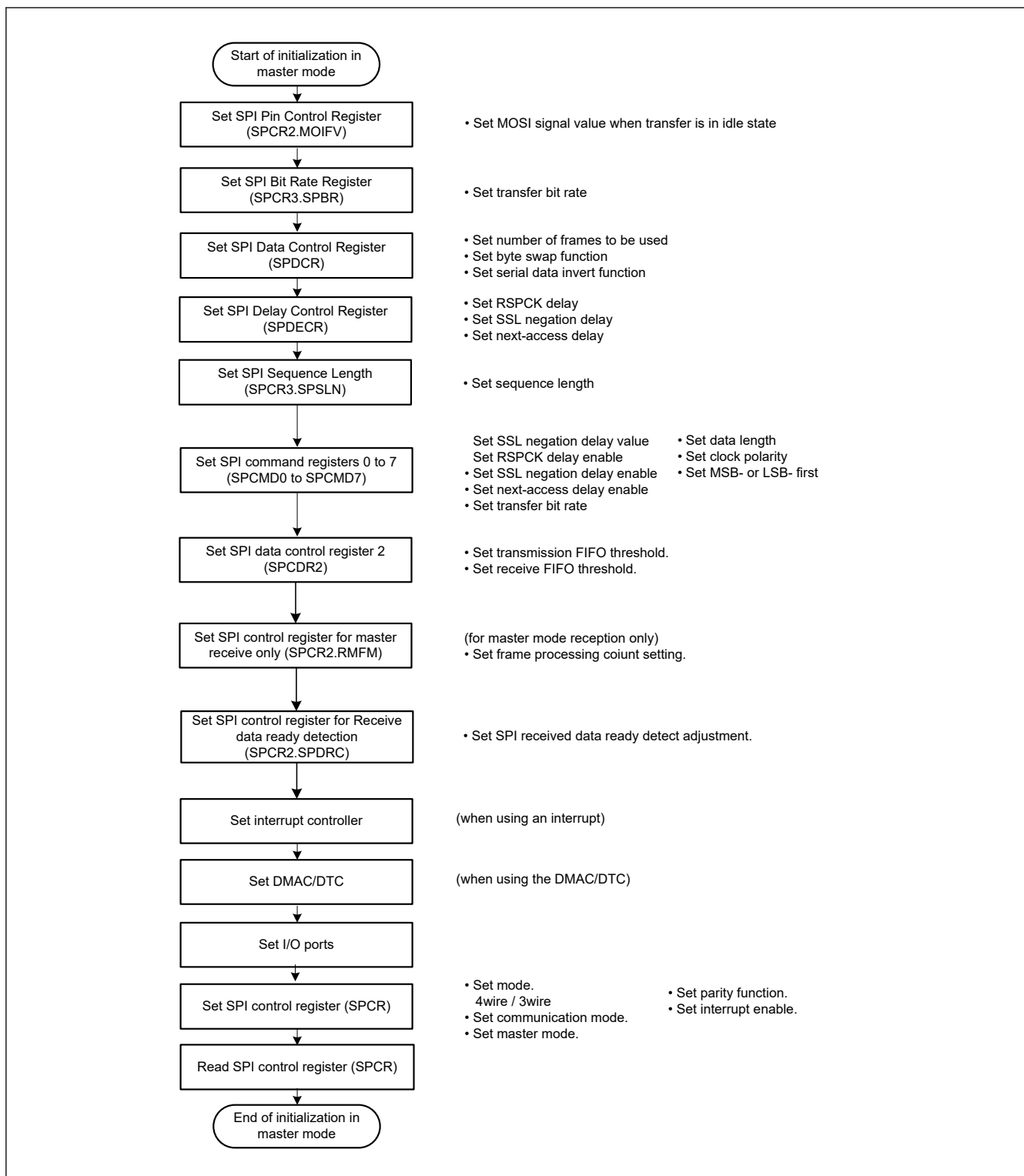


Figure 43.76 Example of initialization flow in master mode for clock synchronous operation

(5) Software processing flow

Software processing during clock synchronous master operation is the same as that for SPI master operation. For details, see (9) Software processing flow in [section 43.3.12.1. Master Mode Operation](#). Mode fault errors do not occur in clock synchronous operation.

43.3.13.2 Slave Mode Operation

(1) Starting serial transfer

When the SPCR.SPMS bit is 1, the first RSPCKn edge triggers the start of a serial transfer in the SPI, and the SPI drives the MISO_n output signal. The SSL_{n0} input signal is not used in clock synchronous operation. For details on the SPI transfer format, see [section 43.3.5. Transfer Formats](#).

(2) Terminating serial transfer

The SPI terminates the serial transfer after detecting an RSPCKn edge corresponding to the final sampling timing. When the number of data stored in the receive FIFO < the number of FIFO stages, on termination of serial transfer, the SPI copies received data from the shift register to the receive buffer of the SPDR register. On termination of a serial transfer, the SPI changes the status of the shift register to empty regardless of the receive buffer.

The final sampling timing changes depending on the bit length of transfer data. In slave mode, the SPI data length depends on the SPCMD0.SPB[4:0] bits setting. For details on the SPI transfer format, see [section 43.3.5. Transfer Formats](#).

(3) Initialization flow

[Figure 43.77](#) shows an example of initialization flow for clock synchronous operation when the SPI is used in slave mode. For a description of how to set up the ICU, DMAC or DTC, and I/O ports, see the descriptions given in the individual blocks.

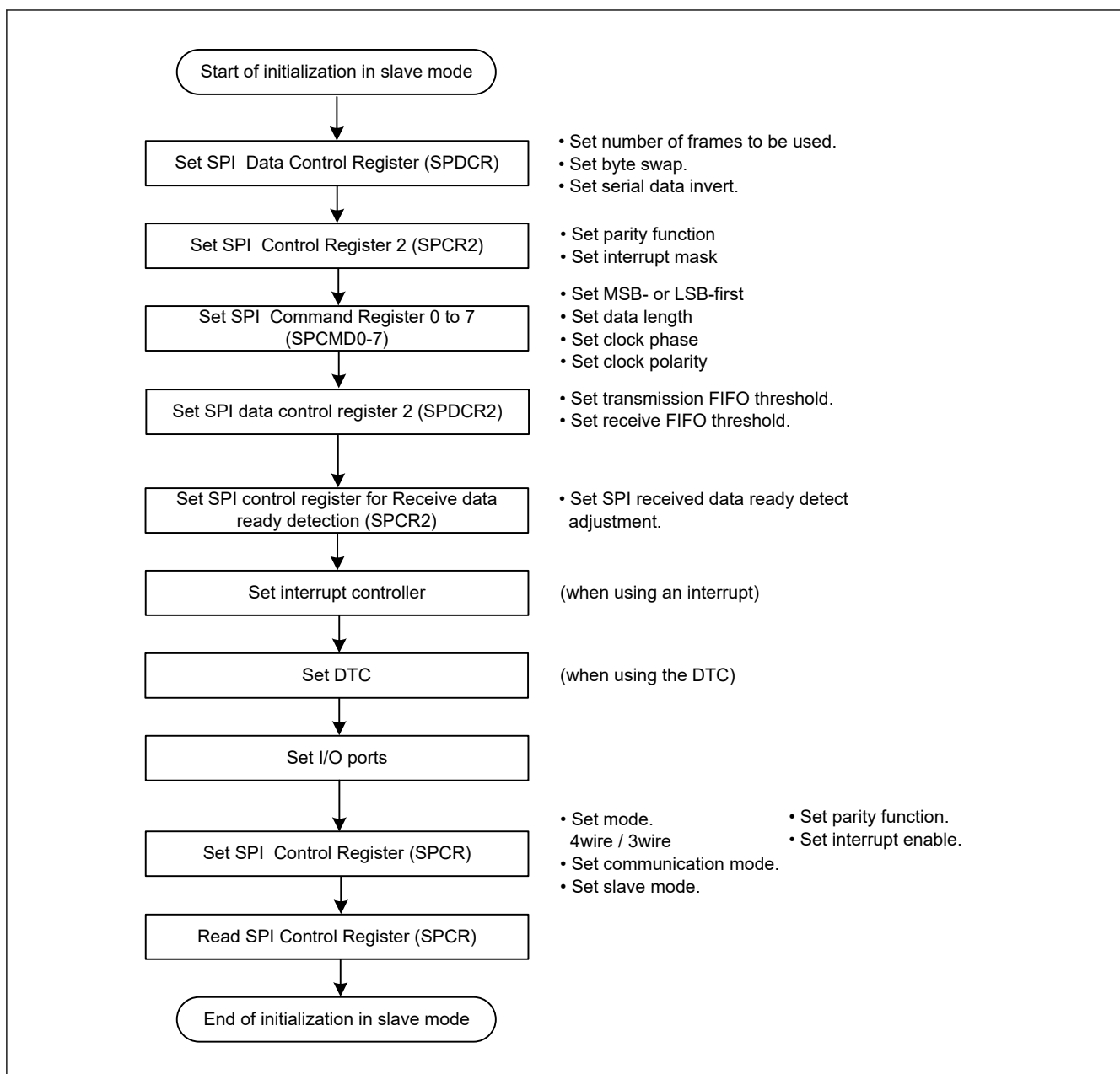


Figure 43.77 Example of initialization flow in slave mode for clock synchronous operation

(4) Software processing flow

Software processing during clock synchronous slave operation is the same as that for SPI slave operation. For details, see [\(6\) Software processing flow](#). Mode fault errors do not occur in clock synchronous mode.

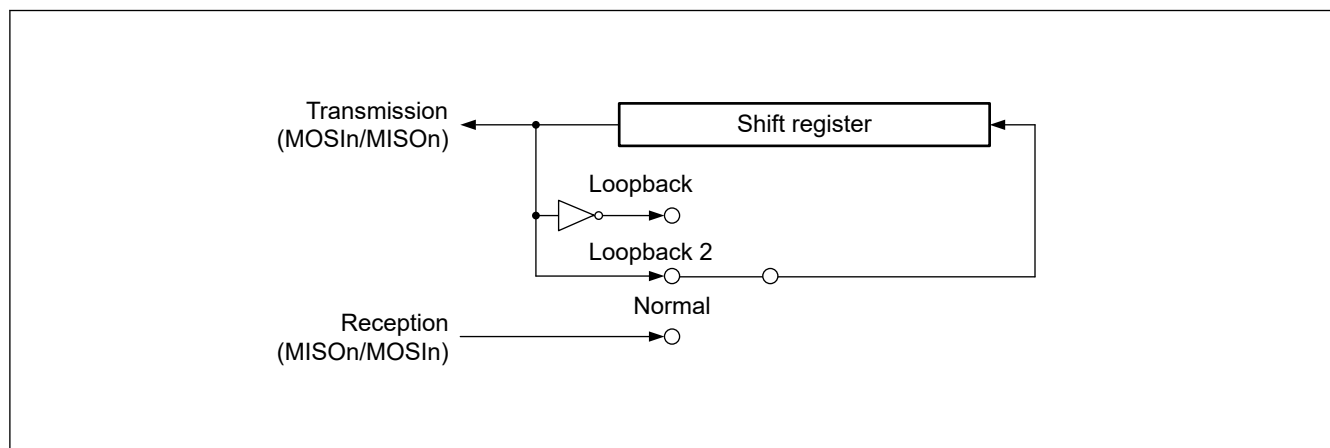
43.3.14 Loopback Mode

When 1 is written to the SPCR2.SPLP2 bit or SPCR2.SPLP bit, the SPI shuts off the path between the MISOn pin and the shift register if the SPCR.MSTR bit is 1, or between the MOSIn pin and the shift register if the SPCR.MSTR bit is 0, and connects the input and output paths of the shift register, establishing a loopback mode. The SPI does not shut off the path between the MOSIn pin and the shift register if the SPCR.MSTR bit is 1, or between the MISOn pin and the shift register if the SPCR.MSTR bit is 0. This is called loopback mode. When a serial transfer is executed in loopback mode, the transmit data for the SPI or the reversed transmit data becomes the received data for the SPI.

[Table 43.13](#) lists the relationship between the SPLP2 and SPLP bits and the received data. [Figure 43.78](#) shows the configuration of the shift register I/O paths when the SPI in master mode is set to loopback mode (SPCR2.SPLP2 = 0, SPCR2.SPLP = 1).

Table 43.13 SPLP2 and SPLP bit settings and received data

SPCR2.SPLP2 bit	SPCR2.SPLP bit	Received data
0	0	Input data from the MOSIn pin or MISOn pin
0	1	Inverted transmit data
1	0	Transmit data
1	1	Transmit data

**Figure 43.78** Configuration of shift register I/O paths in loopback mode for master mode

43.3.15 Self-Diagnosis of Parity Bit Function

The parity circuit consists of a parity bit adding unit used for transmit data and an error detecting unit used for received data. To detect defects in the parity bit adding unit and error detecting unit, the parity circuit performs self-diagnosis as shown in [Figure 43.79](#).

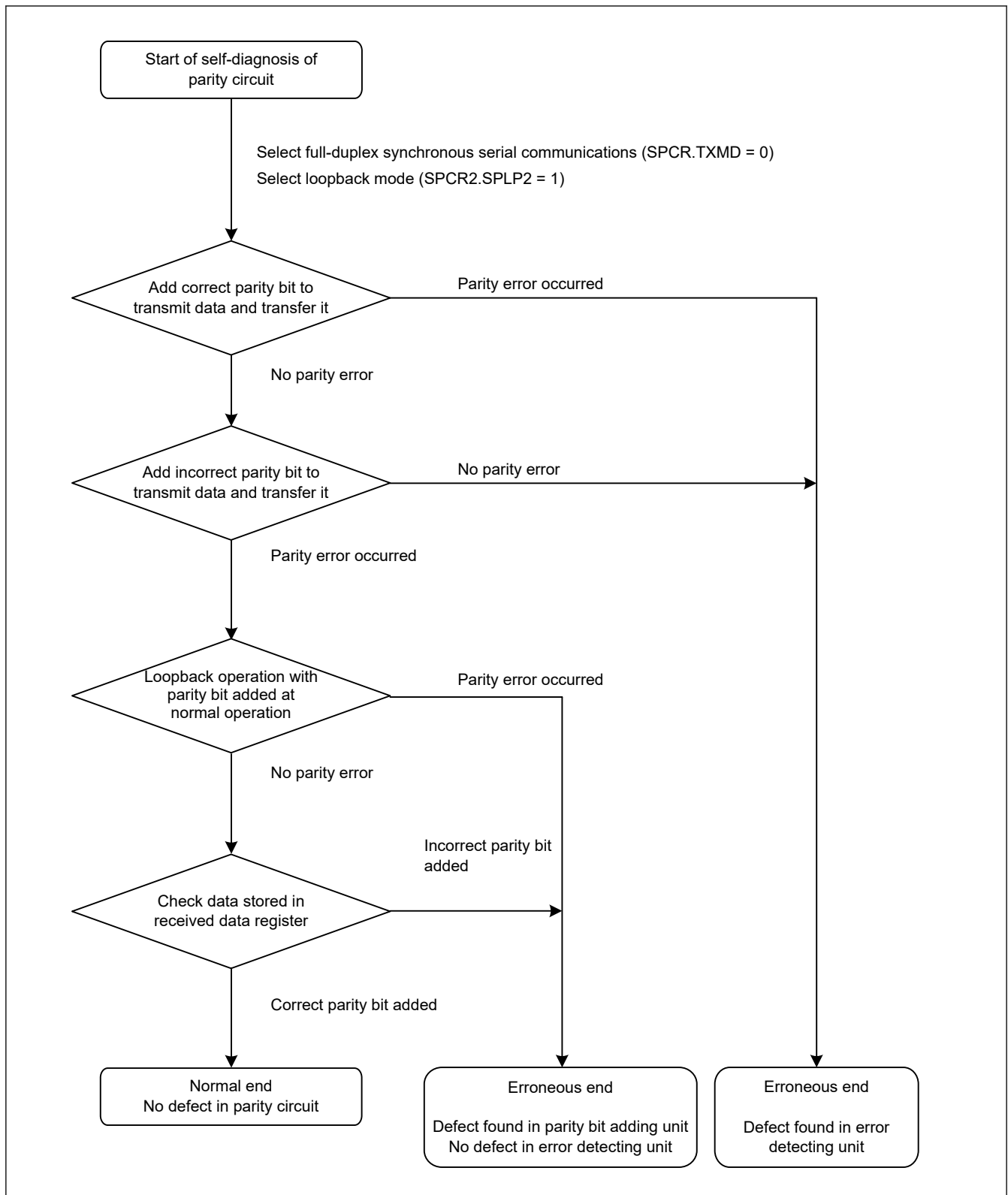


Figure 43.79 Self-diagnosis flow for parity circuit

43.3.16 Interrupt Sources

The SPI has the following interrupt sources:

- Receive buffer full
- Transmit buffer empty

- SPI error (mode-fault, underrun, overrun, or parity error)
- SPI idle
- Communication-end

The DMAC or DTC can be activated by the receive buffer full or transmit buffer empty interrupt to perform data transfer.

Because the vector address for the SPIi_SPEI (SPI error interrupt) is allocated to interrupt requests on mode-fault, underrun, overrun, and parity errors, the actual interrupt source must be determined from the flags. Interrupt sources for the SPI are listed in Table 43.14. An interrupt is generated on satisfaction of one of the interrupt conditions in Table 43.14. Clear the receive buffer full and transmit buffer empty sources through a data transfer.

When using the DMAC or DTC to perform data transmission and reception, you must first set up the DMAC or DTC to be in a transfer-enabled status before setting the SPI. For information on setting up the DMAC or DTC, see [section 17, DMA Controller \(DMAC\)](#) and [section 18, Data Transfer Controller \(DTC\)](#).

If the conditions for generating a transmit buffer empty or receive buffer full interrupt occur while the ICU.IELSRn.IR flag is 1, the interrupt is not output as a request for the ICU but is retained internally (the capacity for retention is one request per source). A retained interrupt request is output when the ICU.IELSRn.IR flag becomes 0. A retained interrupt request is automatically discarded when it is output as an actual interrupt request. The interrupt enable bit (the SPCR.SPTIE or SPCR.SPRIE bit) for an internally retained interrupt request can also be set to 0.

Table 43.14 SPI interrupt sources

Interrupt source	Symbol	Interrupt condition	DTC/DMAC activation
Receive buffer full	SPIi_SPRI	The receive buffer becomes full (SPSR.SPRF flag is 1) while the SPCR.SPRIE bit is 1 or The receive data become ready (SPSR.SPDRF flag is 1) while the SPCR.SPDRF bit is 0	Possible
Transmit buffer empty	SPIi_SPTI	The transmit buffer becomes empty (SPSR.SPTEF flag is 1) while the SPCR.SPTIE bit is 1	Possible
SPI error (mode-fault, underrun, overrun, or parity error)	SPIi_SPEI	The SPSR.MODF, OVRF, or PERF flag sets to 1, or the SPSR.SPDRF and SPDRF flag set to 1 while the SPCR.SPEIE bit is 1	Impossible
SPI idle	SPIi_SPII	The SPSR.IDLNF flag sets to 0 while the SPCR.SPIIE bit is 1	Impossible
Communication-end	SPIi_SPCEND	CENDIE = 1 and CENDF = 1	Impossible

43.4 Event Link Controller Event Output

The Event Link Controller (ELC) can produce the following event output signals:

- Receive buffer full event output
- Transmit buffer empty event output
- Mode-fault, underrun, overrun, or parity error event output
- SPI idle event output
- Transmission-completed event output

The event link output signal is output regardless of the interrupt enable bit setting.

43.4.1 Receive Buffer Full Event Output

When the number of data stored in the receive FIFO > the threshold value, or when the number of data stored in the receive FIFO ≤ the threshold value and SPDRES = 0 has elapsed after writing to the receive FIFO, and the SPDRC [7:0] has elapsed outputs an event.

43.4.2 Transmit Buffer Empty Event Output

An event is output when the number of empty transmission FIFO stage > the threshold or when the SPCR.SPE bit changes from 0 to 1.

43.4.3 Mode-Fault, Underrun, Overrun, Parity Error, or Received Data Ready Event Output

This event signal is output when mode-fault, underrun, overrun, or parity error is detected. See [section 43.5.4. Constraints on Mode-Fault, Underrun, Overrun, Parity Error, or Receive Data Ready Event Output](#) if using this event signal.

(1) Mode-fault

[Table 43.15](#) lists the conditions for occurrence of a mode-fault event.

Table 43.15 Conditions for mode-fault occurrence

SPI mode	SPCR.MODFEN bit	SSLn0 pin	Remarks
SPI operation (SPMS = 0) Slave (SPCR.MSTR = 0) Motorola-SPI (SPCR.SPFRF = 0)	1	Not active	Event is output only when the SSLn0 pin is deactivated during transmission
SPI operation (SPMS = 0) Slave (SPCR.MSTR = 0) TI-SSP (SPCR.SPFRF = 1)	1	active	Event is output only when the SSLn0 pin is activated during transmission

(2) Underrun

This event signal is output in response to an underrun when a serial transfer starts while the transmission data is not ready, and the value of the SPCR.MSTR bit is 0 and the SPCR.SPE bit is 1. Under these conditions, the MODF and UDRF flags are set to 1.

(3) Overrun

This event signal is output in response to an overrun when a serial transfer completes while the receive buffer contains unread data and the value of the SPCR.TXMD[1:0] bits are 00b or 10b. Under these conditions, the OVRF flag is set to 1.

(4) Parity error

This event signal is output in response to a parity error detected on completion of a serial transfer while the value of the SPPE bit in SPCR is 1.

(5) Receive Data Ready

When TXMD[1:0] of SPCR = 00b or 10b and SPDRES = 1 as the receive data ready event output condition, the number of data stored in the receive FIFO is received after writing the receive FIFO. An event will be output when the set value of SPDR[7:0] has elapsed while the number is less than the FIFO threshold.

43.4.4 SPI Idle Event Output

(1) In master mode

In Transmit-Receive / Transmit-only master mode, an event is output when the IDLNF flag in SPSR changes from 1 to 0. The IDLNF flag changes from 1 to 0 only when either of the conditions 1) and 2) below is met.

- The SPE bit in SPCR is cleared to 0 (SPI initialized) during transmission.
- All of the following three conditions are met.
 - The transmit buffer (SPTXn, n = 0 to 3) is empty (next transfer data has not been set).
 - The SPCP [2:0] bits in SPSR are 000b (at the start of sequence control).
 - Operation completed by the next access delay (when the master main state machine transitions to the idle state).

In receive only master mode

Any of the following 2 conditions is met.

- SPE bit of SPCR is 0 (SPI initialization)
- When any of the following is met

- When RMFM [4:0] = 0x00, after writing 1 to RMEDTG, operation completed by the next access delay (when the master main state machine transitions to the idle state).
- When RMFM [4:0] $\neq$ 0x00, after writing 1 to RMEDTG, operation completed by the next access delay (when the master main state machine transitions to the idle state).
- When RMFM [4:0] $\neq$ 0x00, the SPI internal sequencer transitions to the idle state after operation completed by the next access delay (when the master main state machine transitions to the idle state).

(2) In slave mode

In slave mode, an event is output when the SPCR.SPE bit is set to 0 (SPI is initialized).

43.4.5 Communication End Event Output

In master mode, an event is output when the IDLNF flag (SPI idle flag) changes from 1 to 0. In slave mode, an event occurs with conditions shown in [Table 43.16](#) and [Table 43.17](#)

Table 43.16 Communication End Event Generating Conditions (transmit-receive/transmit slave mode)

	Transmit Buffer Status	Shift Register Status	Others
SPI operation (SPMS = 0, SPFRF = 0)	Empty	Empty	SSLn0 input is negated
SPI operation (SPMS = 0, SPFRF = 1)	Empty	Empty	SSL negation delay completed
Clock synchronous operation (SPMS = 1)	Empty	Empty	The last even edge of RSPCK of last data was detected (CPHA = 1)

Table 43.17 Communication End Event Generating Conditions (receive only slave mode)

	Others
SPI operation (SPMS = 0, SPFRF = 0)	After storing the frames corresponding to the SPFC setting value in the receive buffer, negate SSLn0 input.
SPI operation (SPMS = 0, SPFRF = 1)	After storing the frames corresponding to the SPFC setting value in the receive buffer, SSL negation delay completed
Clock synchronous operation (SPMS = 1)	RSPCK last even edge detection when receiving the last frame for the SPFC set value (CPHA = 1)

Regardless of master mode or slave mode, no event is output when 0 is written to the SPCR.SPE bit during transmission or when the SPCR.SPE bit is cleared due to a mode fault error or an underrun error.

A communication end event is output at the following timing. The communication end event output timing in master operation is omitted because it is output at the same timing as an idle event.

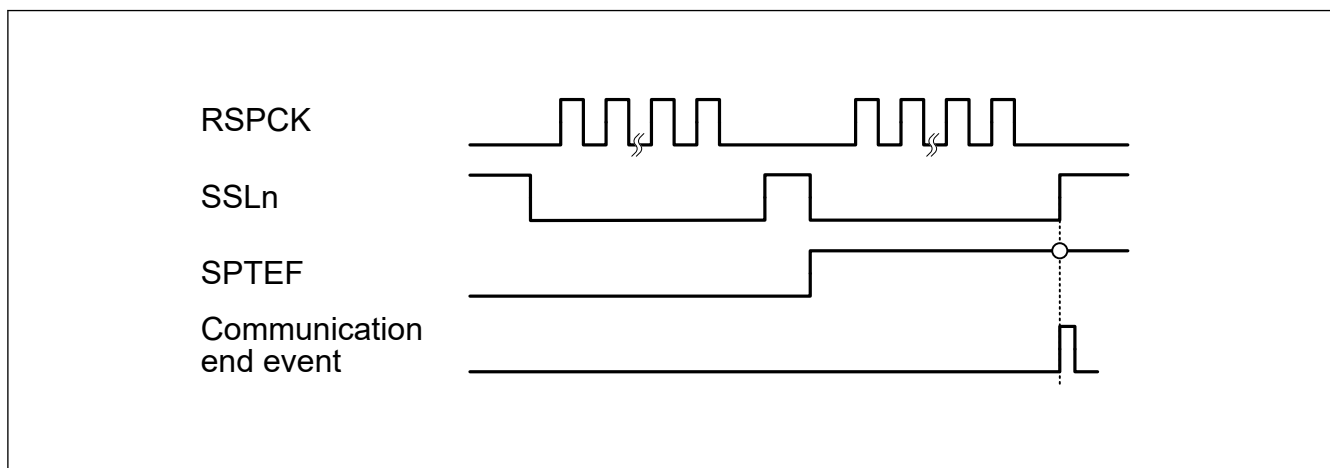


Figure 43.80 Communication End Event Output Timing (Transmit slave mode, Motorola SPI operation)

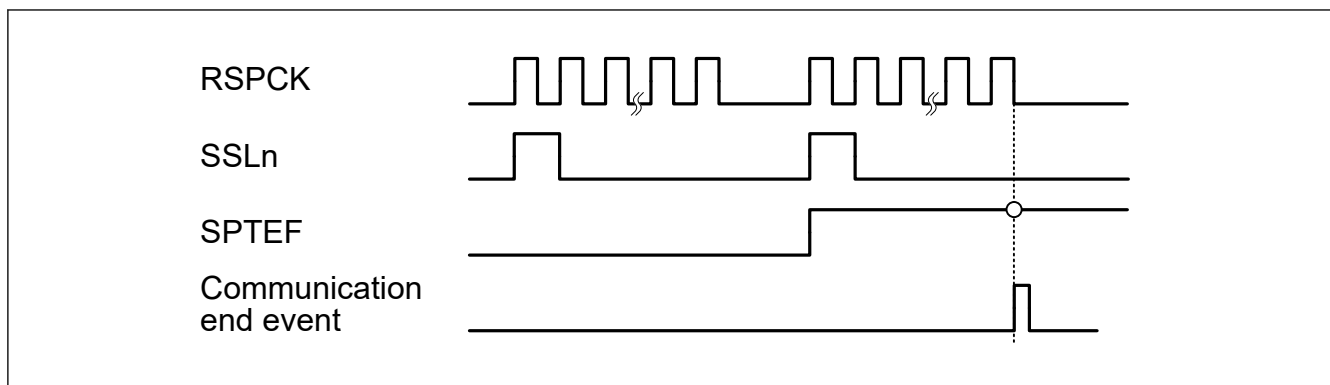


Figure 43.81 Communication End Event Output Timing (Transmit slave mode, TI-SSP Operation)

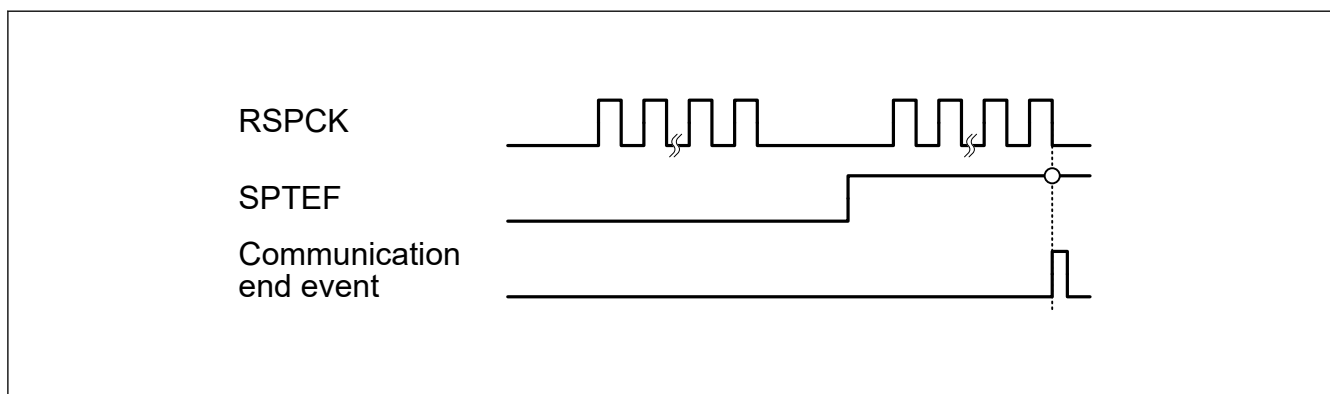


Figure 43.82 Communication End Event Output Timing (Transmit slave mode, Clock Synchronous Operation)

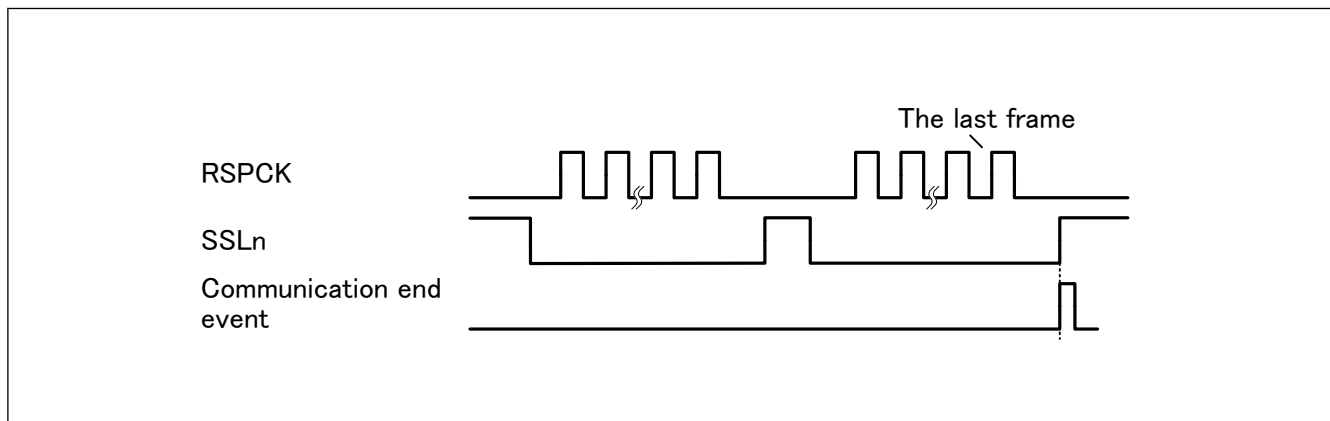


Figure 43.83 Communication End Event Output Timing (Receive only slave mode, Motorola SPI operation)

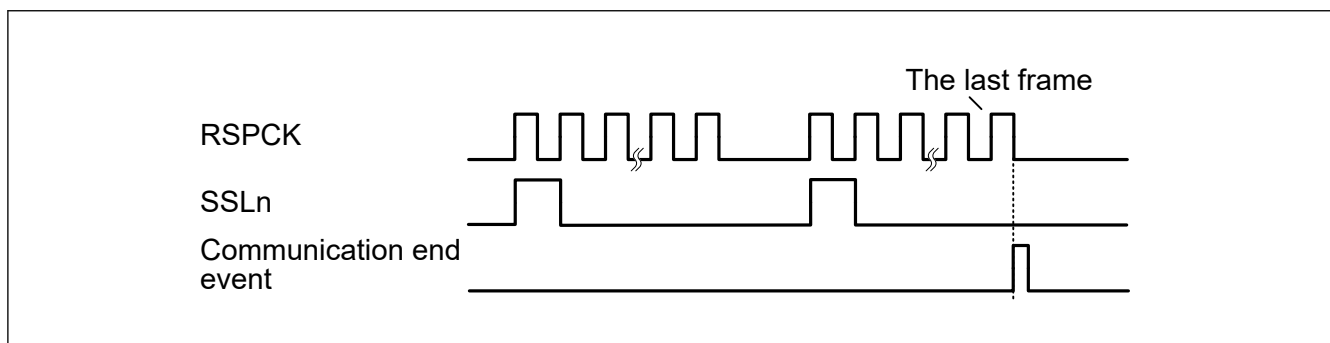


Figure 43.84 Communication End Event Output Timing (Receive only slave mode, TI-SSP Operation)

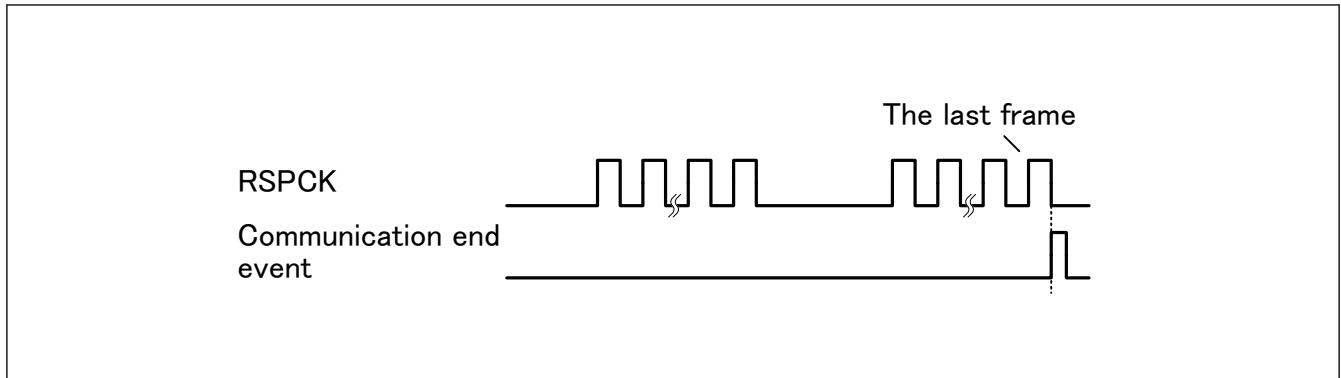


Figure 43.85 Communication End Event Output Timing (Receive only slave mode, Clock Synchronous Operation)

43.4.6 Synchronization Bypass Function

SPI has an internal clock (PCLK) and an operation clock (TCLK), and each has its own operation circuit. Therefore, a synchronization circuit is inserted between the signals between different clocks, and a signal delay between different clocks requires a synchronization delay time.

However, the synchronization circuit can be bypassed by the BPEN = 1 of the SPI control register (SPCR) only when the same clock is input as the internal bus clock and the operation clock. In this case, the synchronization delay time is excluded, and responsiveness is improved.

In addition, SPI has a synchronization circuit between the communication clock (RSPCK) and the operation clock (TCLK), but this synchronization circuit cannot be bypassed.

43.5 Usage Notes

43.5.1 Settings for the Module-Stop State

The Module Stop Control Register B (MSTPCRB) can enable or disable the SPI operation. The SPI is initially stopped after reset. Releasing the module-stop state enables access to the registers. For details on the Module Stop Control Register B, see [section 11, Low Power Mode](#).

43.5.2 Constraint on Low-Power Functions

When using the module-stop function and entering a low-power mode other than CPU Sleep mode or CPU Deep Sleep mode, set the SPCR.SPE bit to 0 before completing communication.

43.5.3 Constraints on Starting Transfer

If the ICU.IELSRn.IR flag is 1 when transfer starts, the interrupt request is internally retained, which can lead to unanticipated behavior of the ICU.IELSRn.IR flag.

To prevent this, use the following procedure to clear interrupt requests before enabling operations (by setting the SPCR.SPE bit to 1):

1. Confirm that transfer stopped (the SPCR.SPE bit is 0).
2. Set the associated interrupt enable bit (SPCR.SPTIE bit or SPCR.SPRIE bit) to 0.
3. Read the associated interrupt enable bit (SPCR.SPTIE bit or SPCR.SPRIE bit) and confirm that its value is 0.
4. Set the ICU.IELSRn.IR flag to 0.

43.5.4 Constraints on Mode-Fault, Underrun, Overrun, Parity Error, or Receive Data Ready Event Output

Using the mode-fault, underrun, overrun, parity error, or receive data ready event is prohibited if the SPI is in multi-master mode (when the SPCR.SPMS bit is 0, the SPCR.MSTR bit is 1, and the SPCR.MODFEN bit is 1).

43.5.5 Constraints on the SPSR.SPRF and SPSR.SPTEF Flags

If the polling flags, SPRF and SPTEF, are used, using the interrupts is prohibited, and you must set the SPCR.SPRIE and SPCR.SPTIE bits to 0. Either the interrupts or the flags can be used, but not both.